

BASICS OF COMPONENT DIAGRAM:

1. What is a Component?

- A component is a modular, replaceable, and self-contained part of a system.
- It encapsulates implementation details (state, behavior) but exposes a public interface for interaction.
- Components can be used in systems of any size, from small utilities to large enterprise systems.
- Unlike packages, components realize and require interfaces, making them swappable and reusable.

Example:

- Payment processing module in an e-commerce system.
- User authentication module.

2. Purpose of Component Diagrams

- To model the physical aspects of a system, such as files, modules, and libraries.
- Show relationships and dependencies between components.
- Support implementation-level design.
- Facilitate reuse, maintainability, and modular design.

3. Main Elements of Component Diagrams

1. Component
 - Represented by a rectangle with <<component>> stereotype.
 - Optional small icon in the top-right corner.
2. Interfaces
 - Provided Interface: Services offered by the component. Shown as a ball (lollipop).
 - Required Interface: Services expected from another component. Shown as a socket.
3. Ports
 - Small square on the component boundary representing interaction points.
 - Can be uni-directional or bi-directional.
 - May list multiple interfaces.
4. Dependencies
 - Dashed arrows showing that one component requires another.

- Example: `<<use>>` dependency.

5. Connectors

- Assembly Connector: Links a required interface to a provided interface (ball-and-socket).
- Delegation Connector: Connects internal parts of a component to the component's interfaces.

6. Artifacts

- Physical files used by components (e.g., `.exe`, `.dll`, `.java`, `.cpp`).

4. Component Views

- Black-Box (External) View:
 - Shows public interfaces only.
 - Encapsulates internal implementation.
- White-Box (Internal) View:
 - Shows realizing classes/components inside.
 - Includes internal connectors, parts, or artifacts.

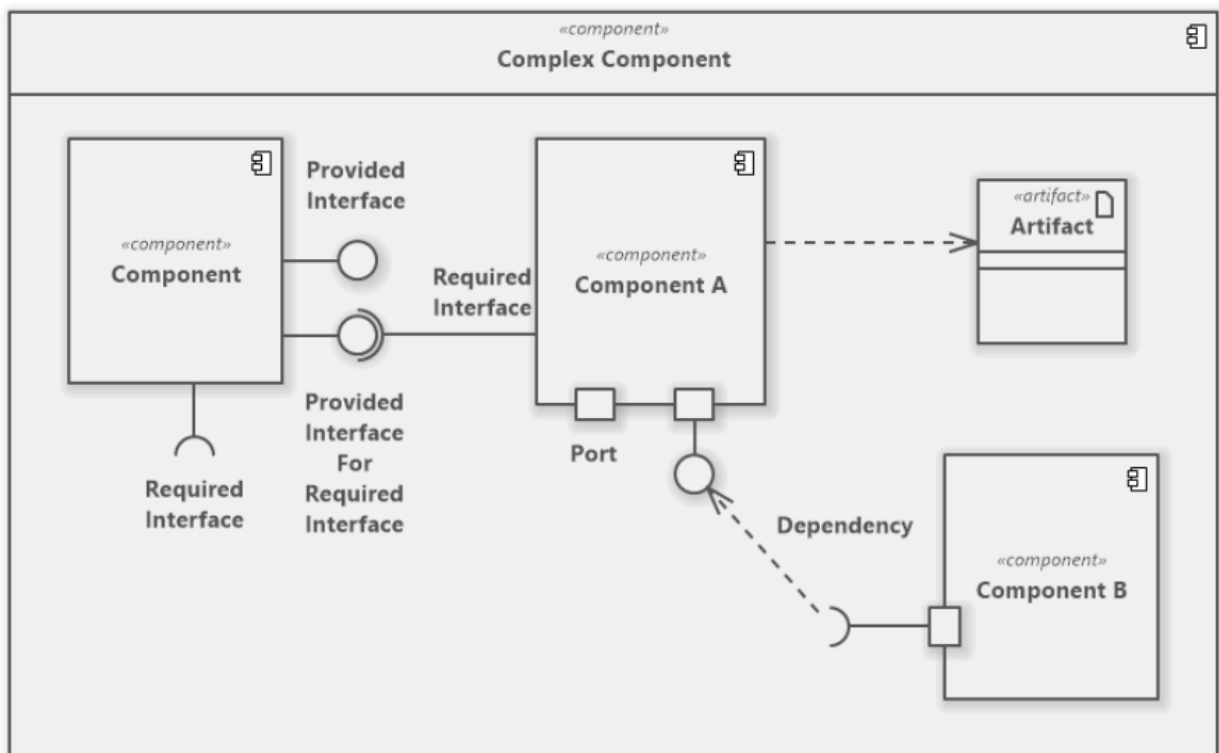
5. Common Component Stereotypes

Stereotype	Description
<code><<application>></code>	Front-end GUI or web pages
<code><<database>></code>	Database component
<code><<library>></code>	Object or function library
<code><<executable>></code>	Executable program
<code><<file>></code>	Data or source file

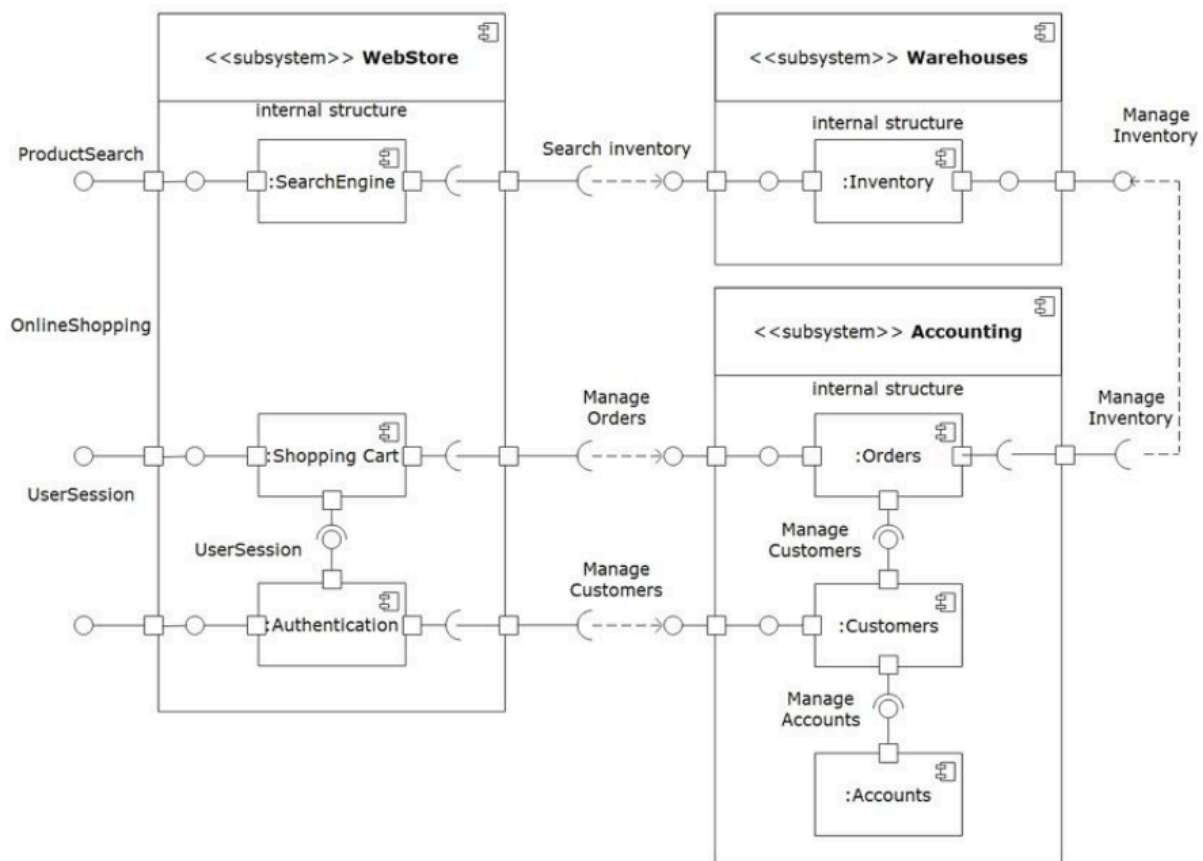
<<web service>>	Web service component
<<document>>	Documentation or data file

6. Key Points

- Components communicate through interfaces; the implementation is hidden.
- A component can be replaced if it provides and requires the same interfaces.
- Supports modular design, reusability, and system maintainability.
- Component diagrams are part of implementation diagrams in UML.



Online Shopping UML Component Diagram



1. Understanding the Notation

This image defines the "grammar" of the diagram you need for your exam.

- **Component:** The large boxes (e.g., Component A, Component B). They represent modular parts of the system.
- **Provided Interface (Lollipop):** A solid line ending in a full circle. This shows a service the component **outputs** or offers to others.
- **Required Interface (Socket):** A solid line ending in a semi-circle. This shows a service the component **needs** or consumes from elsewhere.
- **Assembly Connector:** When a lollipop fits into a socket, it represents a connection between two components.
- **Dependency (Dashed Line):** A dashed arrow represents a relationship where one element requires another to function or exist. In your diagram, Component B has a dependency on the interface of Component A.
- **Port:** The small squares on the edges of components. They act as distinct interaction points where interfaces are attached.

- **Artifact:** Represented as a box with a "folded paper" icon. These are physical files (like a database schema or an executable) that result from the development process.

2. ShopEase E-Commerce Logic

This image shows a complex real-world application of those symbols for an online store.

The WebStore Subsystem (Frontend/User Interface)

- **SearchEngine Component:** Provides the **ProductSearch** interface for users and requires the **Search inventory** interface from the backend.
- **Shopping Cart Component:** Provides the **OnlineShopping** interface and connects to the **Manage Orders** interface.
- **Authentication Component:** Provides **UserSession** and connects to **Manage Customers**.

The Backend Subsystems

- **Warehouses:** Contains the **Inventory** component. It provides the **Search inventory** interface used by the web store.
- **Accounting:** Contains components for **Orders**, **Customers**, and **Accounts**.
 - The **Orders** component provides the **Manage Orders** interface.
 - The **Customers** component provides **Manage Customers**.

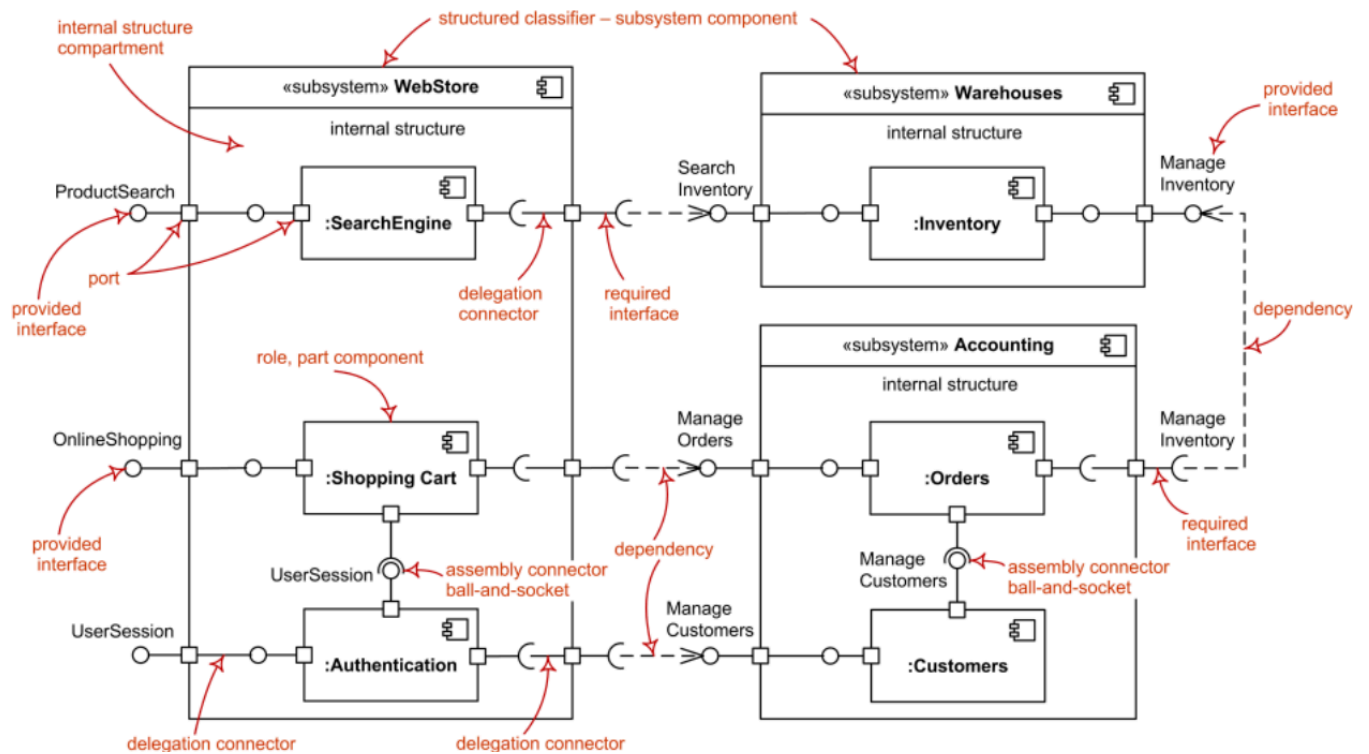
3. Cheat Sheet for Your Exam

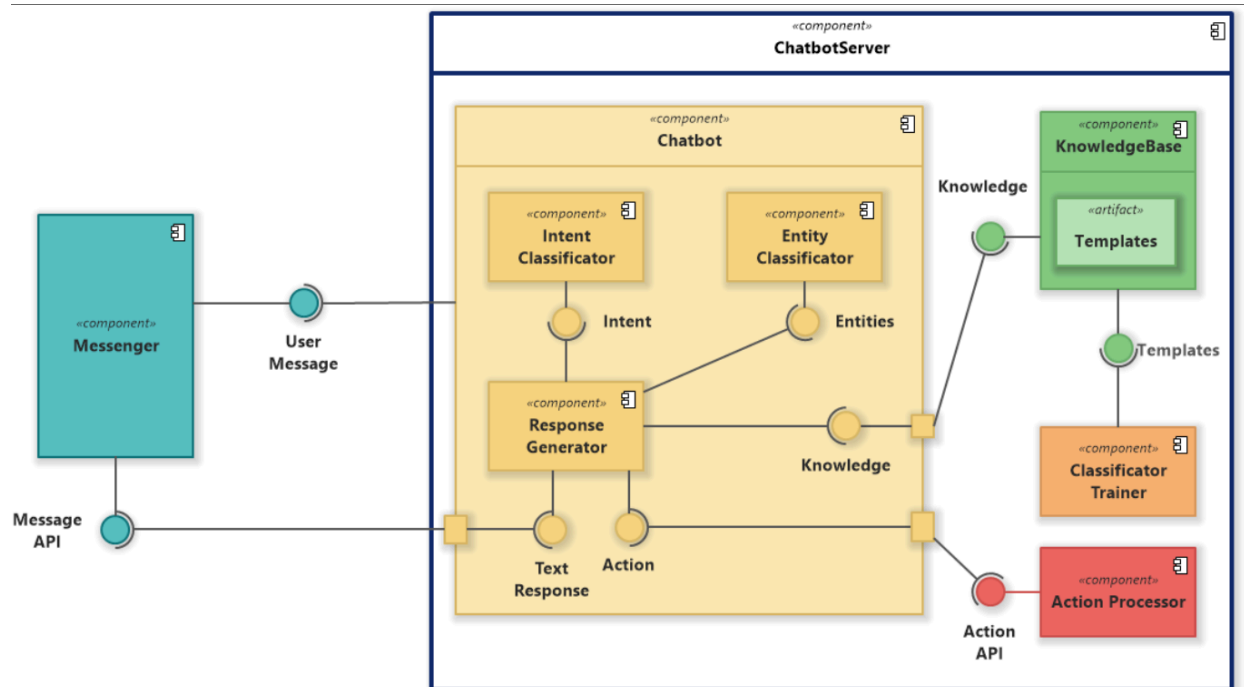
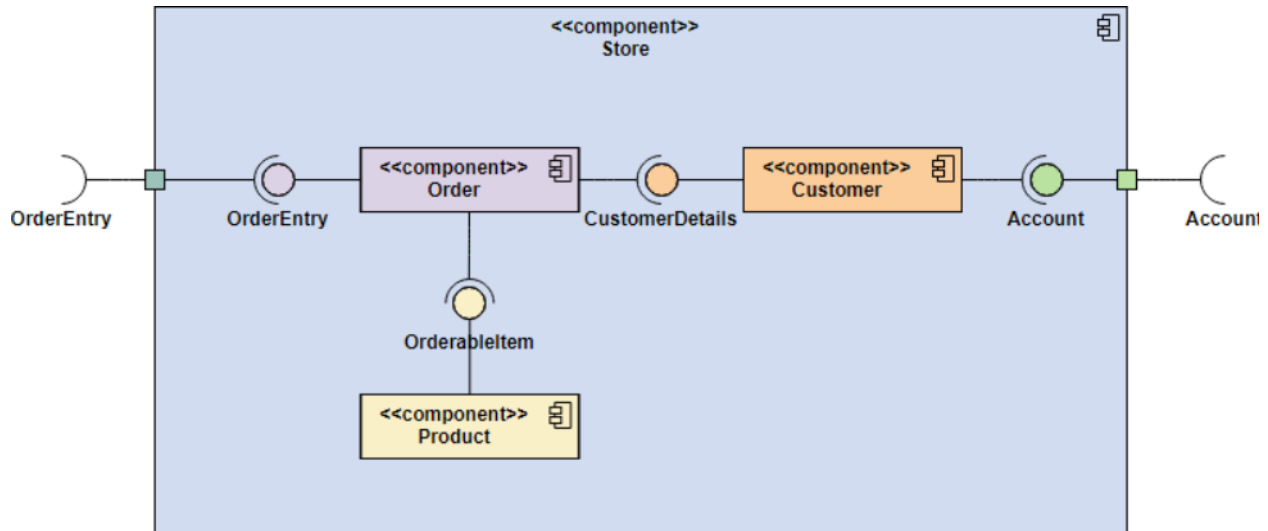
If you have to draw the **ShopEase** scenario, use this structure:

Component	Provided Interface (Lollipop)	Required Interface (Socket)
User Service	User Auth, Profile Management	Database Access
Product Service	Catalog Search, Update Product	Database Access
Cart Service	Cart Management	Product Data, Order Placement

Payment Service	Payment Authorization	External Gateway API (Dependency)
Order Service	Tracking, Returns	Payment Confirmation, Shipping API
Notification Service	Send Email/SMS	User Contact Info

Crucial Drawing Tip: Use **dashed arrows** for dependencies (like the Payment Service depending on a 3rd-party API) and **solid lines with lollipops/sockets** for internal system connections.

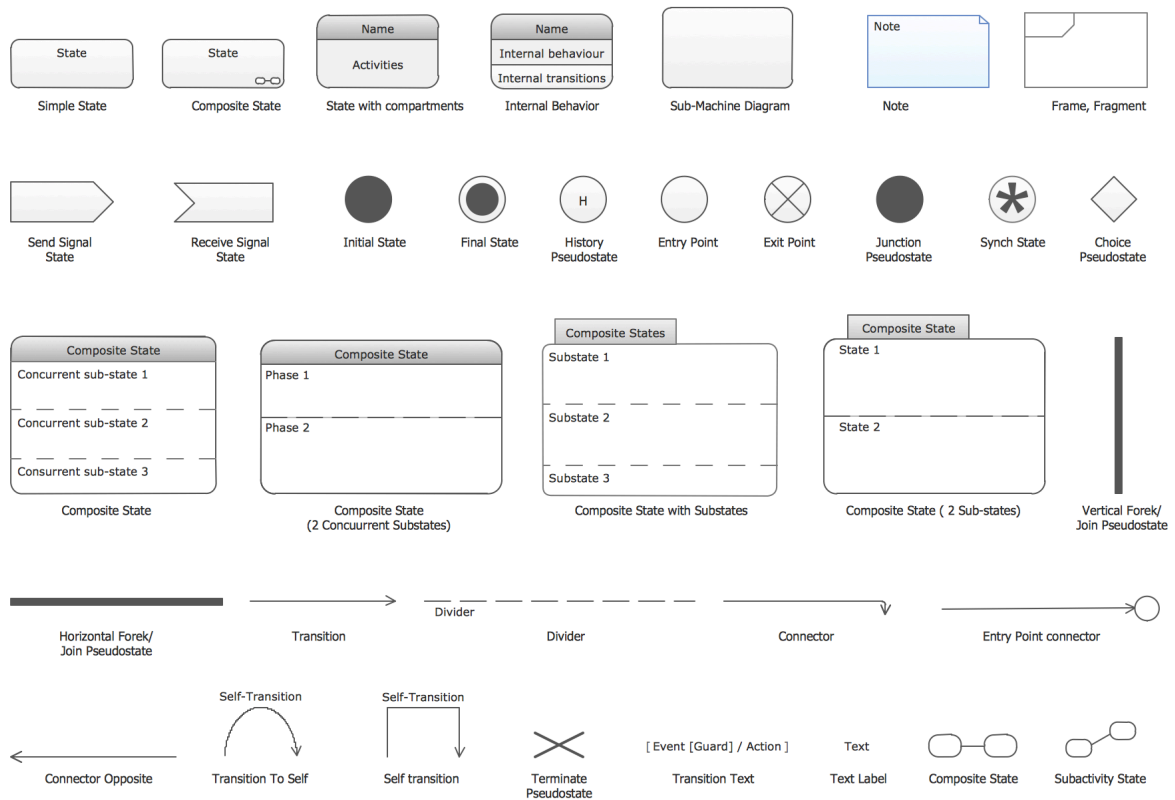




This diagram demonstrates a "white-box" view, showing internal realization within the ChatbotServer.

- **Messenger**: An external component that interacts with the server via the **User Message** and **Message API** interfaces.
- **Internal Communication**: The Chatbot component contains sub-components: the **Response Generator** requires an "Intent" from the **Intent Classifier** and "Entities" from the **Entity Classifier** to build a response.
- **Knowledge Integration**: The **Response Generator** also requires a **Knowledge** interface provided by the KnowledgeBase, which contains a physical **Templates** artifact.

STATE MACHINE DIAGRAM BASICS:



1. What is a State Machine Diagram?

A State Machine Diagram is a type of UML behavioral diagram that shows:

- How an object changes states over time in response to events.
- The lifecycle of an object.
- Transitions between states and the events that trigger them.

2. Core Concepts

Concept	Description	UML Notation
State	A condition during the life of an object where it satisfies some property, performs an activity, or waits for an event.	Rounded rectangle

Initial state	The starting point of the object's lifecycle.	Black filled circle [*]
Final state	End of the lifecycle.	Circle with a dot inside [*] with outer circle
Transition	Movement from one state to another triggered by an event.	Arrow from source to target state
Event	An occurrence that triggers a transition.	Named on the arrow: eventName
Guard condition	A Boolean condition that must be true for the transition to occur.	In brackets: [condition]
Action/Effect	Activity performed during the transition.	event / action

3. Composite States

- A state that contains other states (nested states).
- Useful for grouping behavior.
- Notation: A rounded rectangle containing sub-states.

Example: Washing {

FillingWater

Heating

}

4. Concurrency / Orthogonal Regions

- Composite states can have parallel regions, separated by a dashed line.

- Each region executes concurrently.
- Transition out of the composite state happens only when all regions reach final state.

Notation

```
state Washing {
    state WaterFilling { ... }
    --
    state Heating { ... }
}
```

Dashed line -- separates concurrent regions.

5. History States

- History state saves the last active substate of a composite state.
- Useful for resuming a process after a pause or interruption.

Notation

```
state H <<history>>
```

```
Idle --> H : powerRestored
```

6. Junction Pseudo-States

- Represents a decision point or merge of transitions.
- Helps model conditional branching inside state machines.

Notation

- Small filled circle with multiple outgoing/incoming arrows.

7. Fork and Join (Concurrency)

- Fork: Splits into multiple concurrent regions.
- Join: Waits for all concurrent regions to finish before continuing.

Notation

- Thick horizontal or vertical line connecting multiple arrows.

8. Guard Conditions and Actions

- Guard: `[condition]` → Boolean check before transition.
- Action/Effect: `/ actionName` → Activity during transition.

Example

`PaymentFailed --> PaymentInitiated : [retry < 3] / retryPayment()`

`PaymentFailed --> Cancelled : [retry >= 3]`

9. Example Simple State Machine: Washing Machine

`[*] --> Idle`

`Idle --> Washing : startButtonPressed`

`Washing --> Spinning : waterFilled && heated`

`Spinning --> Rinsing : spinComplete`

`Rinsing --> Done : rinseComplete`

`Done --> [*]`

- Shows initial state, states, transitions, guard conditions, and final state.

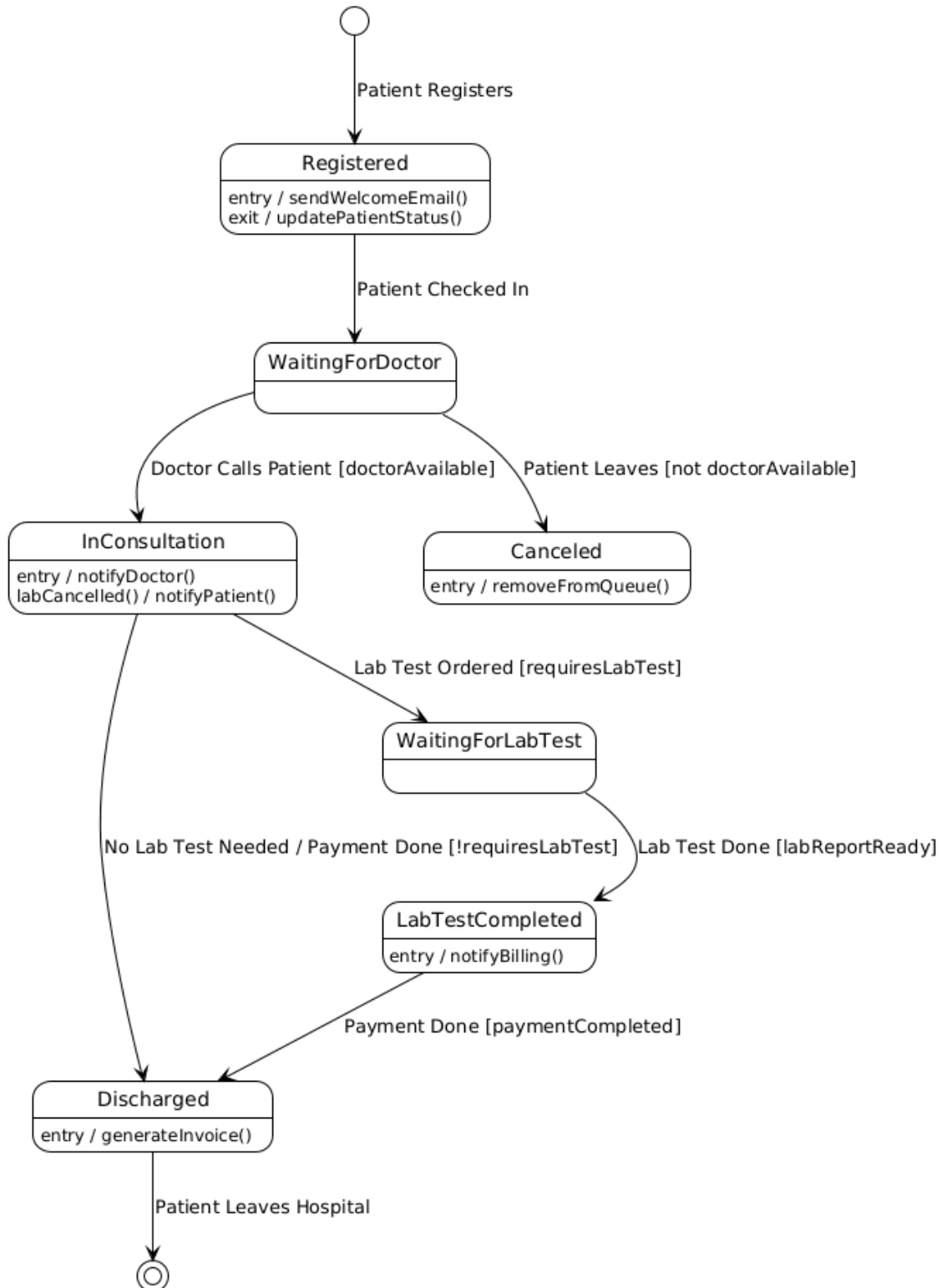
10. Summary

State Machine Diagrams help visualize behavior over time, especially for:

- Objects with distinct states.
- Concurrent activities.
- Resuming processes (history states).
- Decision points (junctions).
- Complex workflows (fork/join).

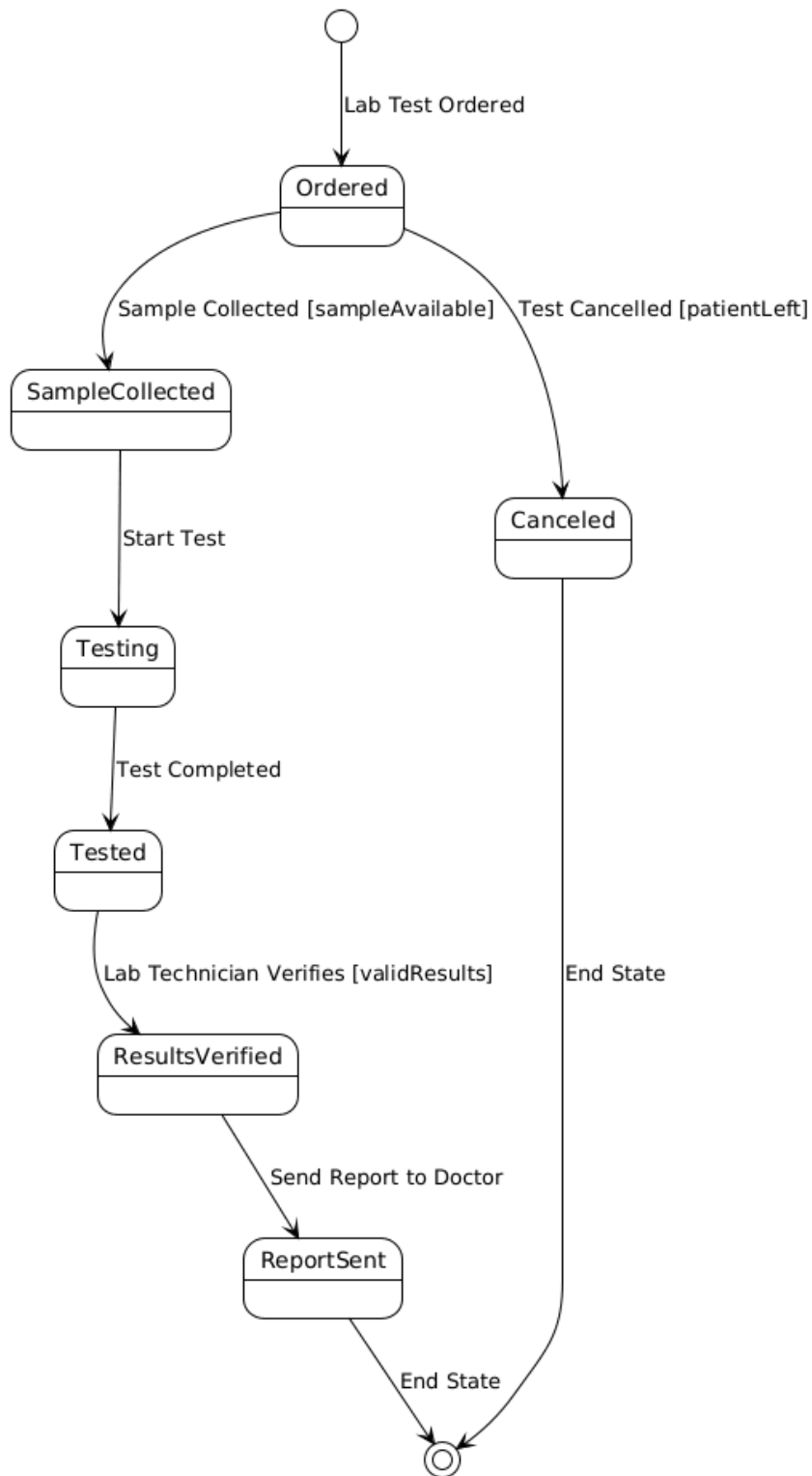
STATE MACHINE DIAG FOR PATIENT

Hospital Management - Patient Admission State Machine

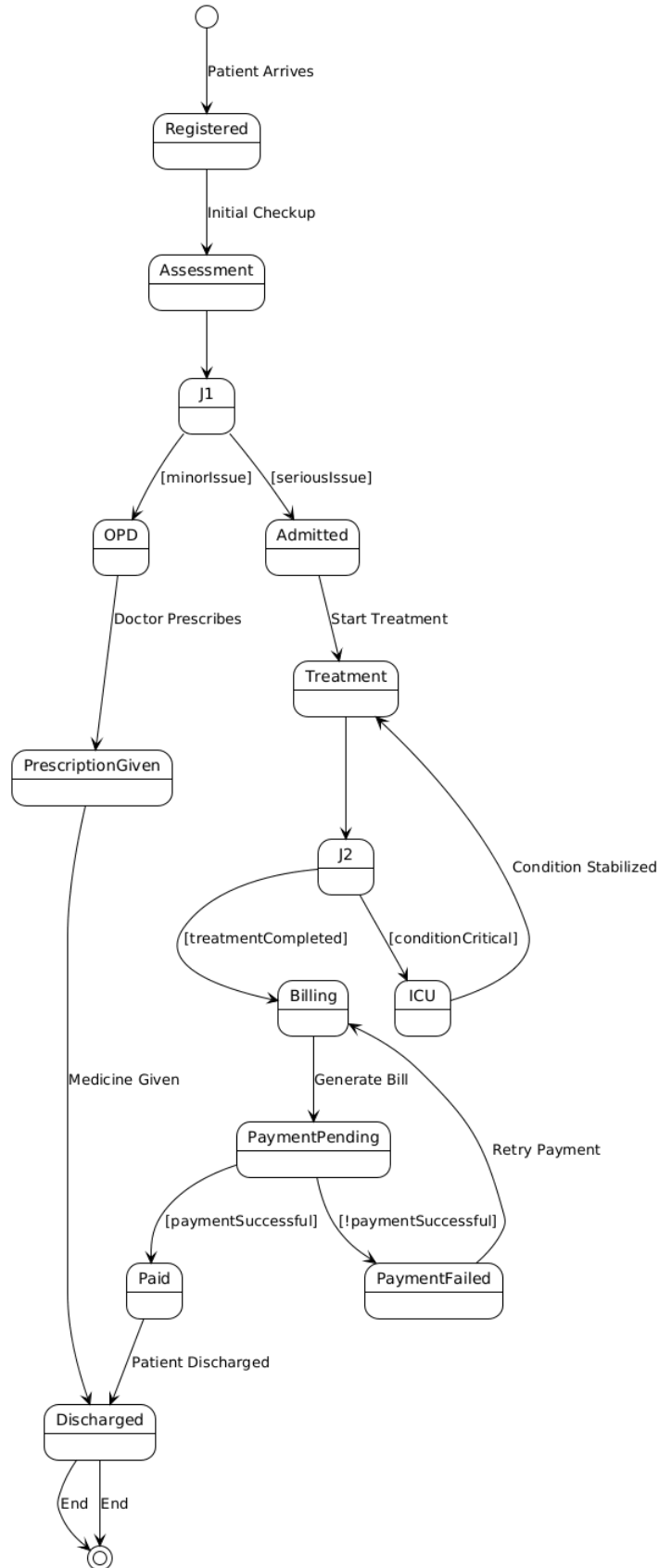


STATE MACHINE DIAGRAM FOR LAB TEST PROCESS:

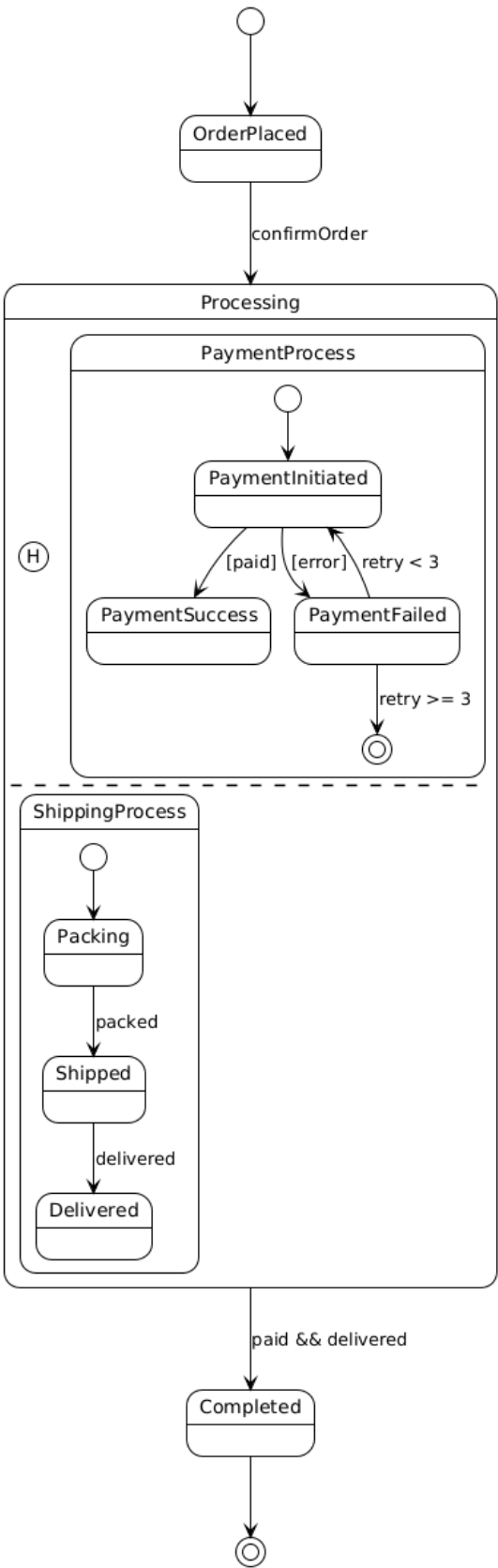
Lab Test Process - State Machine



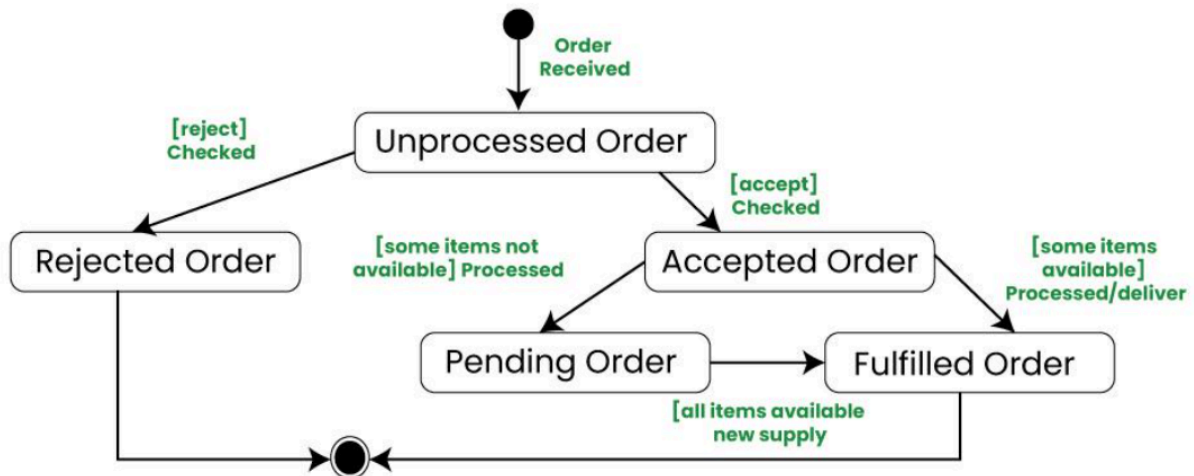
Patient Treatment State Machine (with Junction Pseudostates)



Online Order Processing - State Machine Diagram



State machine diagram for an online order



UML State Machine Diagram for a system like an Automated Teller Machine (ATM):

Core State Elements

- **Initial Pseudo State:** Represented by a solid black circle, this indicates the starting point of the state machine.
- **Sub state:** A simple state that is part of a larger, more complex state. Examples include **Inspection**, **Choice**, and **Transaction**.
- **Composite State:** A state that contains other sub-states. In this diagram, **Active** is the composite state. It includes internal behaviors like:
 - **entry / read card:** An action performed immediately upon entering the state.
 - **exit / eject card:** An action performed immediately before leaving the state.

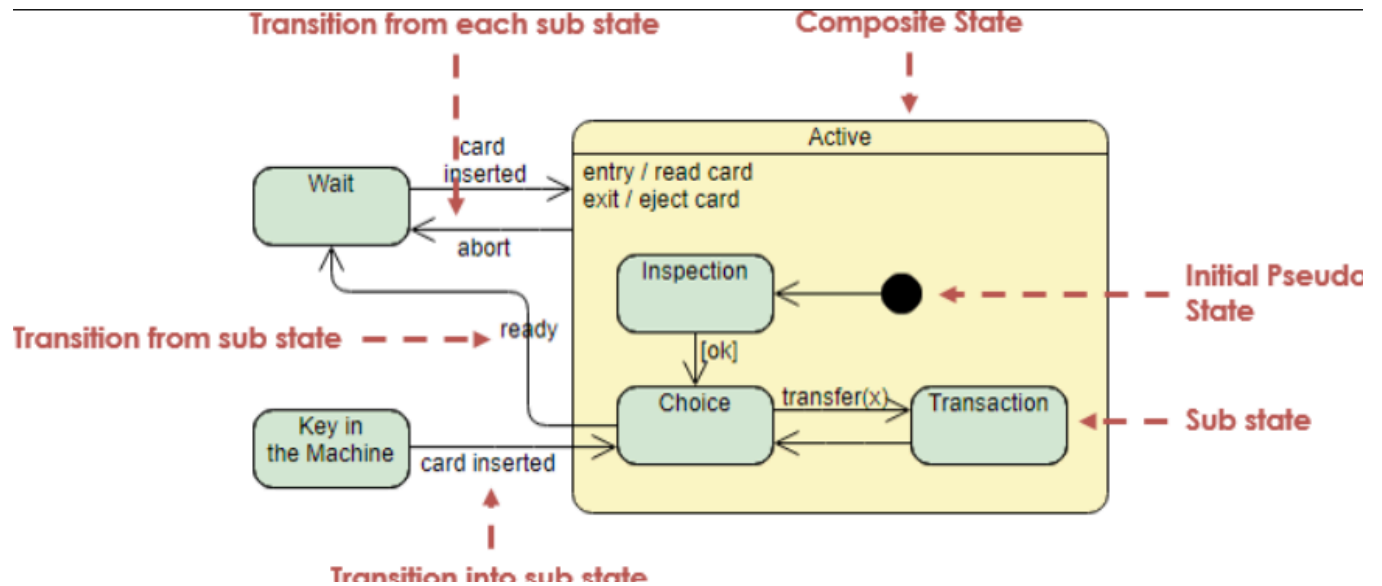
Transition Types

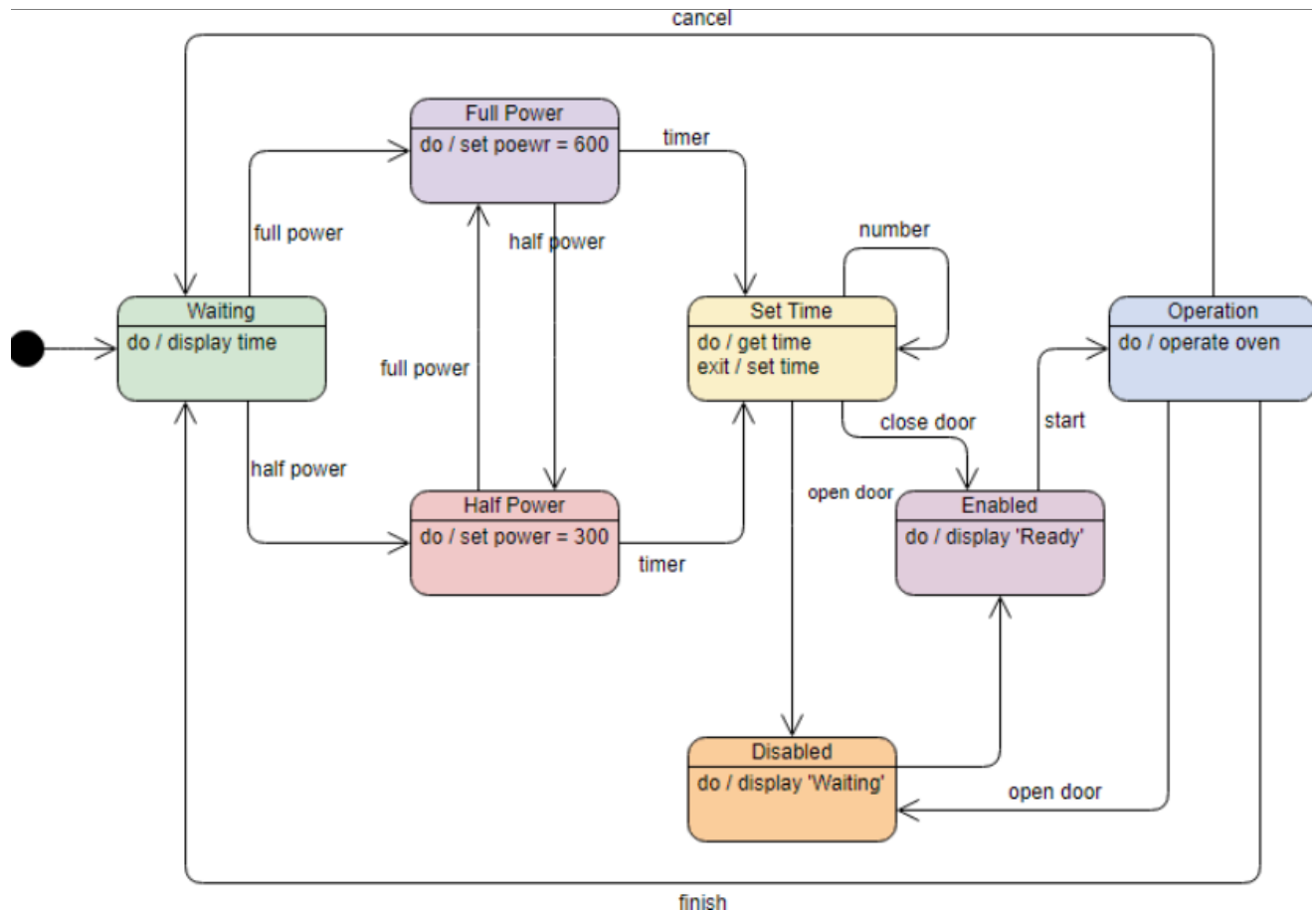
Transitions represent the movement from one state to another, triggered by specific events:

- **Transition into sub state:** An event that moves the system from an external state directly into a state inside the composite state (e.g., from **Key in the Machine** to **Choice** when a "card inserted" event occurs).
- **Transition from sub state:** An event originating from an internal sub-state that moves the system to an external state (e.g., from **Choice** back to **Wait** when "ready").
- **Transition from each sub state:** A high-level transition from the entire composite state. If the "abort" event occurs, the system moves to the **Wait** state regardless of which internal sub-state it was currently in.

Additional Components

- **Wait:** An external state where the system resides before a card is inserted.
- **Guard Conditions:** Written in brackets, like [ok]. The transition from **Inspection** to **Choice** only happens if the "ok" condition is met.
- **Trigger Events:** Labels on the arrows (like "card inserted", "transfer(x)", or "ready") that specify what causes the state change.





Core Elements of the Diagram

A state machine diagram shows the different conditions (states) an object can exist in and how it moves between them based on specific events.

- **Initial Pseudo State:** The solid black circle on the far left indicates the starting point of the process.
- **State:** Represented by rounded rectangles (e.g., **Waiting**, **Full Power**, **Set Time**). These are conditions where the system is performing an action or waiting for input.
- **Transitions:** The lines with arrows showing the movement from one state to another.
- **Events/Triggers:** Labels on the transition lines (e.g., "full power", "timer", "start") that cause the system to change states.
- **State Behaviors:** Actions written inside the state boxes:
 - **do / [action]:** An activity performed while the system is in that state (e.g., **do / display time**).
 - **exit / [action]:** An action performed specifically when leaving the state (e.g., **exit / set time**).

System Workflow and Logic

1. Power Selection

From the initial **Waiting** state, where the system displays the time, the user can choose a power level:

- Selecting "full power" moves the system to the **Full Power** state (do / set power = 600).
- Selecting "half power" moves it to the **Half Power** state (do / set power = 300).
- The user can switch between these power states before proceeding.

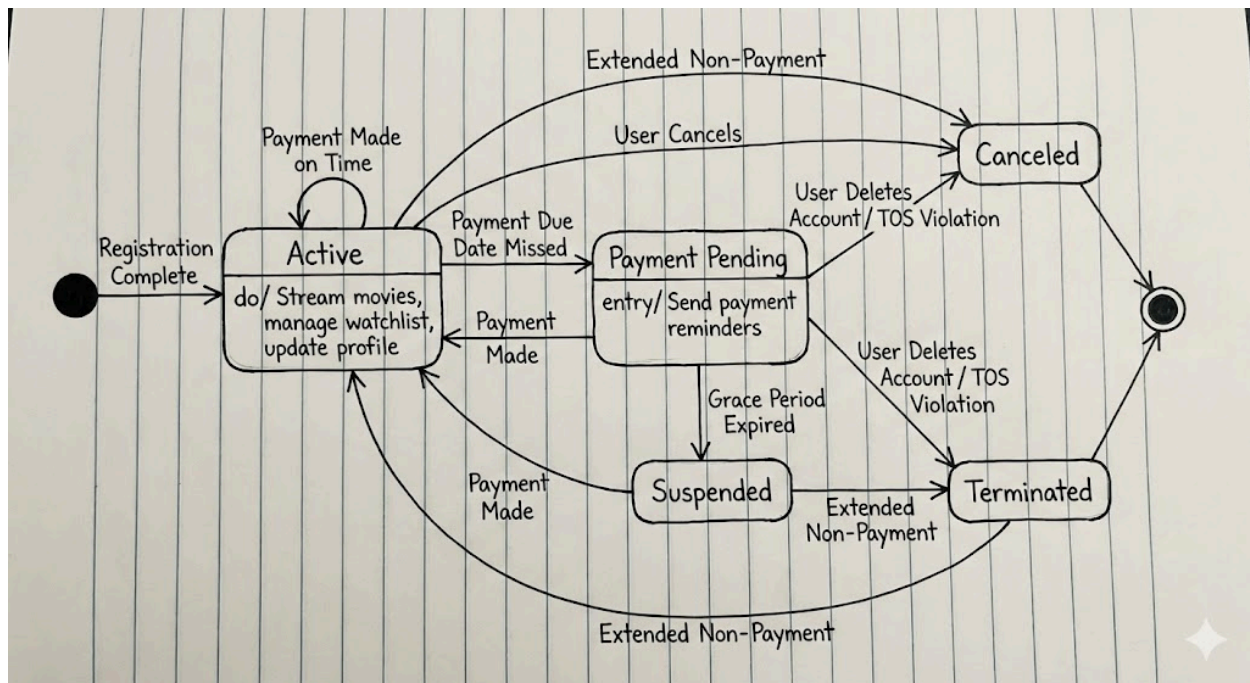
2. Setting and Safety

Once a power level is chosen, the "timer" event moves the system to the **Set Time** state:

- In **Set Time**, the system waits for a "number" input to determine the duration.
- The system then moves to **Disabled** (displaying "Waiting") or **Enabled** (displaying "Ready") based on whether the door is open or closed.
- Safety check: The system cannot move to the **Operation** state unless the door is "close door" and it is in the **Enabled** state.

3. Operation and Termination

- **Operation:** Triggered by the "start" event. The oven begins to do / operate oven.
- **Completion/Cancellation:**
 - If the "finish" event occurs (the timer ends), the system returns to the **Waiting** state.
 - If at any point during operation the user hits "cancel," the system immediately resets to the **Waiting** state.
 - If the "open door" event occurs during operation, the system moves to the **Disabled** state for safety.

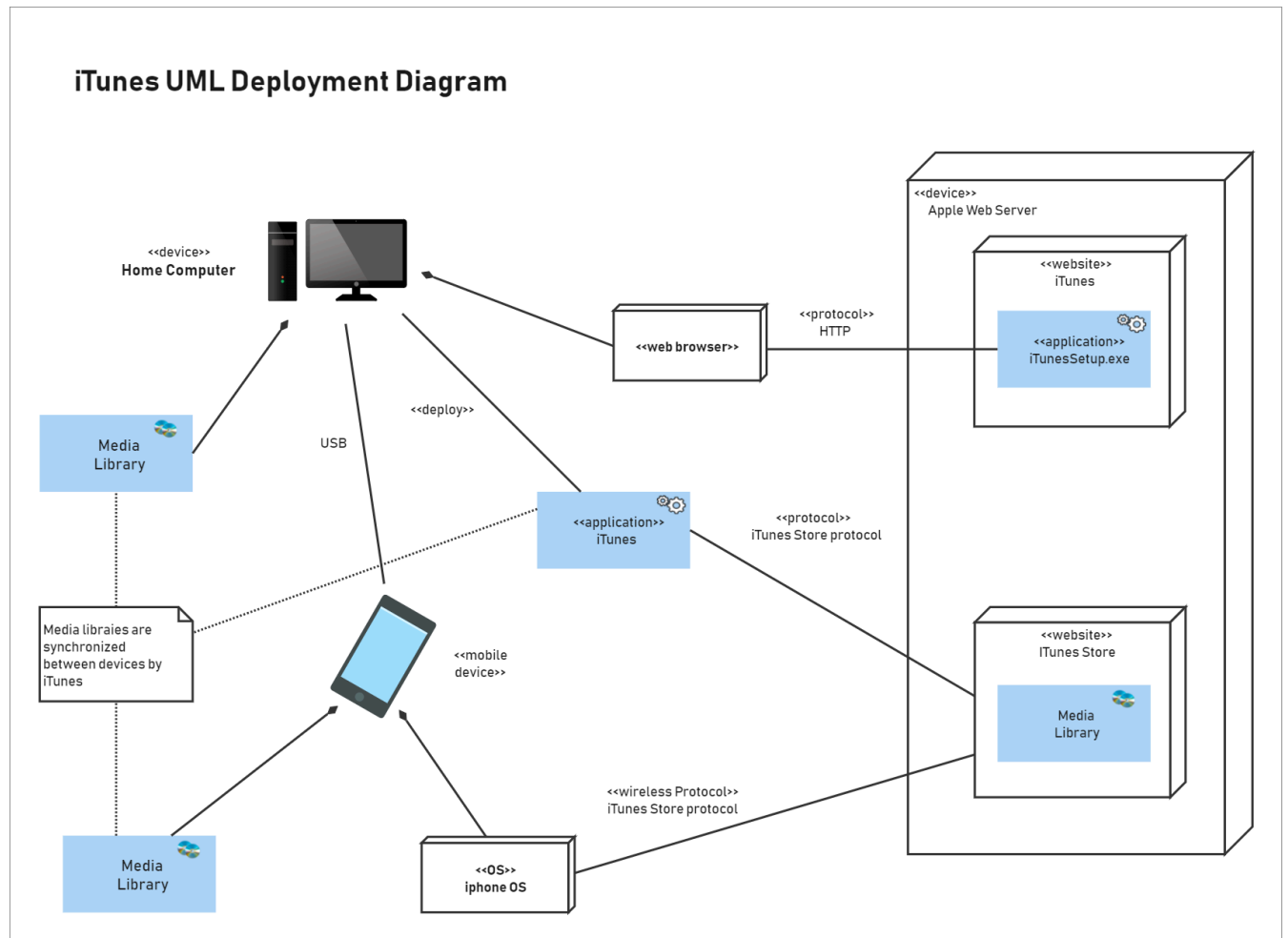


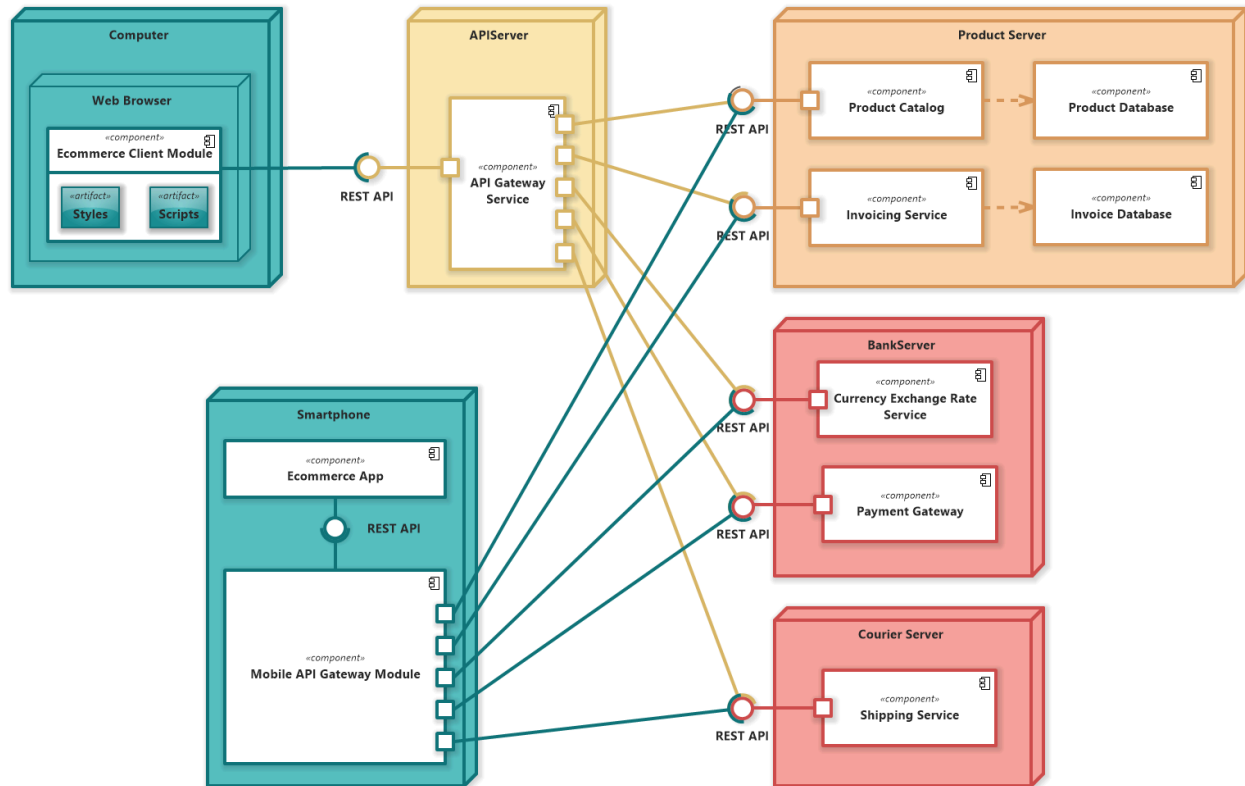
Explanation of Diagram Elements:

- Initial State (●): The starting point of the lifecycle.
- Transition 1 (Start -> Active): The lifecycle begins when the user completes registration (enters details, chooses plan, sets payment).
- State: Active:
 - This is the normal operational state.
 - Internal Activities (do/): The user can perform actions like streaming movies, managing the watchlist, and updating their profile without changing the state.
 - Self-Transition: The account remains Active as long as "Payments are made on time".
- Transition 2 (Active -> Payment Pending): Triggered if the payment due date is missed.
- State: Payment Pending:
 - Entry Action (entry/): Upon entering this state, the system automatically sends payment reminder notifications.
- Transition 3 (Payment Pending -> Active): If the user makes the payment while pending, the account returns to Active.
- Transition 4 (Payment Pending -> Suspended): If the grace period expires without payment.
- State: Suspended: User has restricted access (can log in but cannot stream).
- Transition 5 (Suspended -> Active): If the user resolves payment issues here, the account returns to Active.
- Transition 6 (Suspended -> Canceled): Triggered either by extended non-payment or the user manually canceling.

- Transition 7 (Active -> Canceled): Triggered if the user decides to cancel the subscription while active.
- State: Canceled: A non-recoverable state for this specific subscription instance. It transitions to the Final State.
- Transition 8 (Any state -> Terminated): A transition triggered by permanent user deletion or TOS violations. (Note: For clarity, this is typically shown from active/suspended states, but applies broadly).
- State: Terminated: Account data is deleted. It transitions to the Final State.
- Final State (⦿): The end of the object's lifecycle.

DEPLOYMENT DIAGRAM:





This diagram is a **UML Deployment Diagram** for a large-scale **E-commerce System**. It illustrates the physical hardware (Nodes), the software components running on them, and the communication protocols (REST API) used to connect them.

1. Hardware Nodes (3D Cubes)

The system is distributed across several physical or virtual machines, represented by 3D cubes:

- **Computer:** A client-side machine hosting a **Web Browser**.
- **Smartphone:** A mobile client hosting the **Ecommerce App**.
- **APIServer:** A central middleware server that manages traffic through an **API Gateway Service**.
- **Product Server:** A backend server dedicated to product data and invoicing.
- **BankServer:** A secure third-party or internal server for financial transactions.
- **Courier Server:** A server for managing logistics and shipping.

2. Software Components and Artifacts

Inside the nodes, we see the specific software modules and files:

- **Client Side:**

- The **Ecommerce Client Module** contains specific **artifacts** (physical files) labeled **Styles** and **Scripts**, which are essential for rendering the web interface.
- The **Ecommerce App** on the smartphone connects to a **Mobile API Gateway Module**.
- **Backend Services:**
 - The **Product Server** contains a **Product Catalog** that depends on a **Product Database**.
 - The **Invoicing Service** similarly depends on an **Invoice Database**.
- **External Integrations:** The **BankServer** hosts the **Payment Gateway** and **Currency Exchange Rate Service**, while the **Courier Server** hosts the **Shipping Service**.

3. Communication and Interfaces

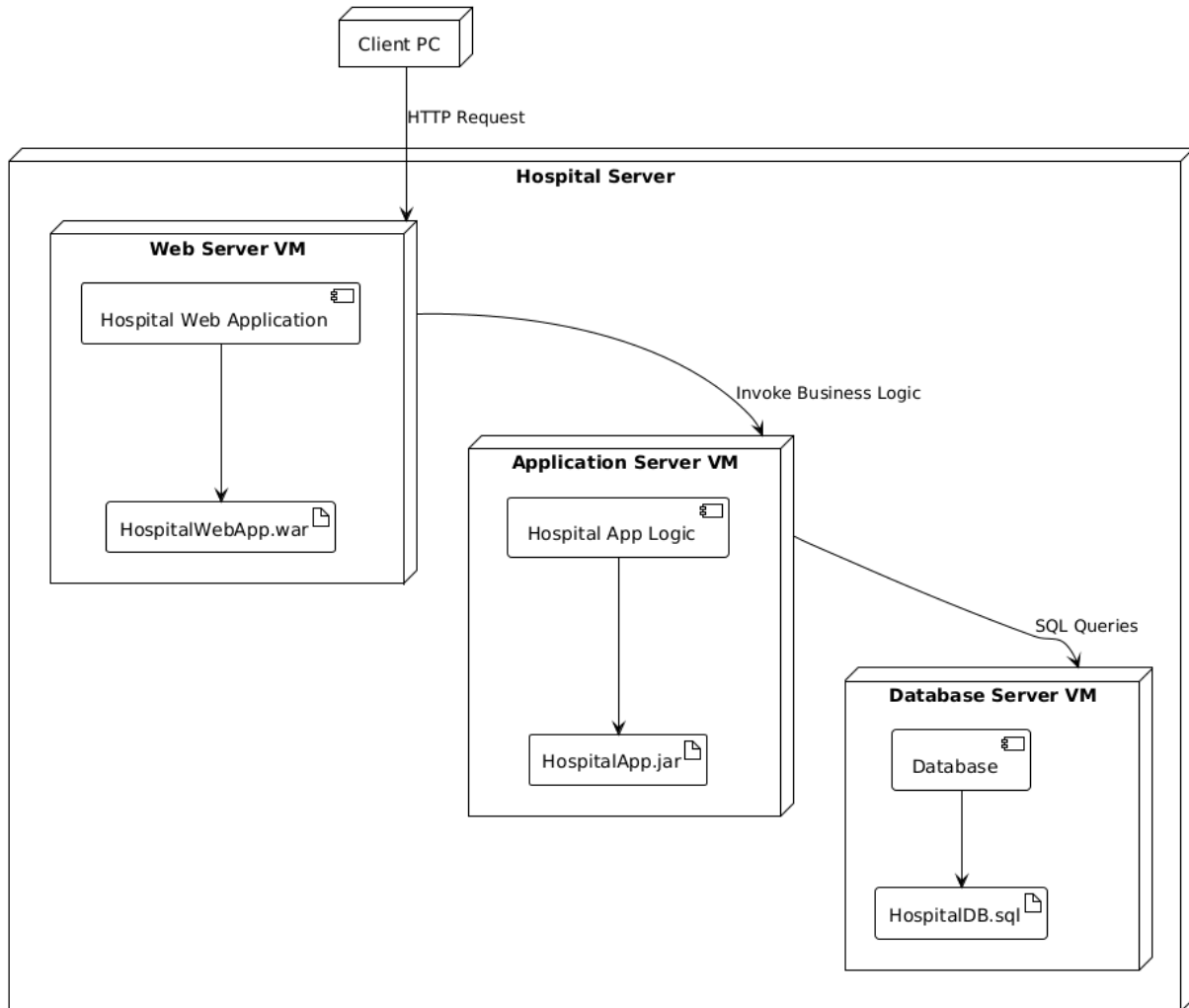
The diagram emphasizes how these disparate parts talk to one another:

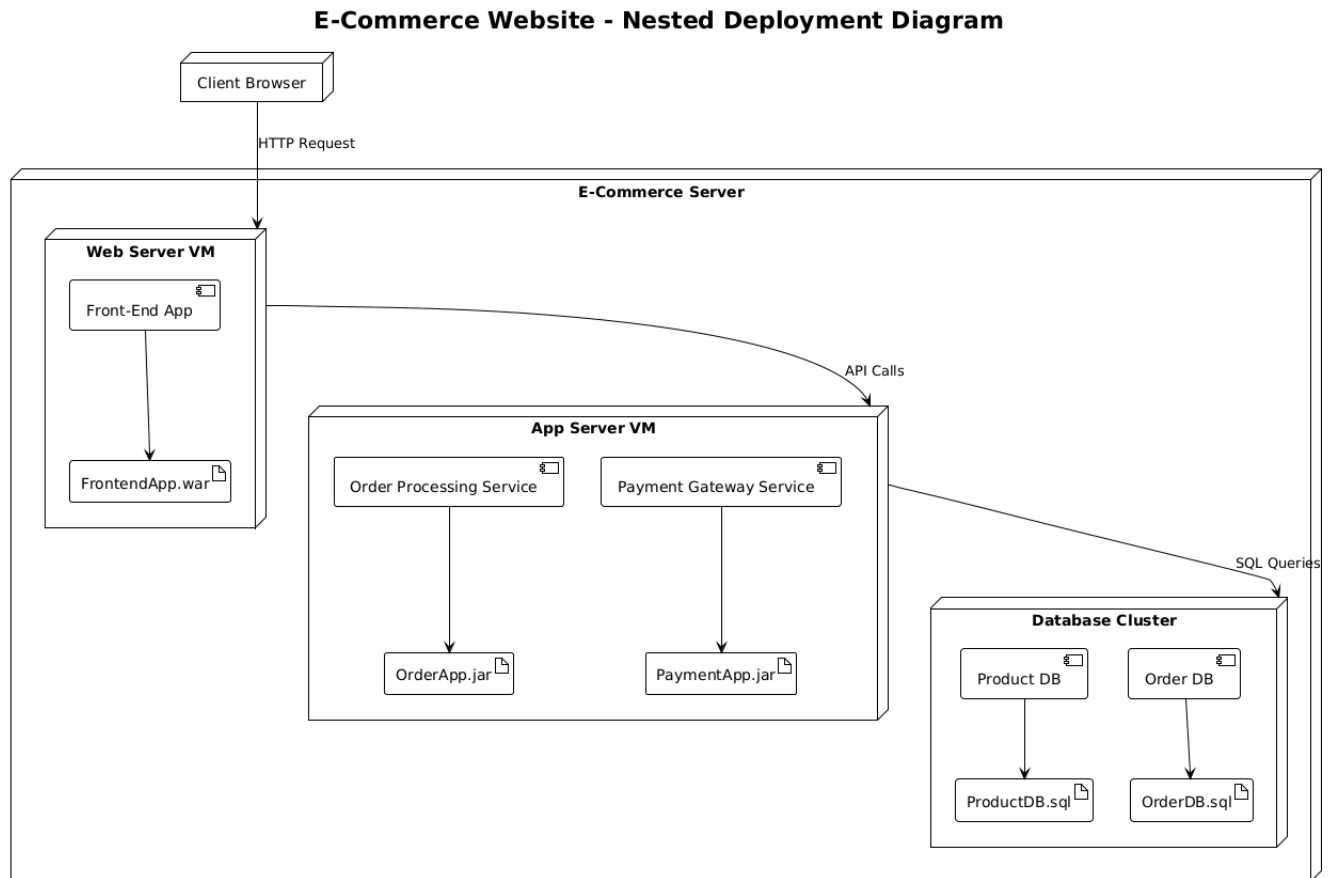
- **REST API:** This is the primary communication protocol used throughout the system.
- **Lollipop & Socket Notation:**
 - **Lollipops (Full circles):** Represent **Provided Interfaces**. For example, the Product Catalog "provides" an interface for the API Gateway to call.
 - **Sockets (Semicircles):** Represent **Required Interfaces**. The API Gateway "requires" a connection to the Product Server to function.
- **Ports (Small squares):** These are the specific interaction points on the edge of components where the interfaces connect.
- **Dashed Arrows:** Indicate **Dependencies**. For instance, the Invoicing Service depends on the Invoice Database artifact to store its data.

Summary of the Workflow

When a user on a **Web Browser** or **Smartphone** takes an action, the request is sent via **REST API** to an **API Gateway**. This gateway then acts as a traffic controller, communicating with the **Product Server** for catalog data, the **BankServer** for payments, and the **Courier Server** for shipping updates.

Hospital Management System - Nested Deployment Diagram





The diagram uses 3D boxes to represent **Nodes**, which are physical or virtual execution environments:

- **Client Browser:** The entry point for users.
- **E-Commerce Server:** A high-level container node.
- **Web Server VM:** A virtual machine nested inside the main server, dedicated to the front-end.
- **App Server VM:** A virtual machine that hosts core business logic services.
- **Database Cluster:** A specialized node for data storage and management.

2. Software Components vs. Artifacts

The diagram distinguishes between logical services and physical files:

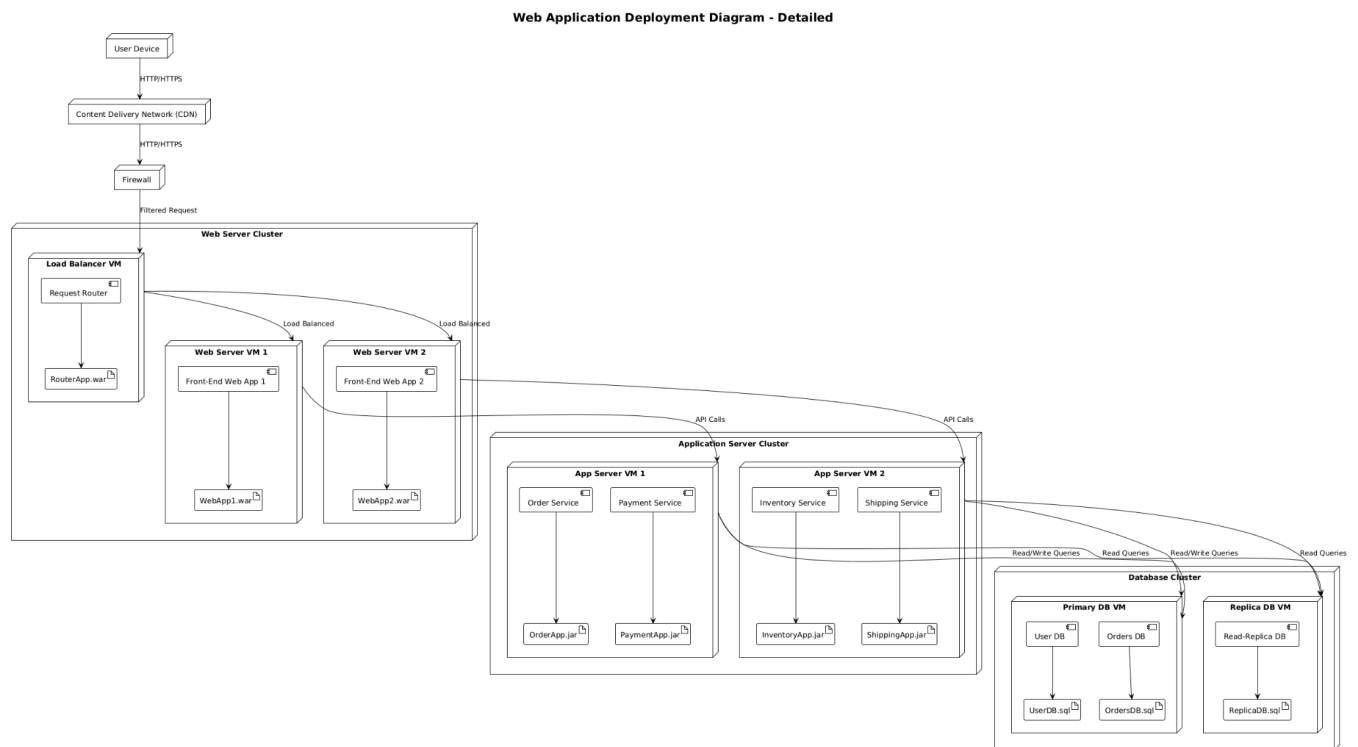
- **Components (Rectangle with icon):** These are logical units of functionality, such as the **Front-End App**, **Order Processing Service**, and **Payment Gateway Service**.
- **Artifacts (Rectangle with "folded paper" icon):** These are the physical manifestations of components, such as **.war**, **.jar**, and **.sql** files.

- For example, the **Front-End App** component is realized by the **FrontendApp.war** artifact.
- The **Product DB** is realized by the **ProductDB.sql** file.

3. Communication and Interaction

Solid lines with arrows indicate how different parts of the system communicate:

- **HTTP Request:** The protocol used between the Client Browser and the Web Server VM.
- **API Calls:** Internal communication between the Web Server (front-end) and the App Server (back-end services).
- **SQL Queries:** The method used by the App Server VM to retrieve or store data in the Database Cluster.



UML Deployment Diagram: Netflix-like Streaming Platform Architecture

Detailed Explanation of Diagram Elements:

This deployment diagram is organized into logical tiers to manage complexity.

1. Client Tier (User Layer)

Located on the far left, representing the diverse devices used to access the platform.

- **Nodes:** Smartphone, Smart TV, Laptop/Tablet.
- **Artifacts:** They all host a <<artifact>> Client Streaming App or browser interface.
- **Connection:** They connect via the internet (usually <<https>>) to the public-facing components (Load Balancer and CDN).

2. Edge & Proxy Tier (Traffic Management & Delivery)

The entry points to the system.

- **Node: Internet Load Balancer:** A critical hardware or virtual appliance that distributes incoming traffic across the web servers to ensure speed and fault tolerance.
- **Node: CDN Edge Server (Distributed Globally):** These are geographically distributed servers.
 - **Artifact:** <<artifact>> Cached Video Content. They serve video streams directly to clients to minimize latency. They fetch content from the main Object Storage if it's not cached.

3. Web & Application Tier (Business Logic)

- **Node: Web Server Cluster (Stacked Boxes):** Multiple web servers acting as the gateway.
 - **Artifact:** <<artifact>> Gateway Service / Session Manager. They handle initial requests and route traffic to the backend.
- **Node: Application Server Cluster (Stacked Boxes):** Handles monolithic core business logic.
 - **Artifact:** <<artifact>> Core Authentication & Logic.
- **Execution Environment: Kubernetes Cluster:** A containerized environment managing microservices.
 - **Artifacts:** Independent microservices run here, e.g., <<artifact>> Recommendation Service and <<artifact>> Billing Service. These communicate via lightweight protocols like <<REST/gRPC>>.

4. Data & Storage Tier (Persistence Layer)

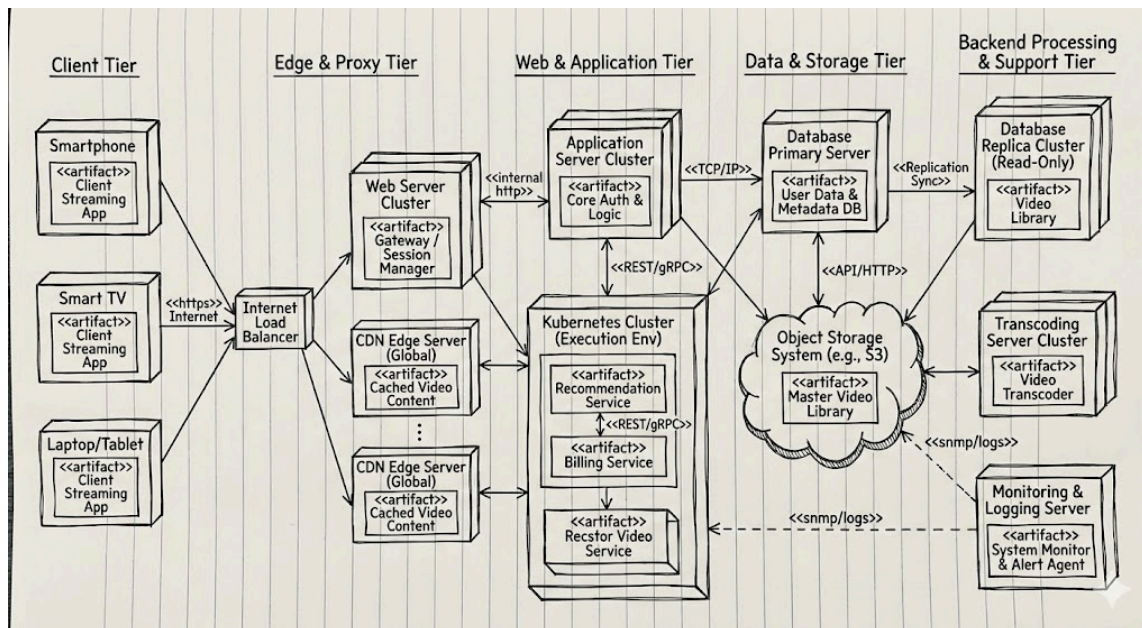
Handles data storage with high availability.

- **Node: Database Primary Server:** Handles write operations for user data.
 - **Artifact:** <<artifact>> User Data & Metadata DB.
- **Node: Database Replica Server(s) (Stacked):** Handles read operations to balance load and provides failover capabilities. It replicates data from the Primary via internal sync protocols.
- **Node: Object Storage System (e.g., Amazon S3):** A highly scalable storage system for massive media files.
 - **Artifact:** <<artifact>> Master Video Library.

5. Backend Processing & Support Tier

Async operations and system health.

- **Node: Transcoding Server Cluster:** Dedicated high-performance servers.
 - **Artifact:** <<artifact>> Video Transcoder. They fetch raw video from Object Storage, convert it into multiple formats, and save it back.
- **Node: Monitoring & Logging Server:** Collects data from all other nodes.
 - **Artifact:** <<artifact>> System Monitor & Alert Agent. Uses internal network protocols (e.g., <<TCP/UDP>>, <<SNMP>>) to track system health.



Question 2: Design Patterns**[20 points]**

You are tasked with integrating a new online flight reservation system with an existing legacy booking system. The new system supports multiple airlines and allows users to book flights using a variety of payment methods. However, the legacy system only supports a specific airline and payment method, and it does not provide direct access to its data. The challenge is to make the new system compatible with the legacy system so users can book flights seamlessly.

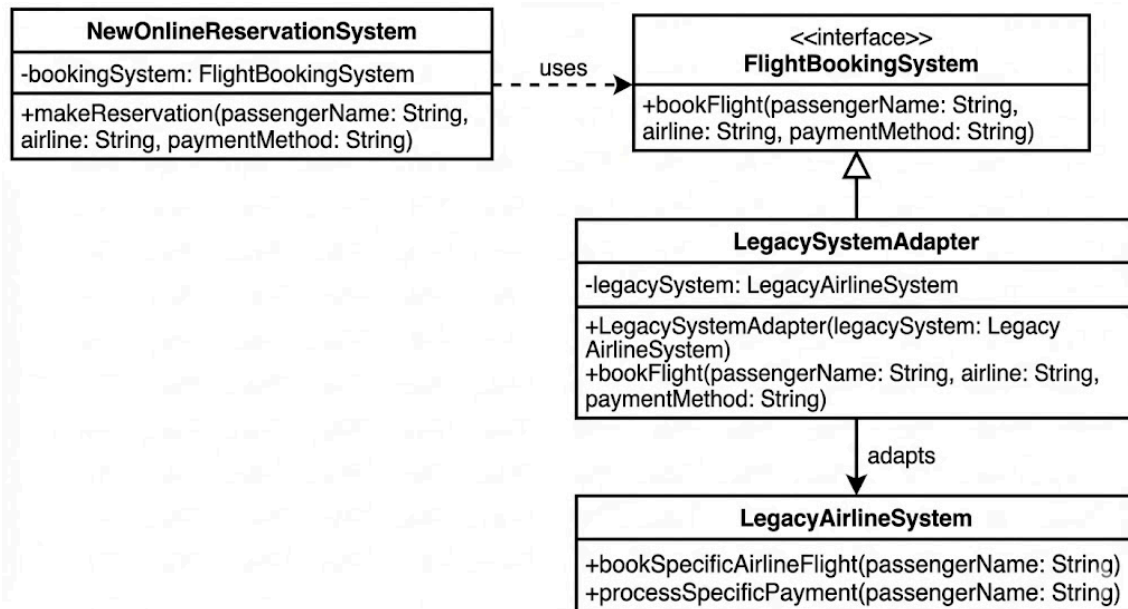
- Identify the pattern which best suits the above scenario.
- Write down the java source code for your selected design pattern based application.
- Write some useful and sensible driver code in the main entry point of application that will demonstrate proper and complete functionality of the application.
- Draw the diagram for your selected design pattern

Note: If you identify wrong pattern, then answers to all subsequent parts will also be invalid. Demonstrate. In this scenario, you need to also make some assumptions of the proper objects and their attributes and behaviors. Your assumption must not contradict with the scenario. Marks will be granted on the proper analysis of the system and its implementation.

a. Identify the pattern which best suits the above scenario.

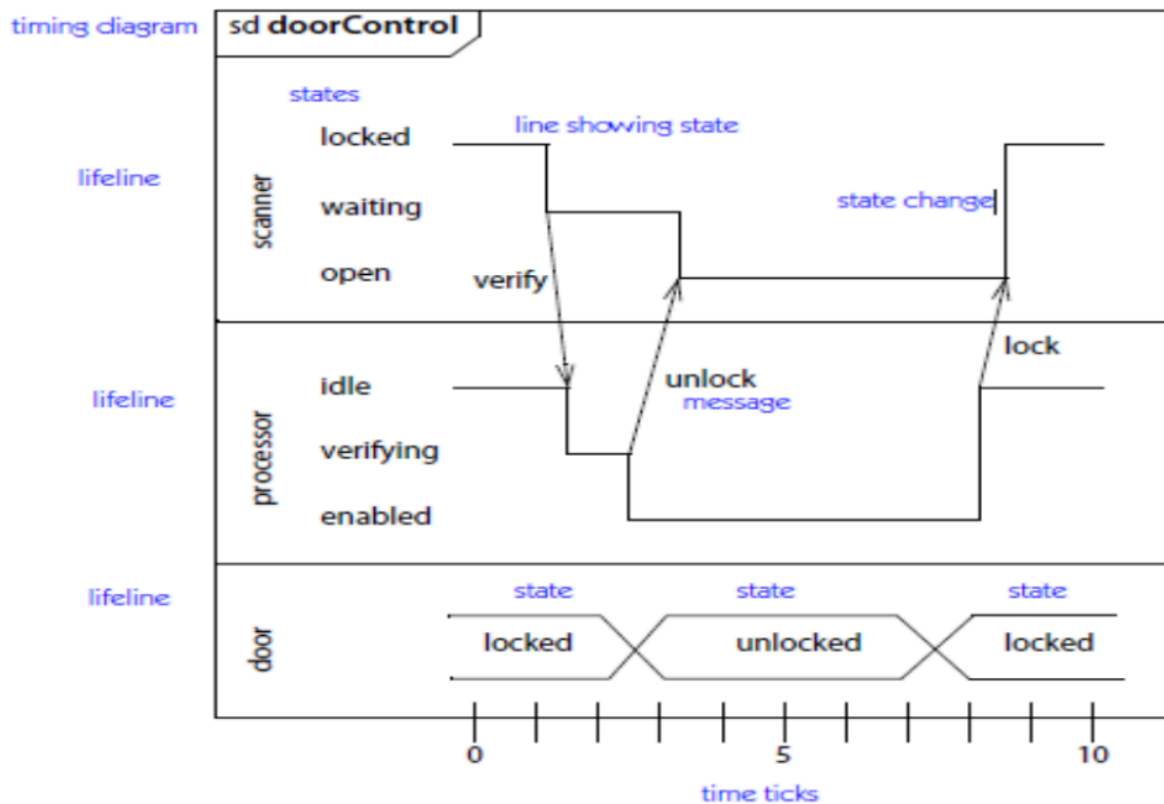
The problem describes a situation where a new system needs to integrate with an existing legacy system that has an incompatible interface. The new system expects a generic way to book flights with multiple airlines and payment methods, while the legacy system only supports a specific airline and payment method and doesn't provide direct data access.

This scenario is a perfect fit for the **Adapter Design Pattern**. The Adapter pattern allows classes with incompatible interfaces to work together by wrapping its own interface around that of an already existing class. It acts as a bridge between the new system and the legacy system.



Client uses Target/Interface which inherits from Adapter which uses Adaptee(Existing Class) which is not compatible directly with the Target.

TIMING DIAGRAM:



Types of Timing Diagram Notations

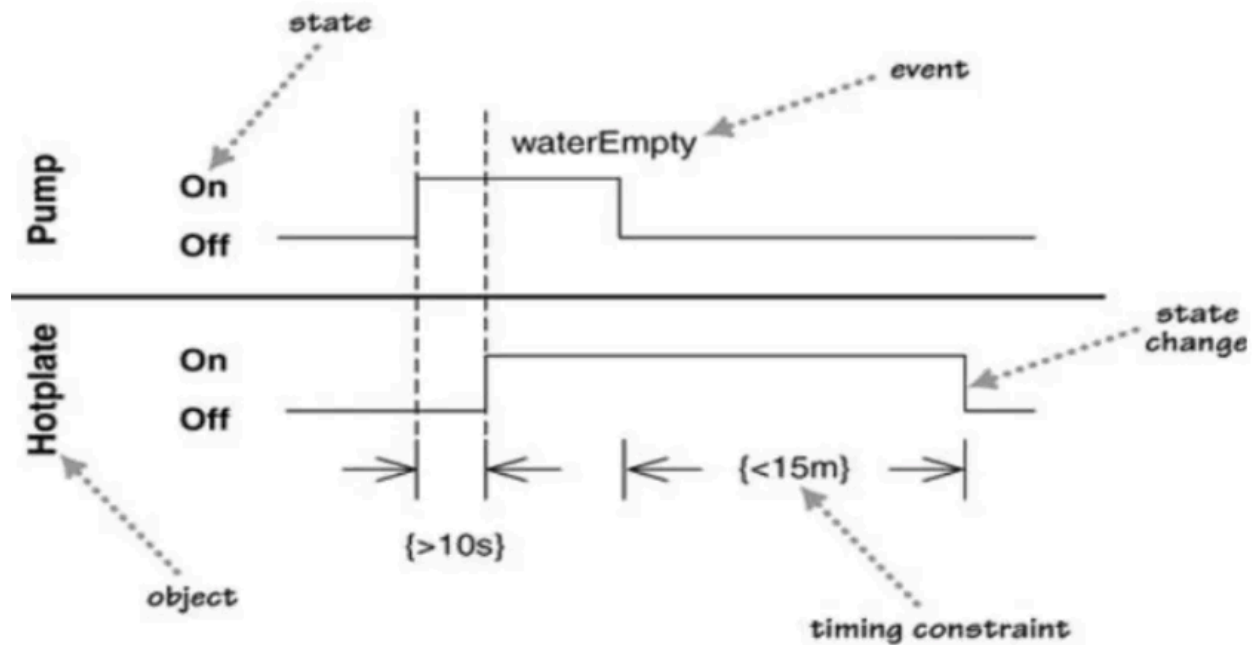
Type	Description	Also Known As
Concise Notation	States shown as <i>areas</i>	Value Lifeline Diagram
Robust Notation	States shown as <i>horizontal lines</i>	State Lifeline Diagram

Examples

Coffee Pot Example

- Rule: At least **10 seconds** must pass between pump turning on and hotplate turning on.
- Pump turns off when water reservoir is empty.
- Hotplate can stay on $\leq$ **15 minutes** after pump off.

This example visualizes how **timing constraints** coordinate multiple devices.



Seminar Example

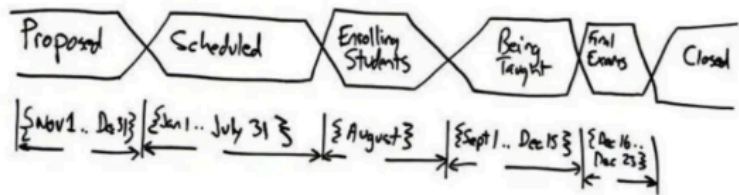
Concise Notation:

Shows life cycle of seminar states:

Proposed → Scheduled → Enrolling → Being Taught → Final Exams → Closed

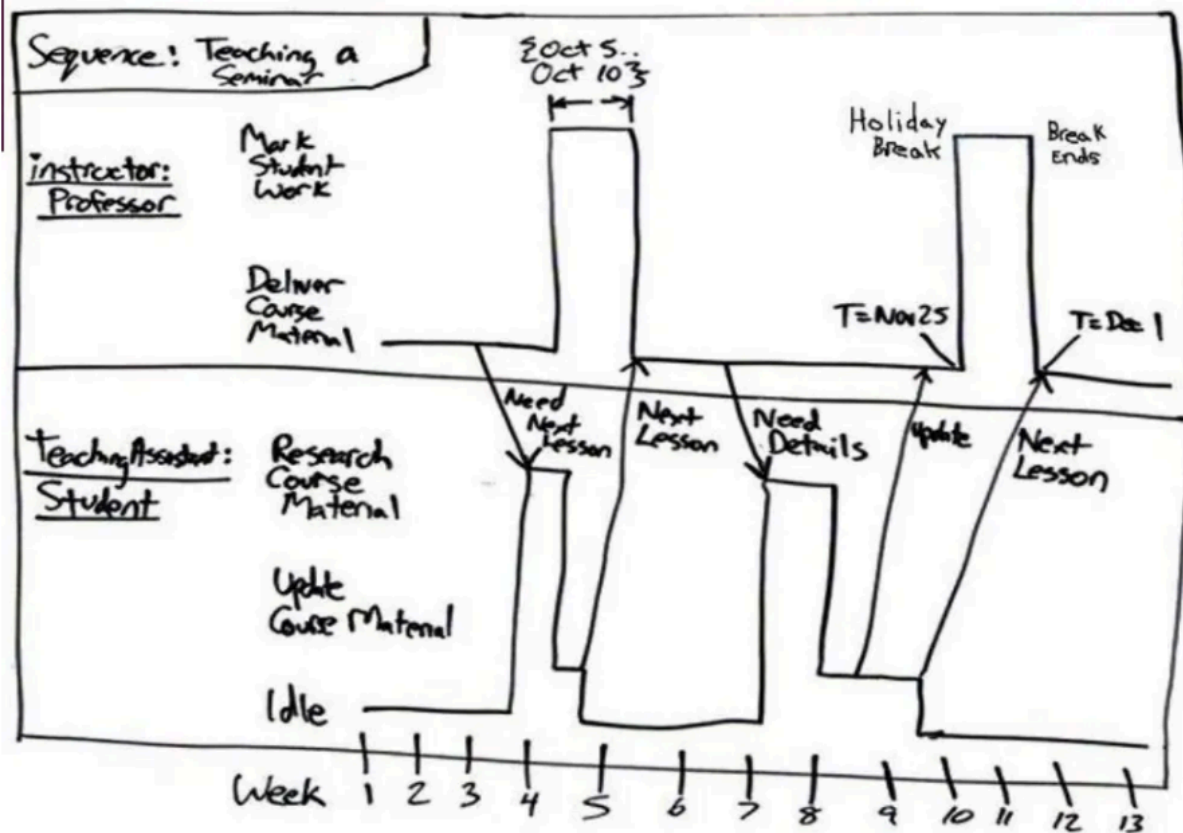
- Life Cycle of Seminar
- The critical states that the seminar exhibits
 - Proposed,
 - Scheduled,
 - Enrolling Students,
 - Being Taught,
 - Final Exams,
 - Closed

:Seminar



Robust Notation:

Shows *interactions between professor, TA, and seminar object* over 13 weeks with events like **Holiday Break** and **Break Ends**.

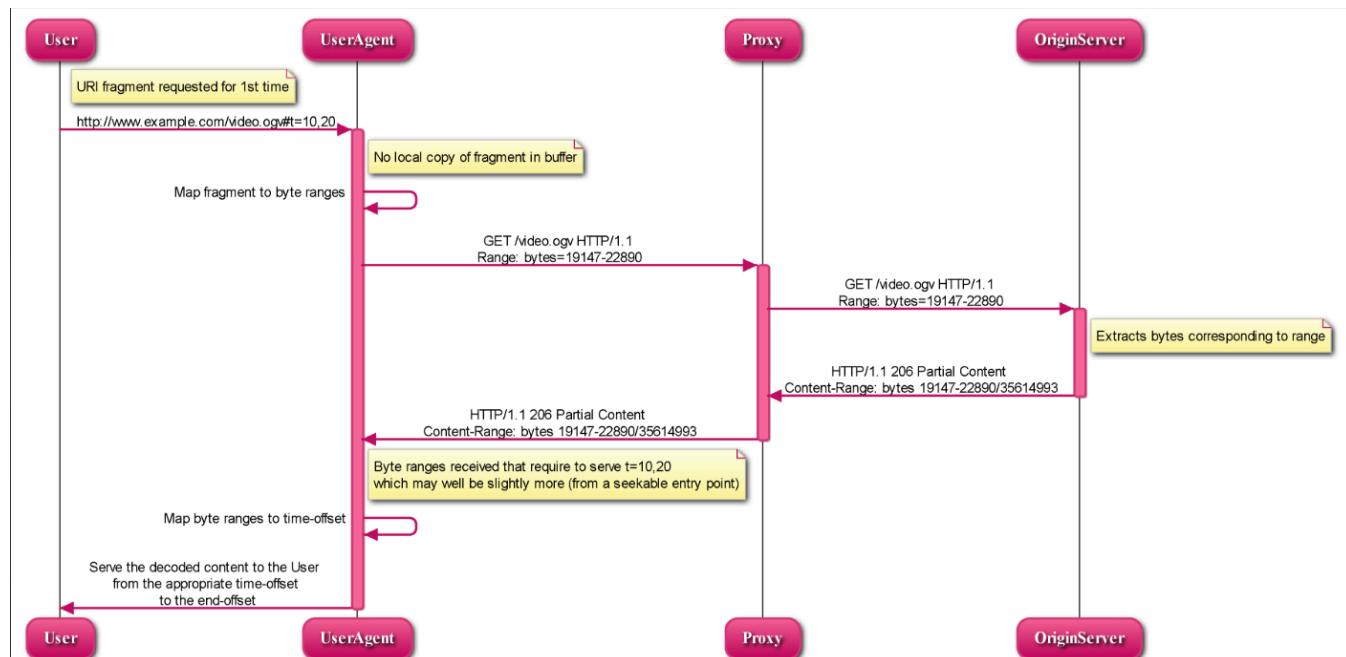
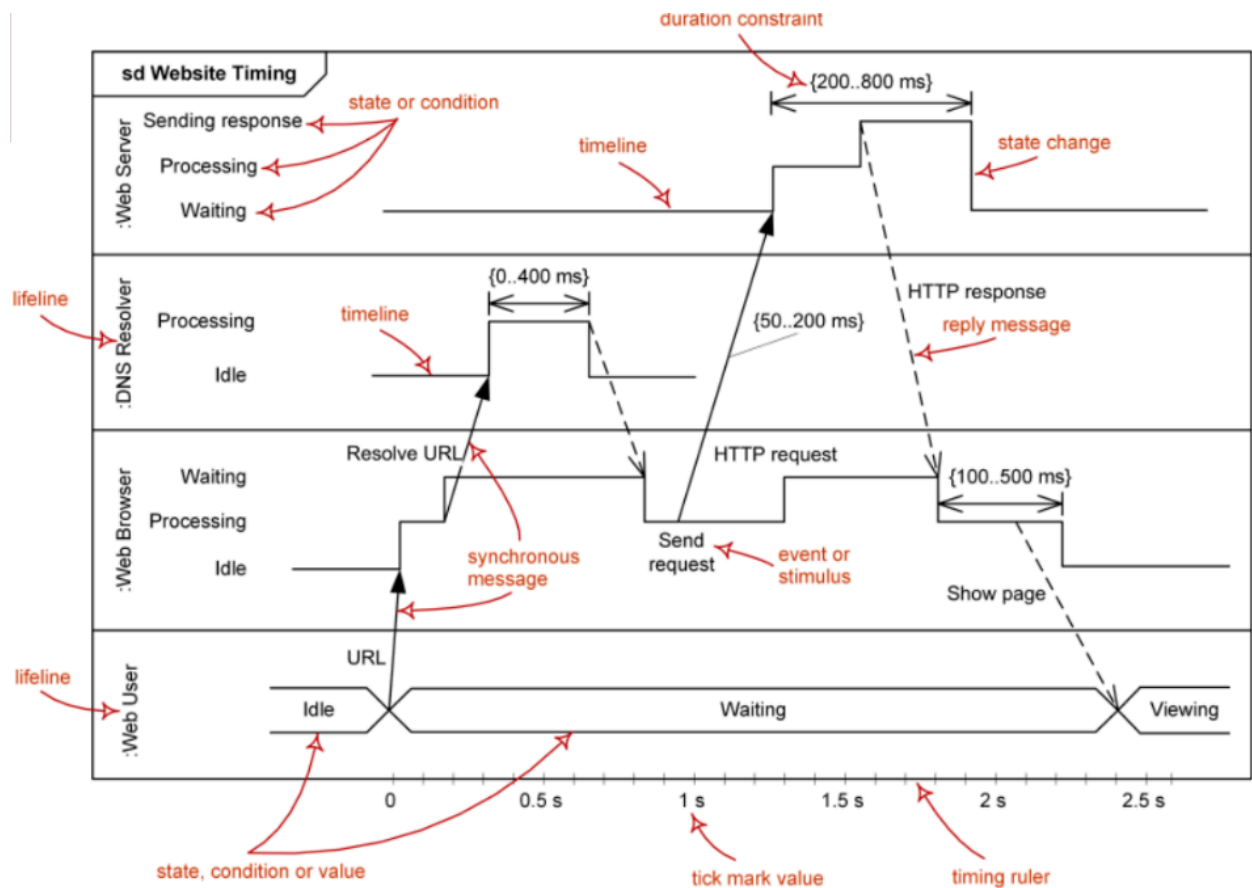


Website Latency Example

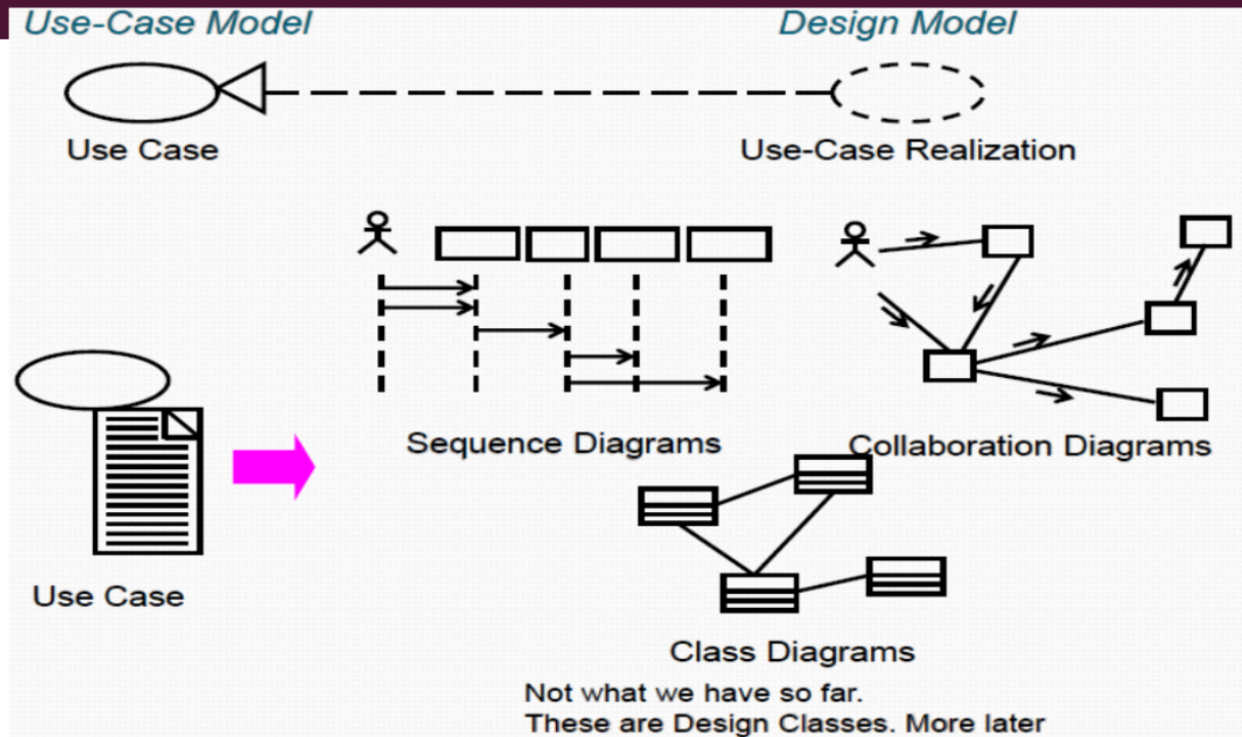
Shows time constraints in user experience:

1. URL entered → DNS resolution (1ms–several sec)
2. Server request → Data processing → Response
3. Browser rendering time before full view.

Helps analyze performance delays and duration constraints.



USE CASE REALIZATION



Use Case Realization

Definition:

Use-case realization describes how a particular use case is implemented in the design model using interacting objects. It focuses on how objects collaborate to achieve the functionality specified in a use case. It bridges the gap between requirements (use cases) and design (classes/objects).

Purpose:

- Ensure the behavior specified in the use case is fully implemented by objects.
- Support traceability from requirements to design and implementation.
- Identify the roles of **Boundary, Control, and Entity** classes in the use case.

Who Performs Use-Case Realization?

- Usually done by a **designer or analyst**.
- Coordinates with class designers to ensure correct object interactions.
- Helps define class responsibilities, interfaces, and collaborations.

Diagrams Used:

1. Interaction Diagrams:

- **Sequence diagrams:** Show messages exchanged between objects over time.
- **Collaboration diagrams:** Show object relationships and message flows.

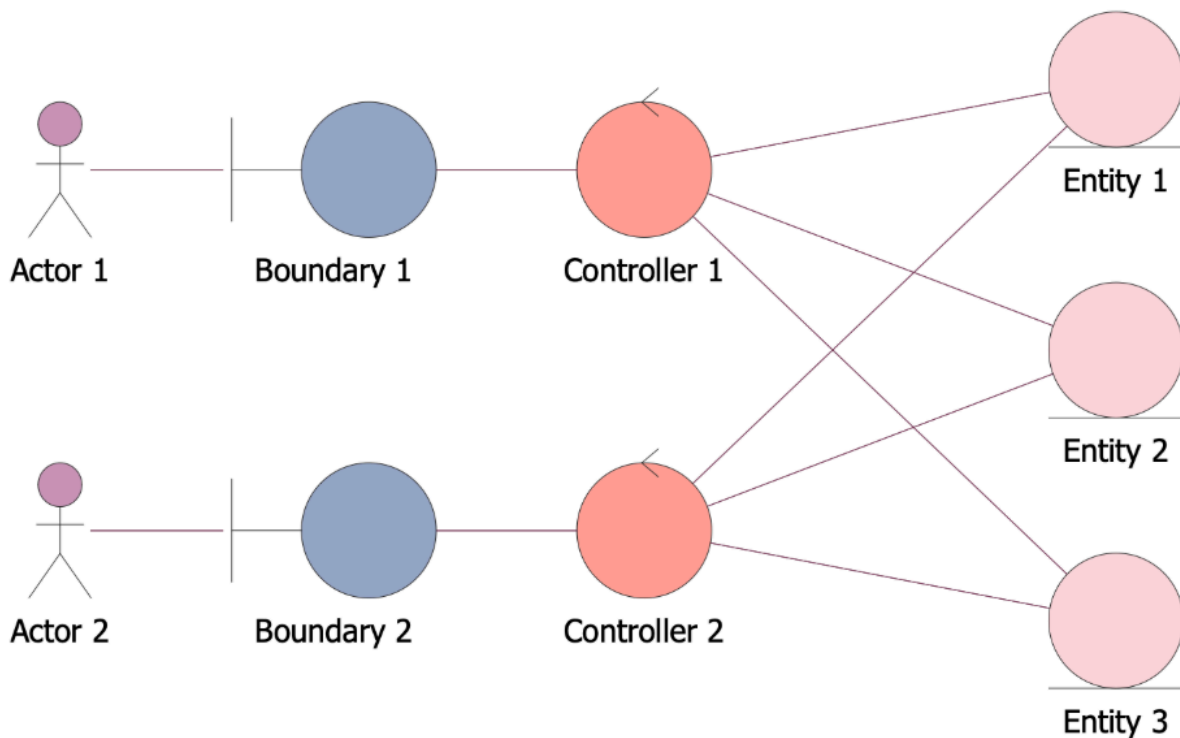
2. Class Diagrams:

- Show classes participating in the use case.
- Show supporting relationships like associations or dependencies.

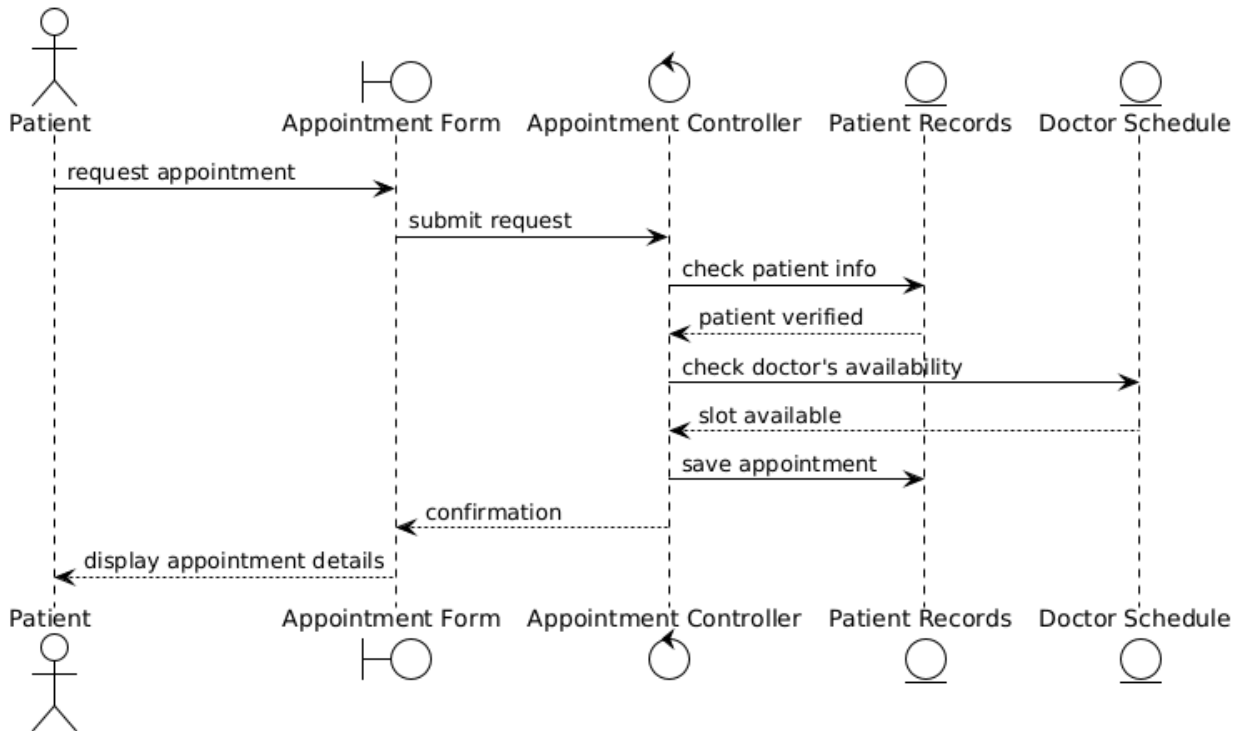
Steps to Realize a Use Case:

1. Identify actors and their interactions with the system.
2. Identify candidate classes (Boundary, Control, Entity).
3. Assign responsibilities to classes based on use-case steps.
4. Model object interactions with sequence or collaboration diagrams.
5. Ensure all normal and alternative flows are covered.

ENTITY CONTROL BOUNDARY ECB:



ECB - Hospital Appointment Booking

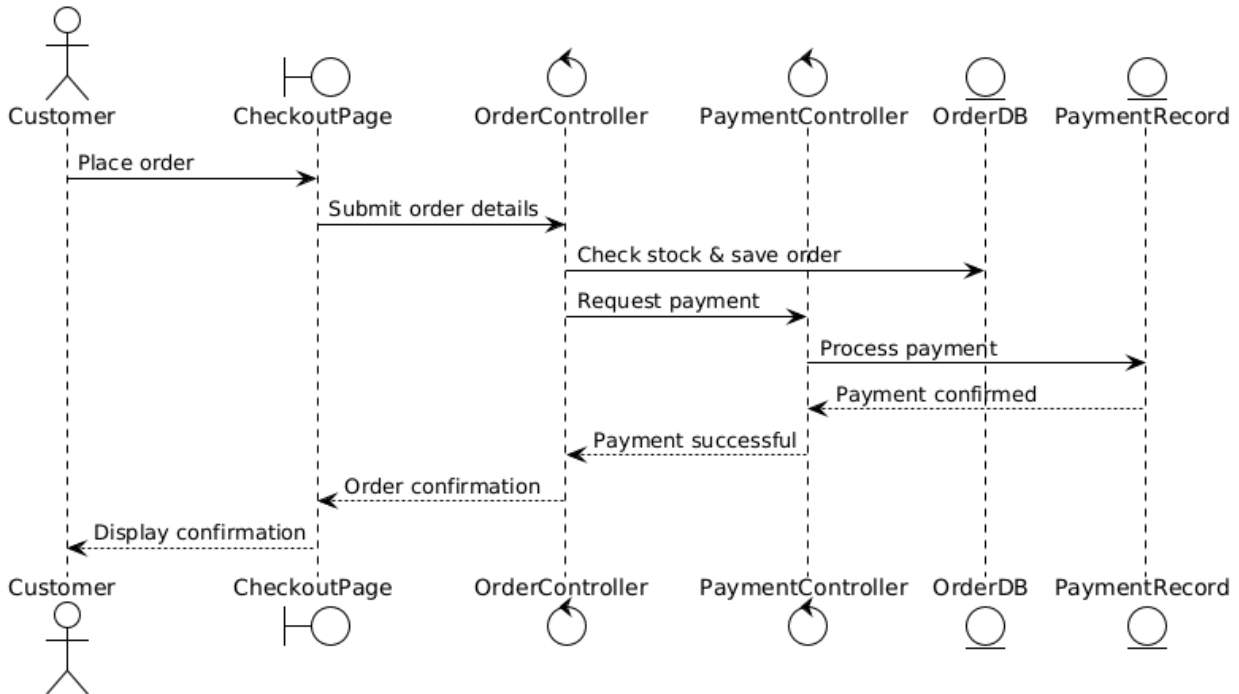


Explanation:

- Boundary: Appointment Form interacts with patient.
- Control: Coordinates checking and saving the appointment.
- Entities: Patient Records and Doctor Schedule store persistent data.

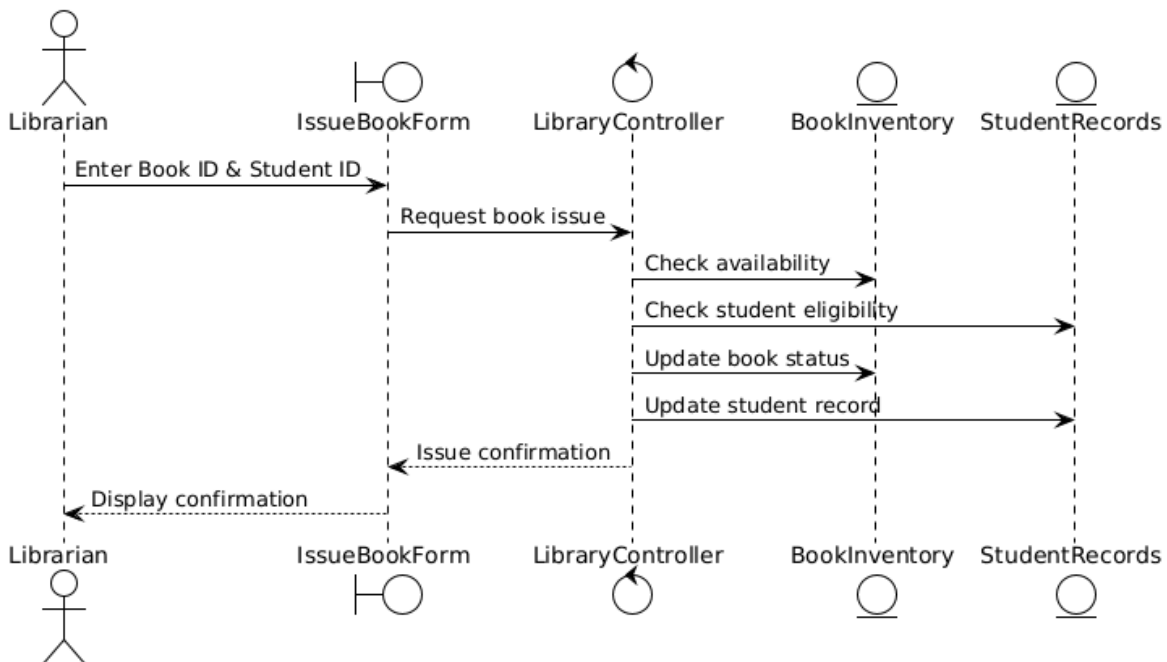
1. Online Shopping Checkout – ECB Sequence Diagram

Online Shopping Checkout - ECB Pattern



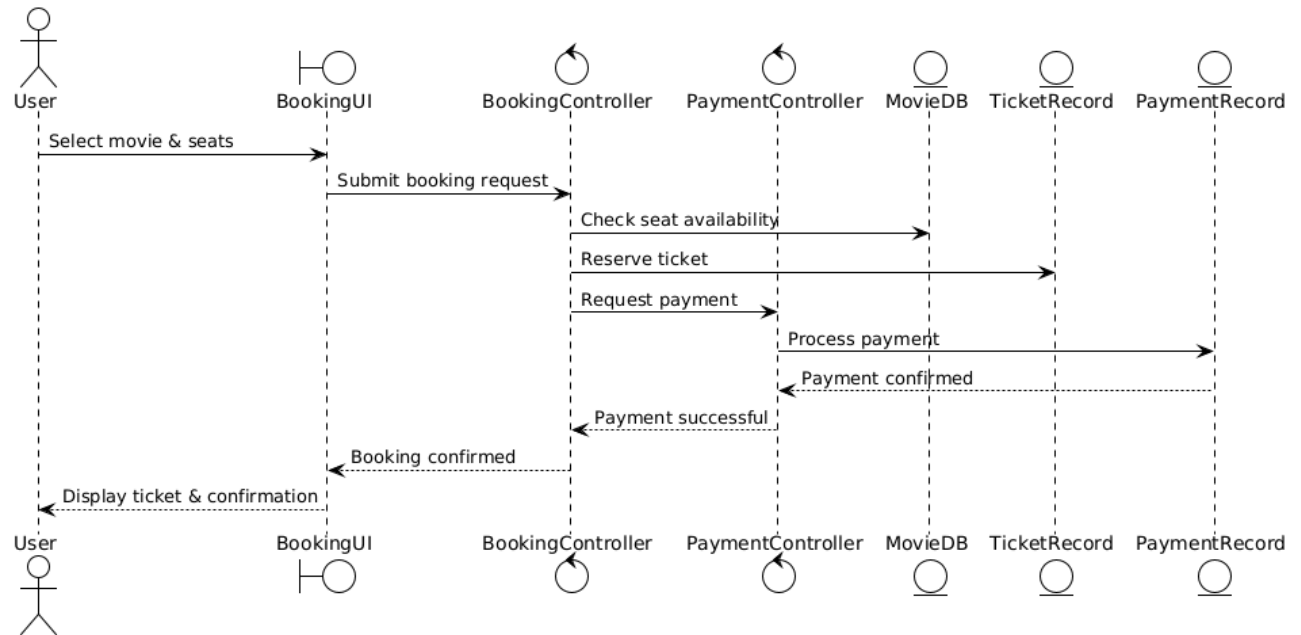
2. Library Management System – ECB Sequence Diagram

Library Book Issue - ECB Pattern

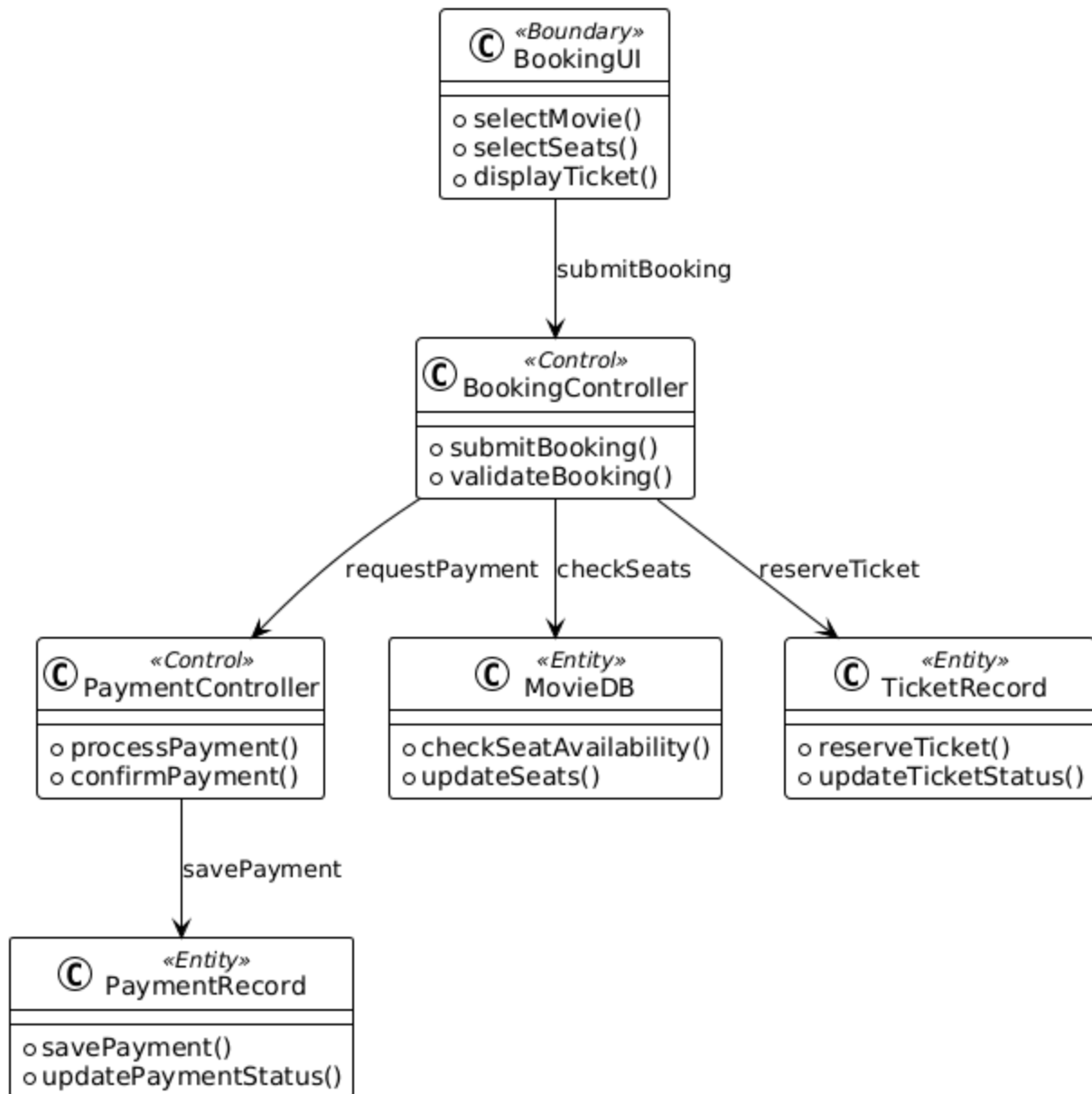


3. Movie Ticket Booking – ECB Sequence Diagram

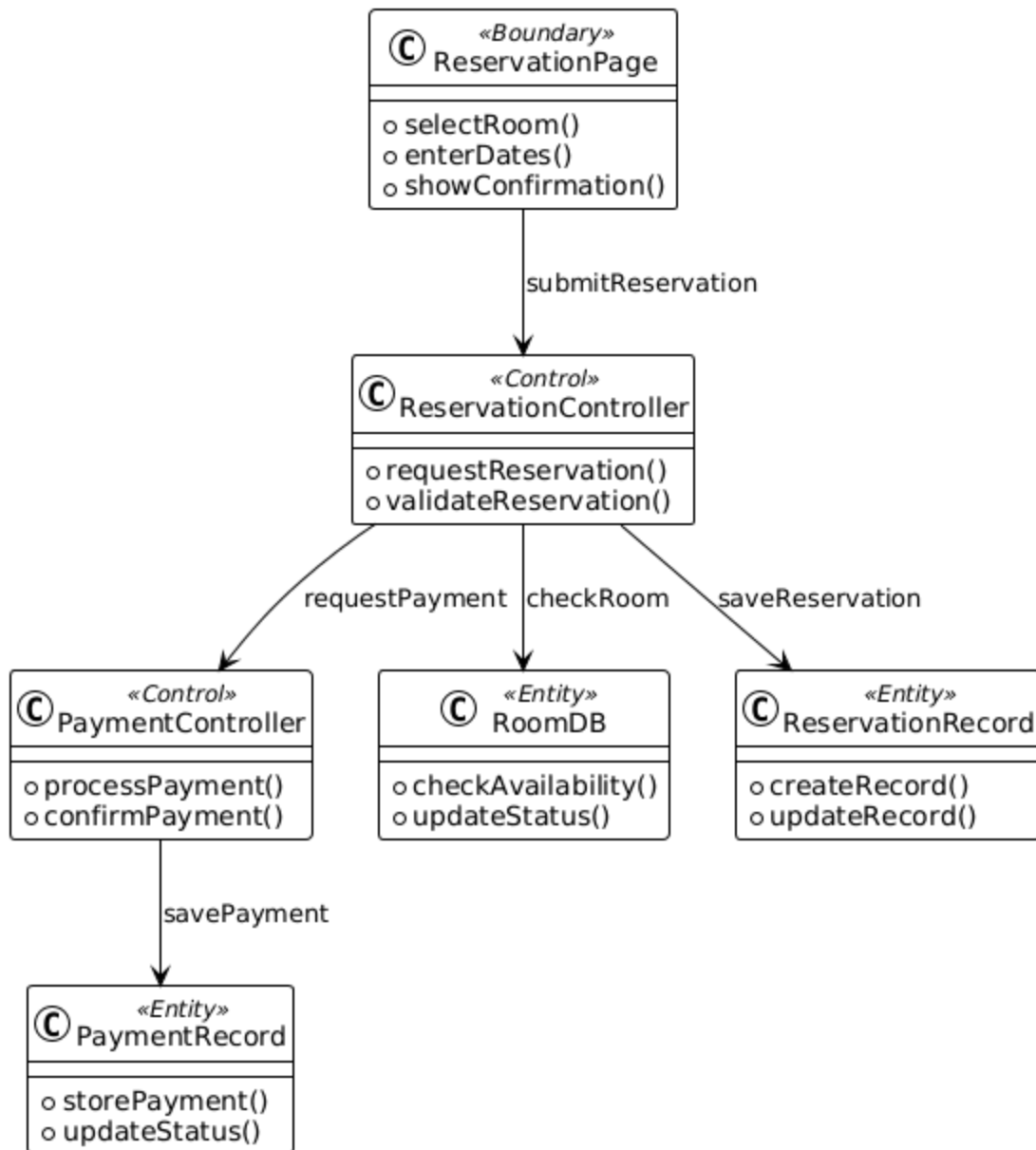
Movie Ticket Booking - ECB Pattern



Movie Ticket Booking - ECB Classes



Hotel Reservation System - ECB Classes



Question 1 (A): ECB class diagram**[10 points]**

A multinational e-commerce company, GlobalShop that operates an online marketplace where users can buy and sell products in various categories, including electronics, clothing, home goods, and more. The platform is available in multiple countries, with different regional warehouses, currencies, and payment gateways. The company provides a marketplace for both customers and sellers while also having internal staff such as administrators. The platform ensures compliance with local laws, supports multiple languages, and integrates with international shipping and payment systems. GlobalShop offers the following features:

1. Customer Portal: Customers browse products, make purchases, manage shopping carts, track orders.
2. Seller Portal: Sellers can list products, update inventory, manage orders, and view customer feedback.
3. Admin Portal: Administrators oversee the entire platform, manage users, handle customer service, and ensure the platform adheres to company policies and local regulations.
4. Payment System: Manages transactions, supports multiple currencies, and integrates with various payment gateways to facilitate purchases globally.

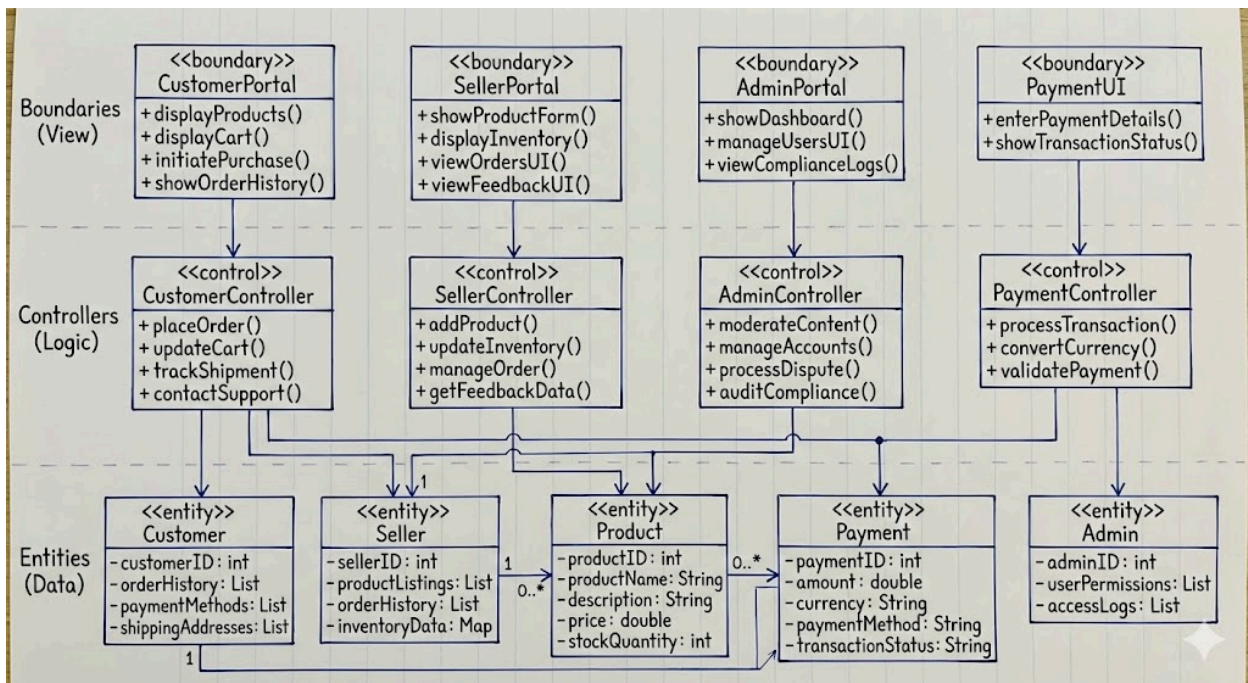
The key entity resources that contain and process information are: Customer that represents individuals who purchase products on the platform. A customer has Customer ID, order history, payment methods, and shipping addresses. Seller that represents businesses or individuals who sell products on the platform. A seller has some data attributes such as Seller ID, product listings, order history, inventory data. Product that represents products listed by sellers on the platform. Product has the attributes Product ID, product name, description, price, stock quantity. Payment which represents the payment for an order. A payment can have the attributes such as Payment ID, amount, currency, payment method, transaction status and the Admin that represents the internal administrators who manage the platform. Here Admin has attributes such as admin ID, user permissions, and access logs.

The controllers represent the business logic that allows users to interact with the system and provide the mechanisms that handle data operations for each role. Customer Controller is responsible for managing customer activities like placing orders, managing shopping carts, tracking shipments, and interacting with customer support. Seller Controller handles seller operations such as adding products, updating inventory, managing orders, and viewing customer feedback. Admin Controller manages the overall system, moderates content, manages user accounts, processes disputes, and ensures compliance

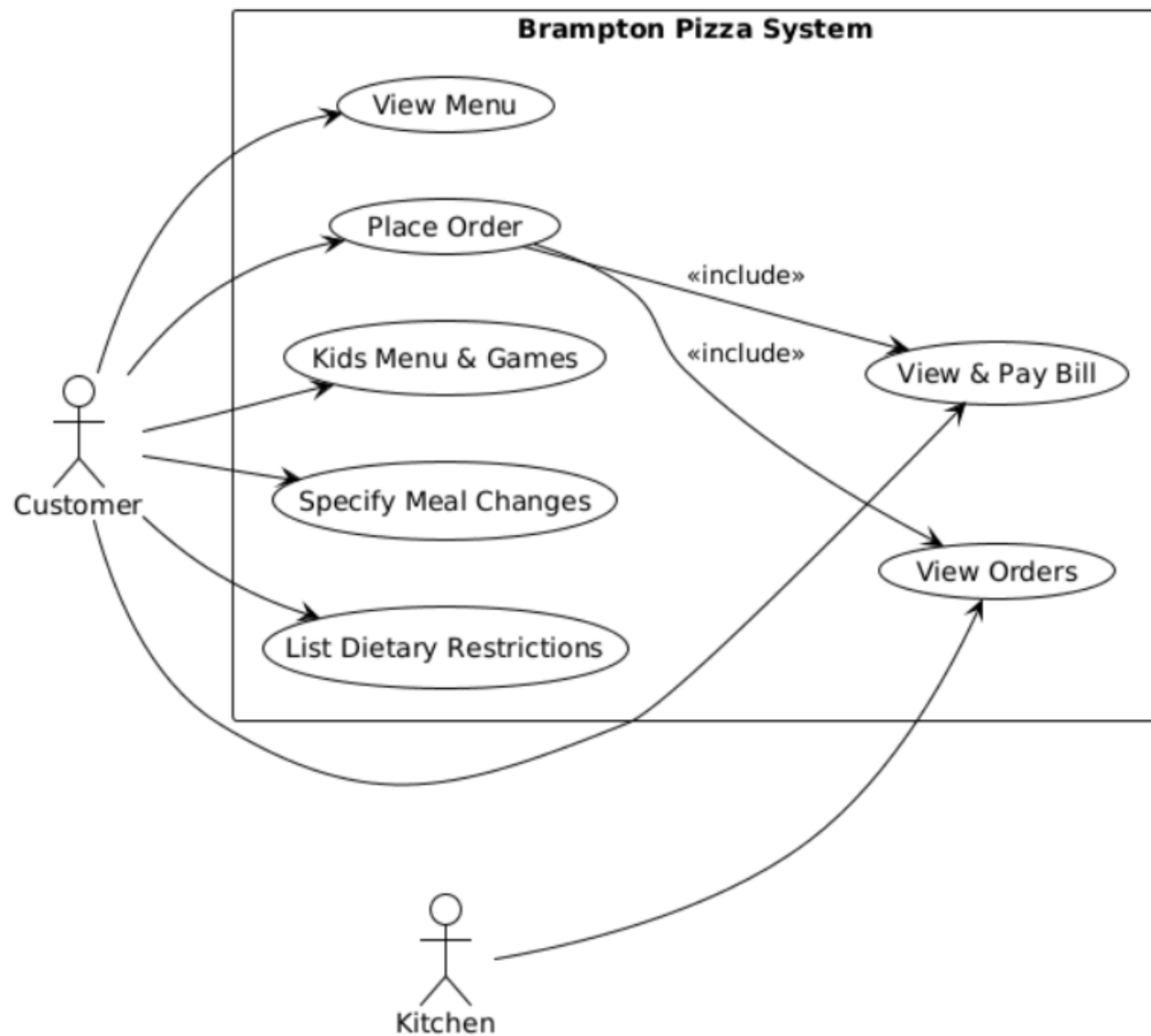
with regional laws. Payment Controller manages transactions, handles currency conversion, and ensures secure payment processing.

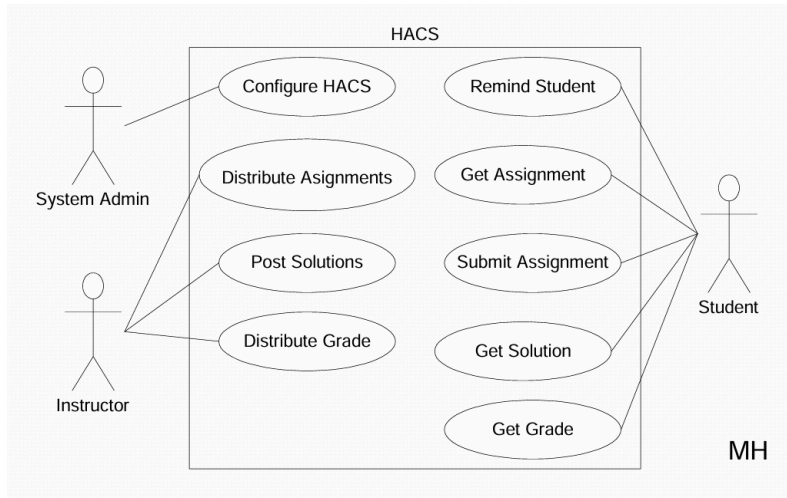
In this scenario boundaries define the access limits between entities and controllers to ensure each user role only has access to their relevant resources. Such as Customer Boundary can access personal order history, shopping cart, payment methods, product details, and shipping information. A Seller can have the Access to product listings, order history for their own products, inventory management tools, and customer feedback. Admin can access to all platform resources, including user management, seller activities, customer transactions, payment logs, and platform settings. A Payment Boundary can Access the Transaction details, payment amounts, and currency information, but only for their own orders or sellers' transactions. Draw ECB diagram (use stereo types in a regular class diagram. Show methods and attributes as well)

NOTE: In your analysis, you need to take care of all kinds of cardinalities in UML class model, be careful to think about all possible kinds of relationships between different use cases and classes. Think of all possible elements of UML modeling that you can infer from the given scenario, however do not overthink to extend the requirements of the given scenario, just analyze what is provided to you in the scenario.



USE CASE DIAGRAM:





HACS Use-Cases

Use case: **Distribute Assignments**

Actors: Instructor (initiator)

Type: Primary and essential

Description: The Instructor completes an assignment and submits it to the system. The instructor will also submit the due date and the class the assignment is assigned for.

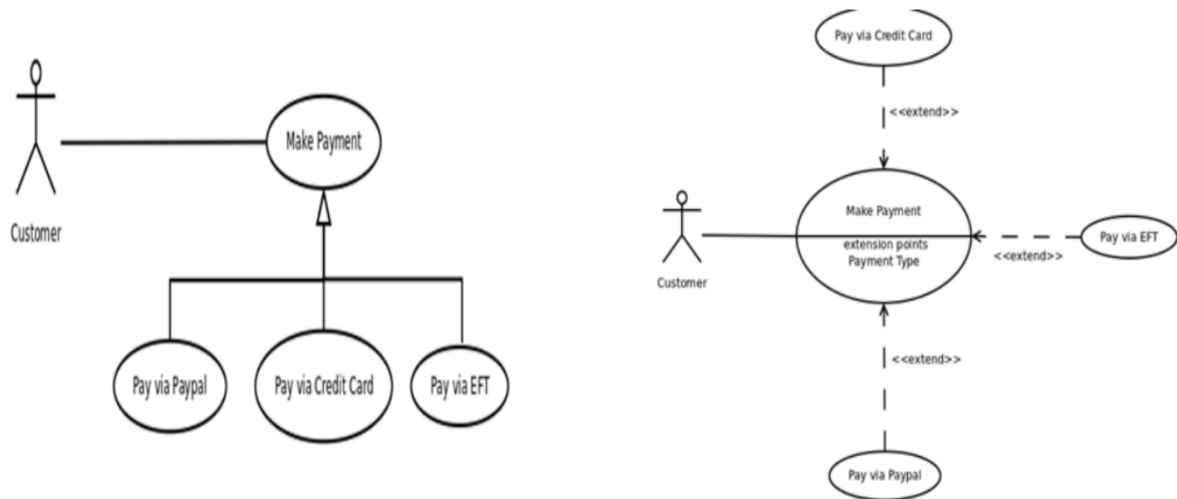
Cross Ref.: Requirements XX, YY, and ZZ

Use-Cases: *Configure HACS* must be done before any user (Instructor or Student) can use HACS

Q: Draw a use case diagram for the hospital reception system. In this system, receptionist can schedule patient appointment and patient hospital admission after the patient registration. Both types of patients i.e. outpatient and inpatient can be admitted in the hospital. Receptionist also checks the insurance and claim forms and put them in file. Patient medical report is also filed by the receptionist.



Generalization vs Extend:



USE CASE TEMPLATES:

Name	View Bill
Participating Actors	Customer
Goals	View the Bill for the Order
Triggers	Request to View Bill
Pre-Condition	Menu Items on Menu, Selecting Dish, Placing Order
Post-Condition	View Bill and Pay for Bill
Basic Flow	1) User Requests to View Bill 2) User Views Bill
Alternate Flows	User Gets Wait Staff to Print and Bring Them Bill
Exceptions	2A) Display error message if no dishes were ordered
Qualities	Bill Takes Less than 10 Seconds to Load

Use Case Name:	Create new order
Brief Description:	When customer calls to order, the order clerk and system verify customer information, create a new order, add items to the order, verify payment, create the order transaction, and finalize the order.
Actors:	Telephone sales clerk
Related Use Cases:	Includes: <i>Check item availability</i>
Stakeholders:	Sales department: to provide primary definition Shipping department: to verify that information content is adequate for fulfillment Marketing department: to collect customer statistics for studies of buying patterns
Preconditions:	Customer must exist. Catalog, Products, and Inventory items must exist for requested items.
Postconditions:	Order and order line items must be created. Order transaction must be created for the order payment. Inventory items must have the quantity on hand updated. The order must be related (associated) to a customer.

Flow of Events:	1. Sales clerk answers telephone and connects to a customer. 2. Clerk verifies customer information. 3. Clerk initiates the creation of a new order. 4. Customer requests an item be added to the order. 5. Clerk verifies the item (<i>Check item availability</i> use case). 6. Clerk adds item to the order. 7. Repeat steps 4, 5, and 6 until all items are added to the order. 8. Customer indicates end of order; clerk enters end of order. 9. Customer submits payment; clerk enters amount.
Exception Conditions:	2.1 If customer does not exist, then the clerk pauses this use case and invokes <i>Maintain customer information</i> use case. 2.2 If customer has a credit hold, then clerk transfers the customer to a customer service representative. 4.1 If an item is not in stock, then customer can a. choose not to purchase item, or b. request item be added as a back-ordered item. 9.1 If customer payment is rejected due to bad-credit verification, then a. order is canceled, or b. order is put on hold until check is received.

Question 1(B): UML Real time Activity model ✓

[10 points]

A Ticket Vending Machine (TVM) automates the process of purchasing tickets for public transportation. The process involves several steps, interactions, and decision points:

User Interaction: The user starts by selecting their preferred language on the TVM screen. The user selects the type of ticket: single ride, return ticket, or multi-day pass. If the user chooses a return ticket, the system asks for the return time and date. The user enters their destination or selects it from a list of predefined stations. **Pricing and Options:** The system calculates the ticket price based on the user's selection and displays the amount.

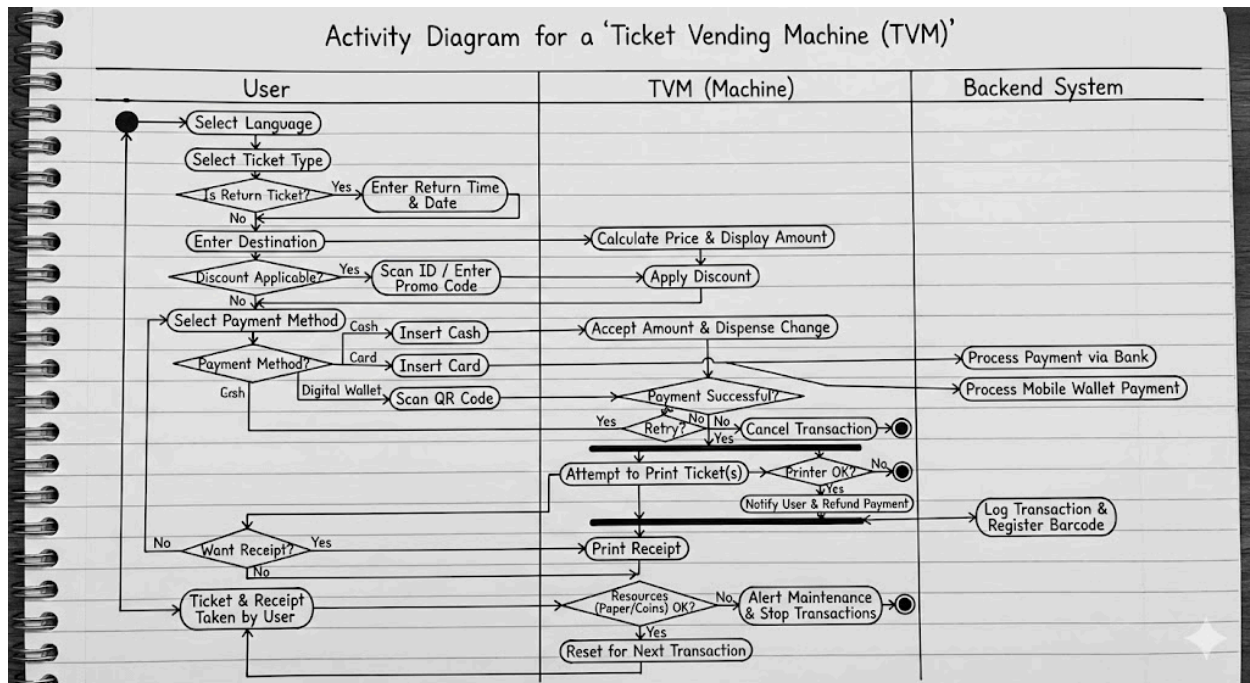
If the user qualifies for a discount (e.g., senior citizen, student), they can apply it by scanning a valid ID or entering a promo code. **Payment Process:** The user selects a payment method:

Cash: The machine accepts the required amount and returns change if applicable.

Card: The user inserts their card, and the machine processes the payment via the bank.

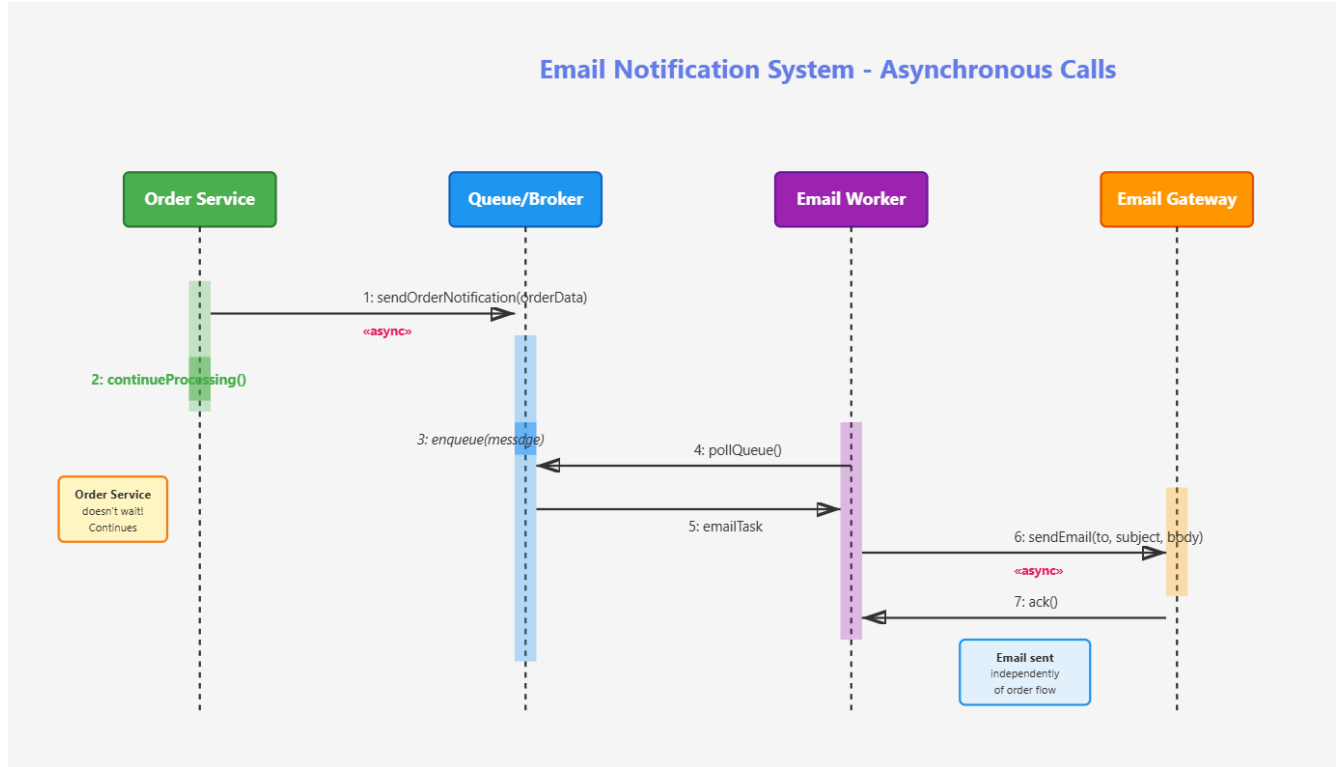
Digital Wallet: The user scans a QR code to complete the payment using a mobile wallet. If the payment fails, the system prompts the user to retry or cancel the transaction. **Ticket Printing** Once payment is successful, the system prints the ticket(s). If the user selected a multi-day pass, the ticket includes specific details like validity period and usage instructions. Concurrently, the system updates the backend database to log the transaction and ensure the ticket's barcode is registered for validation. If the printer malfunctions, the system notifies the user and refunds the payment. If the machine runs out of paper or coins for change, the system stops new transactions and alerts the maintenance team. After receiving the ticket, the user can optionally print a receipt. The system resets and is ready for the next transaction. Create an Activity Diagram for a "Ticket Vending Machine" (TVM) Process based on the detailed scenario below. Your diagram must include Swimlanes, Forks, Joins, and Decision Points to demonstrate the process comprehensively. Ensure the workflow reflects interactions between the user, machine, and backend systems. Ensure a logical workflow that begins with language selection and ends with ticket printing and user satisfaction.

Activity Diagram for a 'Ticket Vending Machine (TVM)'



SEQUENCE DIAGRAM:

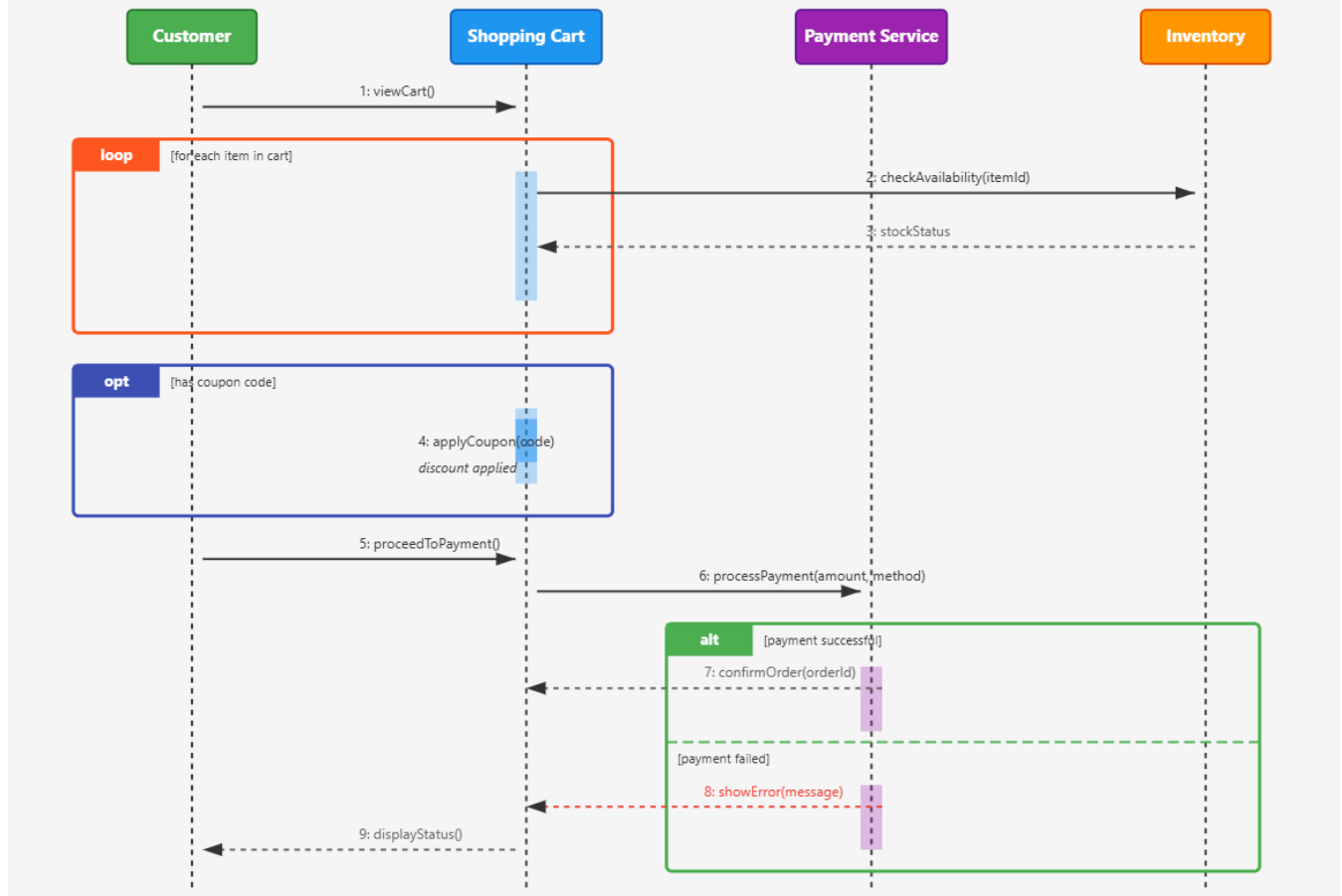
A sequence diagram is a UML behavioral diagram that represents object interactions arranged in a time sequence to describe how a particular scenario is executed.



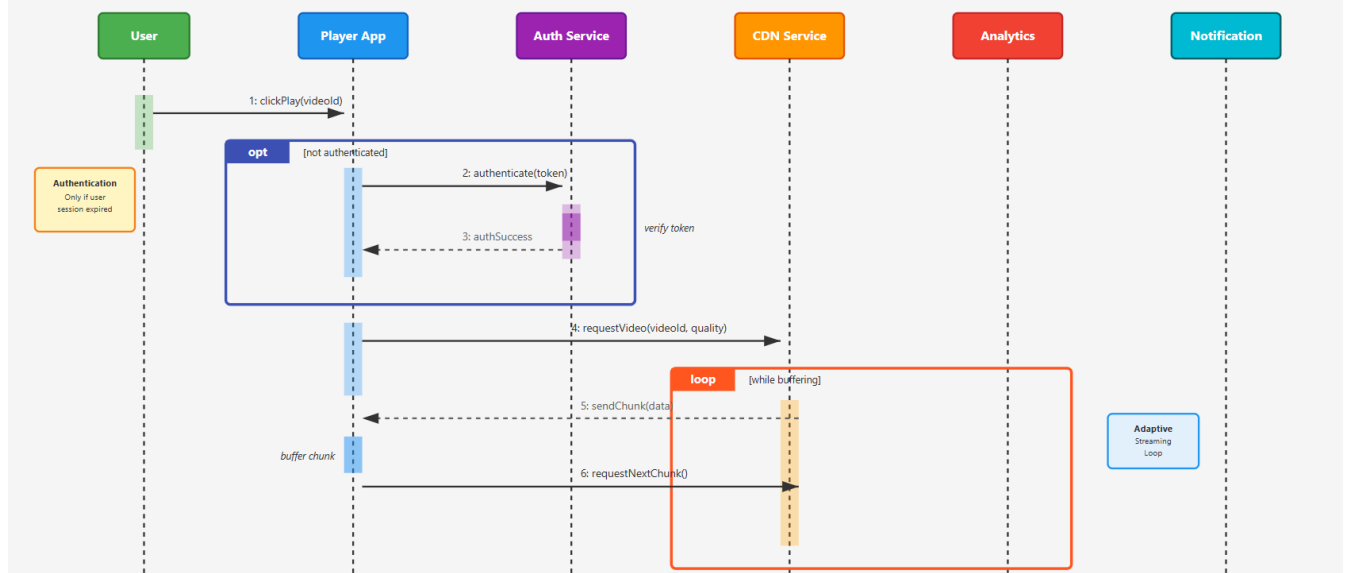
5. Quick Reference Guide

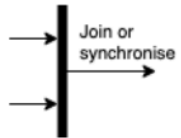
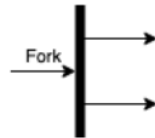
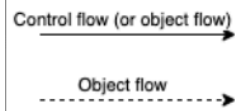
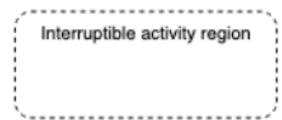
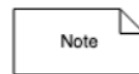
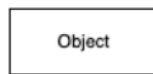
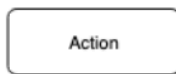


E-Commerce Checkout - With Fragments

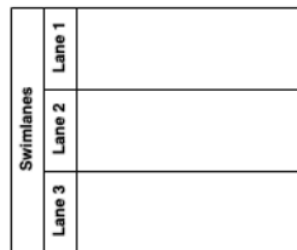
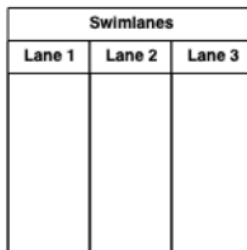
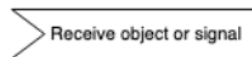
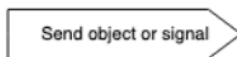
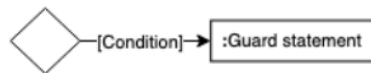
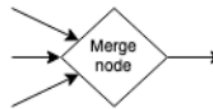


Video Streaming - All Concepts Combined



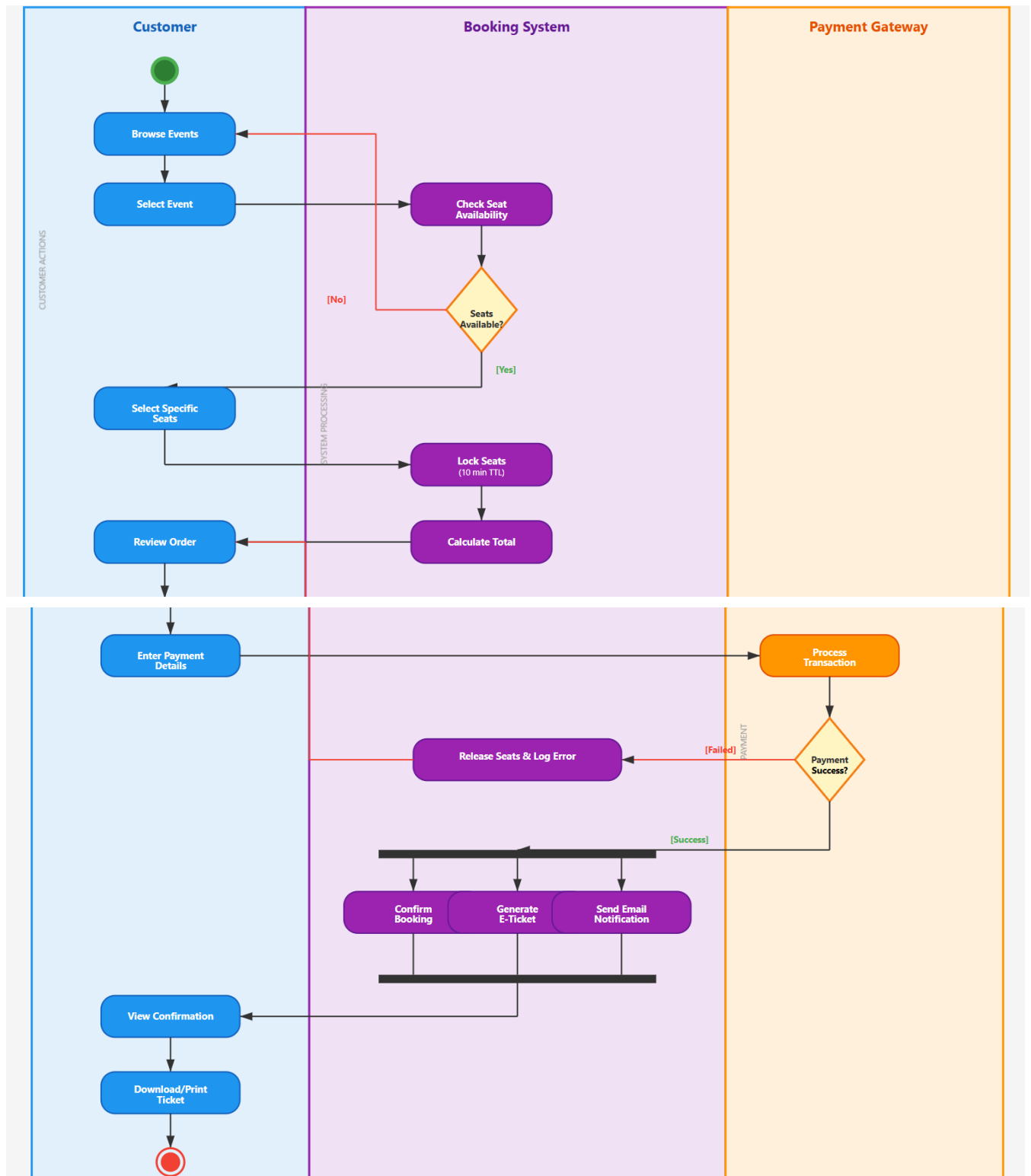


or



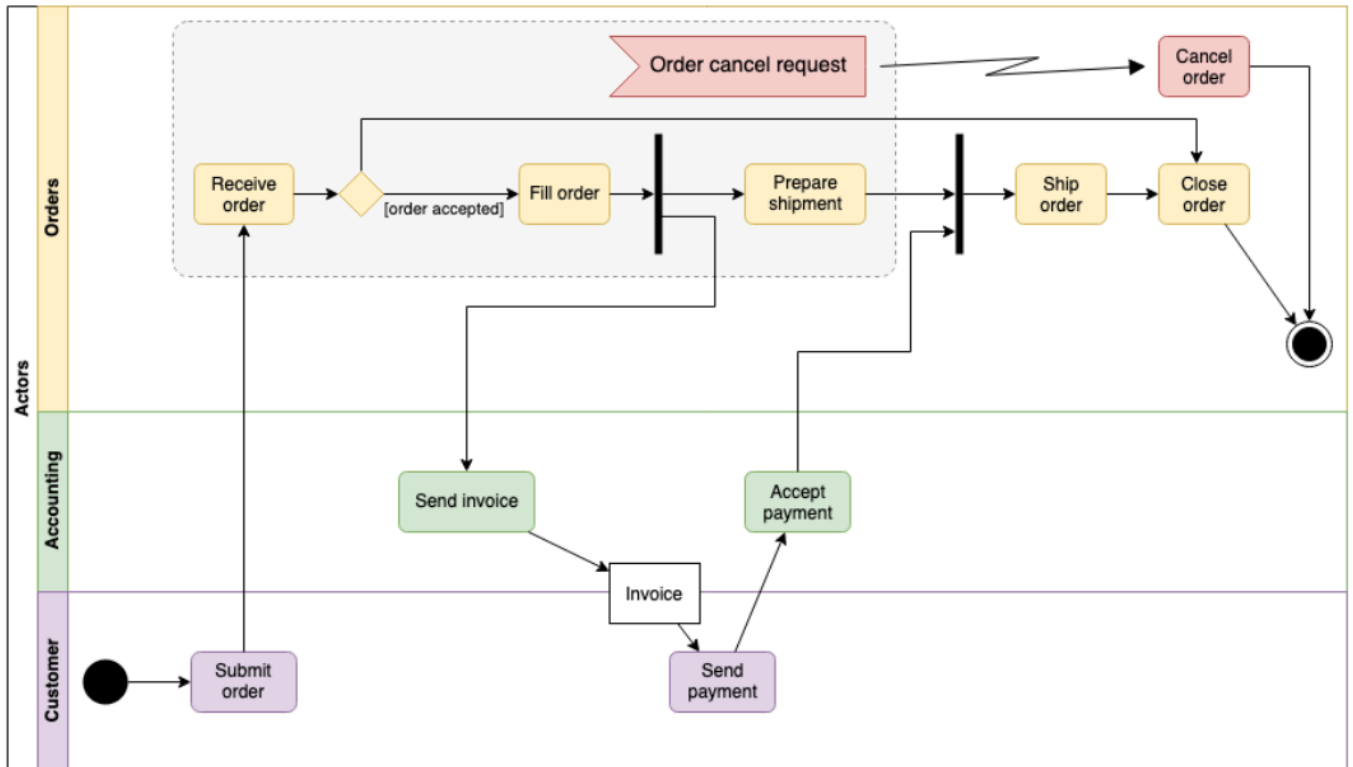
Event Ticket Booking System

UML Activity Diagram with Swimlanes

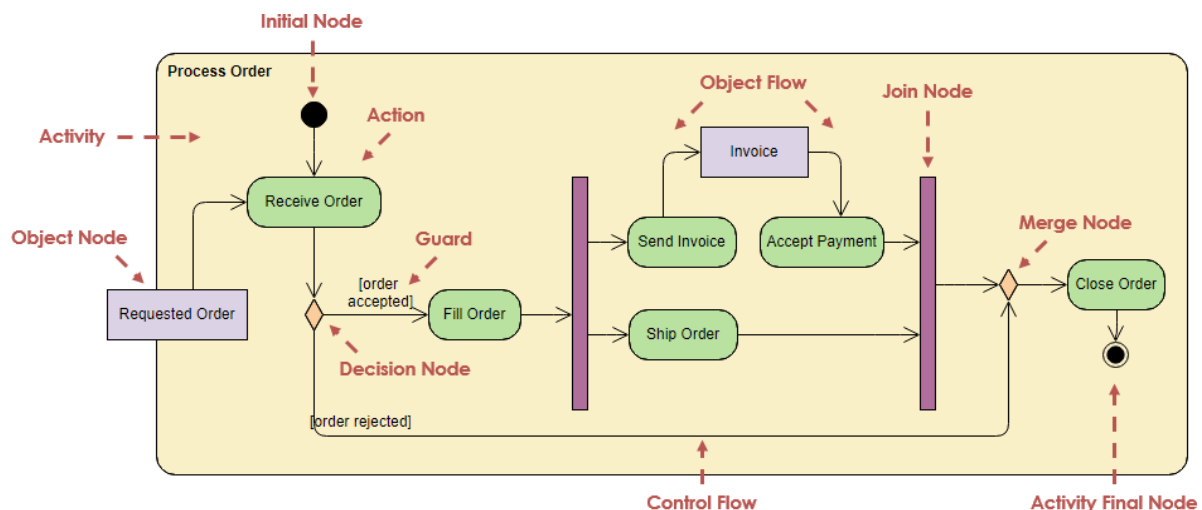


Key Features of This Diagram:

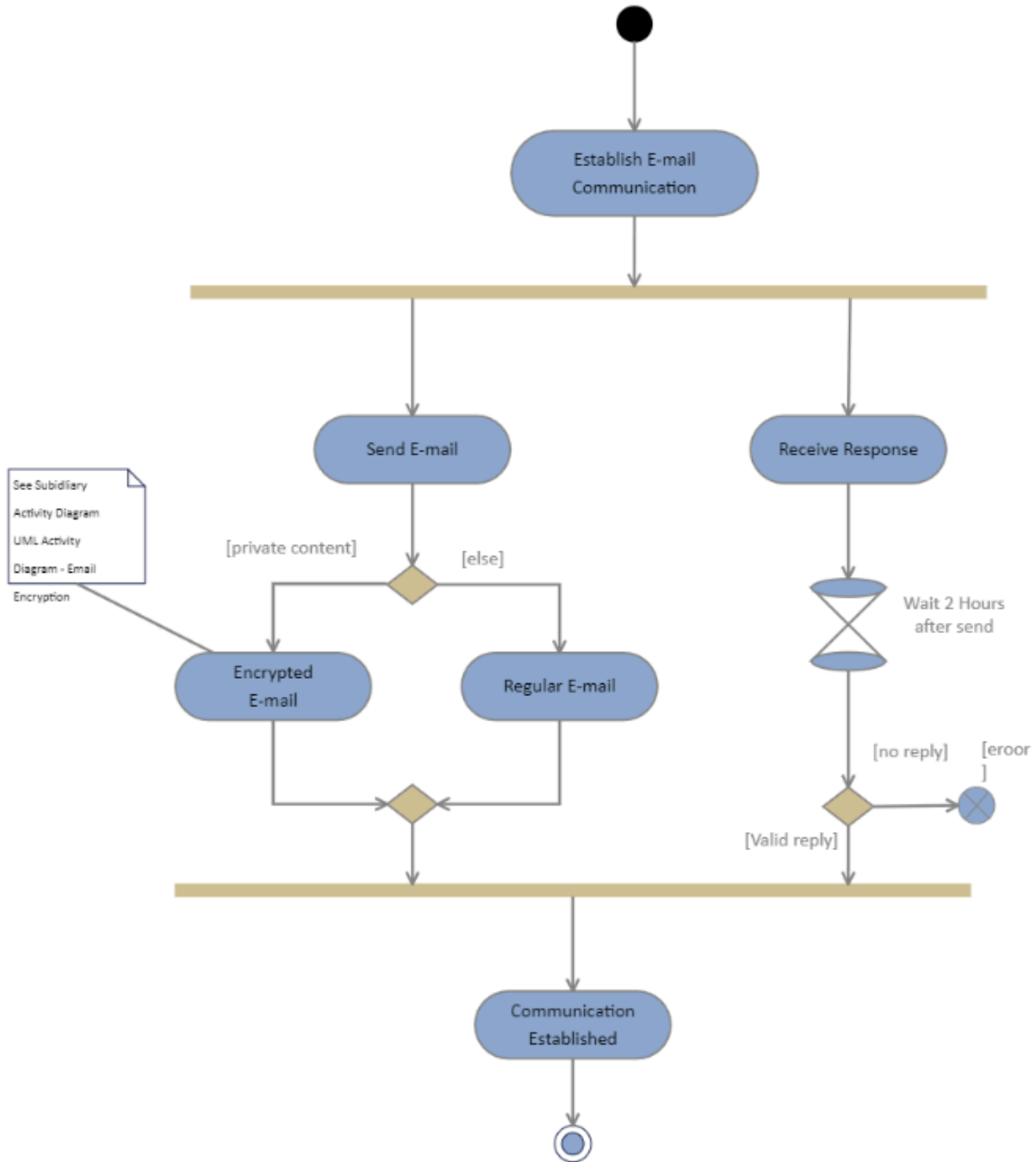
- **Three Swimlanes:** Clear separation of Customer, Booking System, and Payment Gateway responsibilities
- **Two Decision Points:** Seat availability check and payment success verification
- **Concurrency (Fork/Join):** After payment success, three activities run in parallel: booking confirmation, e-ticket generation, and email notification
- **Error Handling:** Two error paths - no seats available (returns to browse) and payment failure (releases seats and returns to review)
- **Seat Locking Mechanism:** Temporary 10-minute lock prevents double-booking
- **Complete Flow:** From start (filled circle) to end (bull's eye target)
- **Proper UML Notation:** Rounded rectangles for activities, diamonds for decisions, bars for fork/join



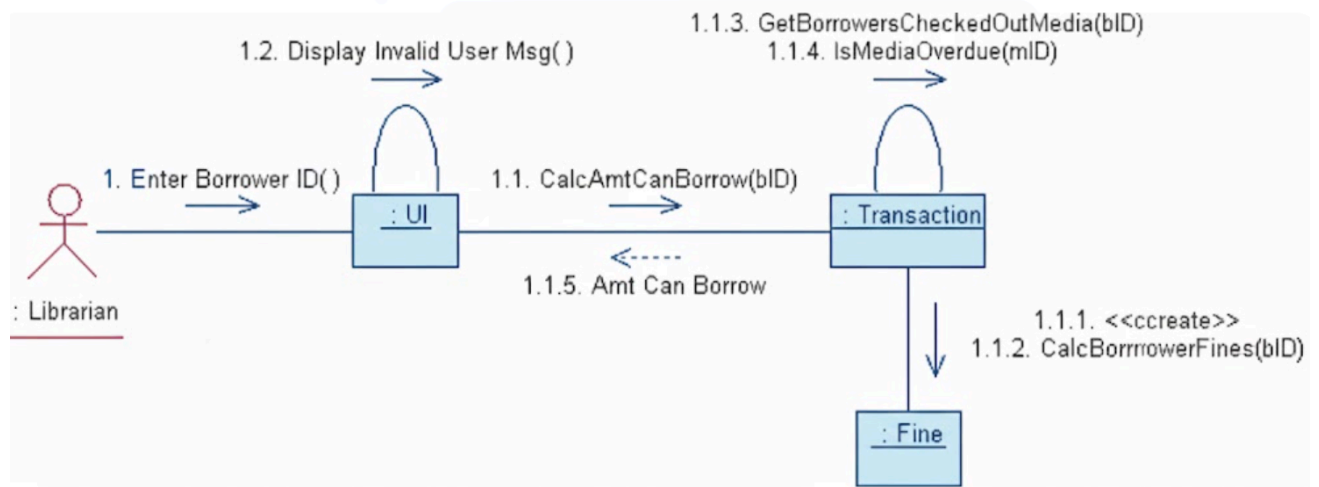
Activity diagrams can be partitioned into swimlanes. This one has an interruptible activity region (solid grey background with a dashed outline).



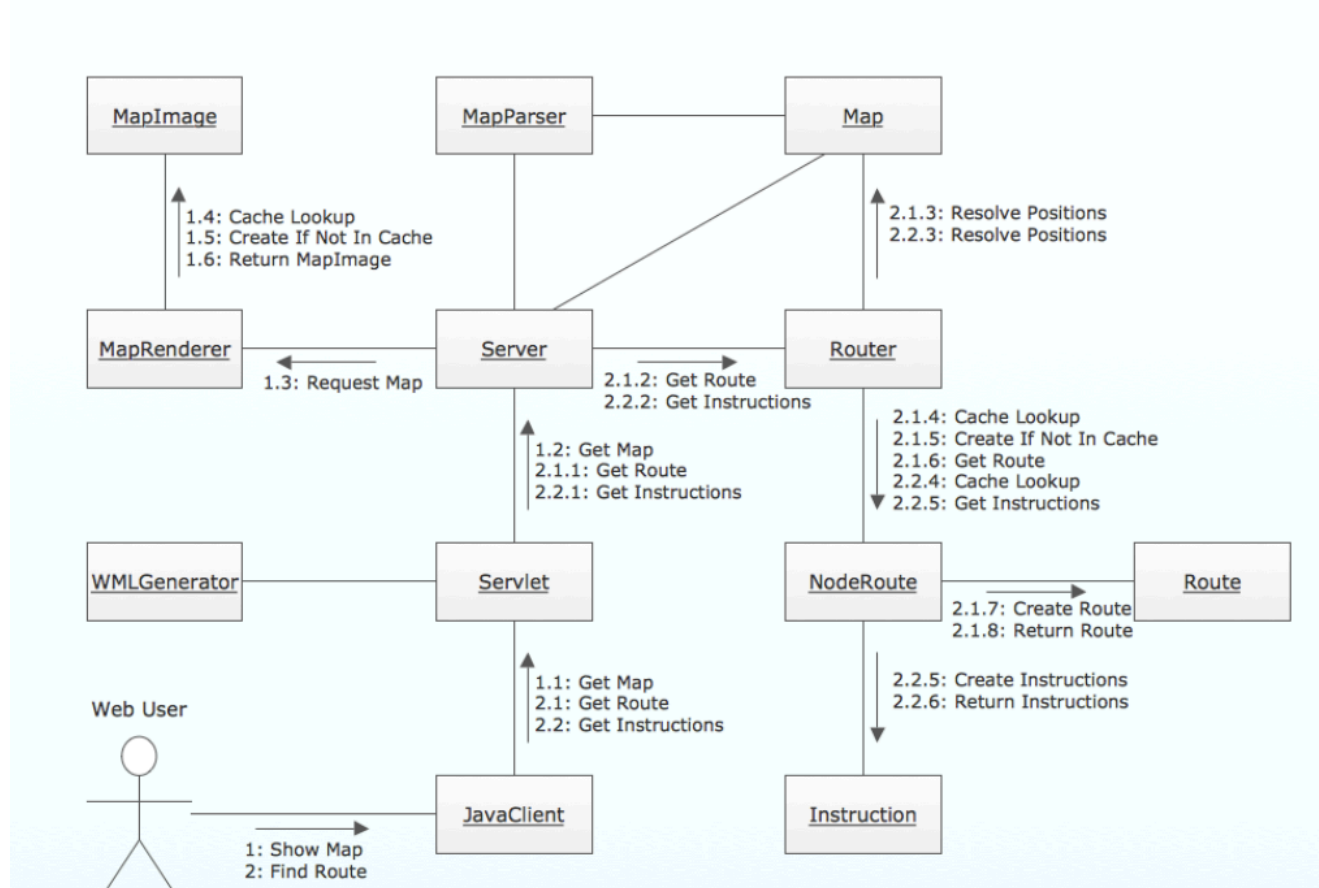
EMAIL CONNECTION:



COLLABORATION/COMMUNICATION DIAGRAMS:



Client Server Access



Online Order: Collaboration Diagram

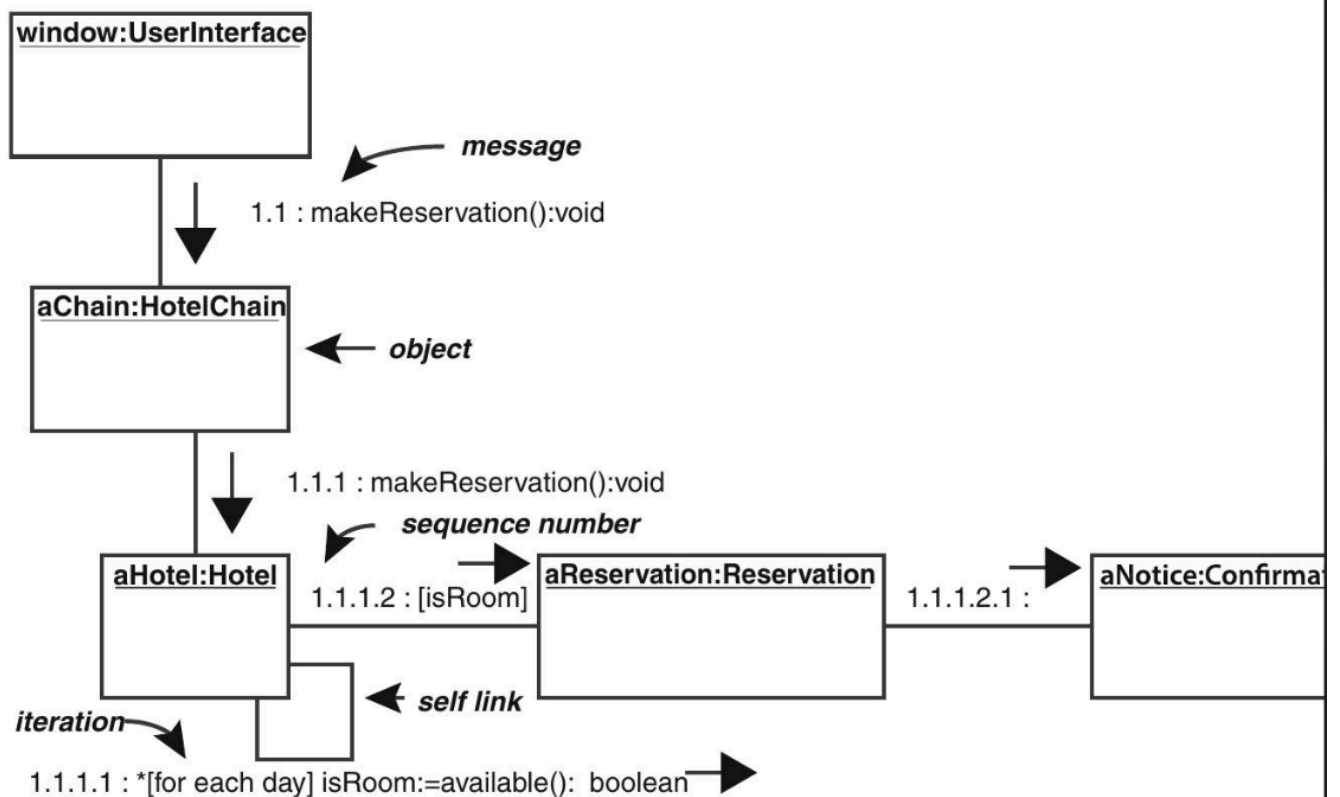
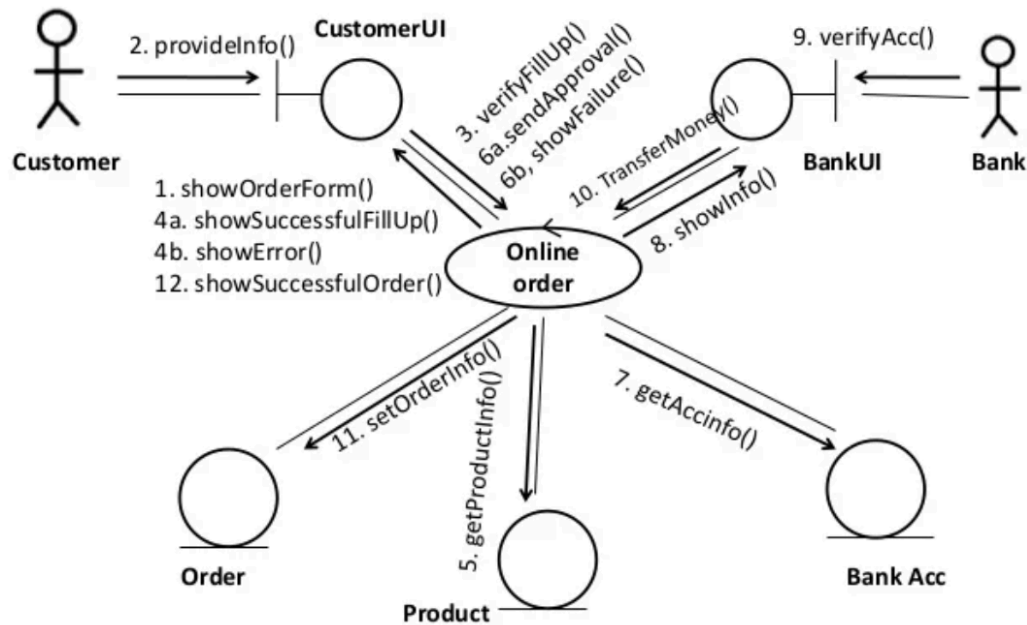


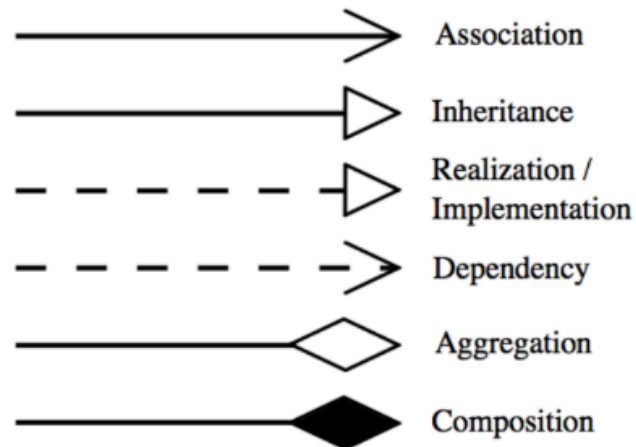
Fig. 3.18 Collaboration diagram (Miller 2003)

CLASS DIAGRAMS:

RELATIONSHIPS

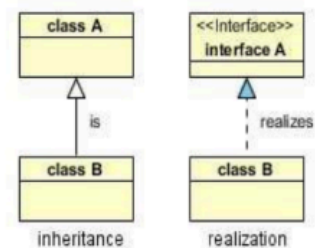
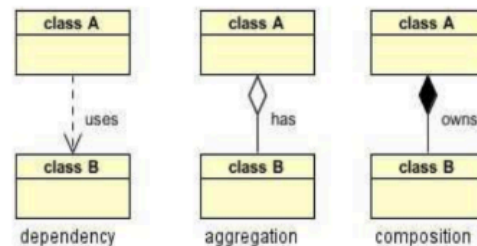
In UML, object interconnections (logical or physical), are modeled as relationships.

There are six kinds of relationships in UML:



RELATIONSHIPS IN A NUTSHELL

- Dependency : class A uses class B
- Aggregation : class A has a class B
- Composition : class A owns a class B
- Inheritance : class B is a Class A (or class A is extended by class B)
- Realization : class B realizes Class A (or class A is realized by class B)



Explanation of Online Shopping UML Class Diagram

This UML **class diagram** represents an **Online Shopping System**. The system is divided into **packages** to improve modularity and clarity. It includes **classes, interfaces, enumerations, relationships, and exception handling**, which together describe the static structure of the system.

1 Packages (Modular Organization)

Packages group related classes and help manage complexity in large systems.

UserManagement Package

Handles user-related functionality.

- **Customer**
 - Attributes: **customerId**, **name**, **email**
 - Represents a person who interacts with the shopping system.
- **WebUser**
 - Attributes: **username**, **password**, **status**
 - Represents a registered online user.
- **UserStatus (Enumeration)**
 - Defines possible states of a web user:
 - **NEW**, **ACTIVE**, **BLOCKED**, **BANNED**
- **Relationship**
 - A **Customer** may register as **zero or one WebUser**.
 - This supports customers who order via phone or catalog without web registration.

OrderManagement Package

Manages customer orders.

- **Order**
 - Attributes: **orderId**, **status**, **totalAmount**
 - Represents a purchase made by a customer.
- **LineItem**
 - Attribute: **quantity**
 - Represents individual items within an order.
- **OrderStatus (Enumeration)**
 - Defines lifecycle states of an order:
 - **NEW**, **HOLD**, **SHIPPED**, **DELIVERED**, **CLOSED**
- **Composition Relationship**

- An **Order** contains one or more **LineItems**.
- Composition indicates strong ownership (line items cannot exist without an order).

ProductCatalog Package

Handles product-related information.

- **Product**
 - Attributes: **productId**, **name**, **price**, **stock**
 - Represents items available for sale.
- **Association**
 - Each **LineItem** refers to **exactly one Product**.
 - A **Product** may be linked to multiple line items or none.

PaymentProcessing Package

Manages payment functionality.

- **PaymentService (Interface)**
 - Declares the operation **processPayment()**
 - Defines a contract for all payment methods.
- **CreditCardPayment** and **PayPalPayment**
 - Implement the **PaymentService** interface.
 - Allow multiple payment methods using **polymorphism**.
- **Usage Relationship**
 - **Order** uses **PaymentService** to complete payments.

Exceptions Package

Models abnormal or error conditions in the system.

- **PaymentException**
 - Thrown when a payment fails.
- **InsufficientStockException**
 - Thrown when ordered quantity exceeds available stock.
- **UnauthorizedUserException**
 - Thrown when an unauthorized or blocked user attempts an action.
- **Inheritance**
 - All exceptions inherit from the base **Exception** class.

- **Exception Usage**
 - **Order** may throw **InsufficientStockException**
 - **PaymentService** may throw **PaymentException**
 - **WebUser** may throw **UnauthorizedUserException**

2 Enumerations (Enums): Enumerations restrict values to a fixed set, improving data integrity.

✓ **UserStatus**

Defines valid user states.

✓ **OrderStatus**

Defines valid order lifecycle stages.

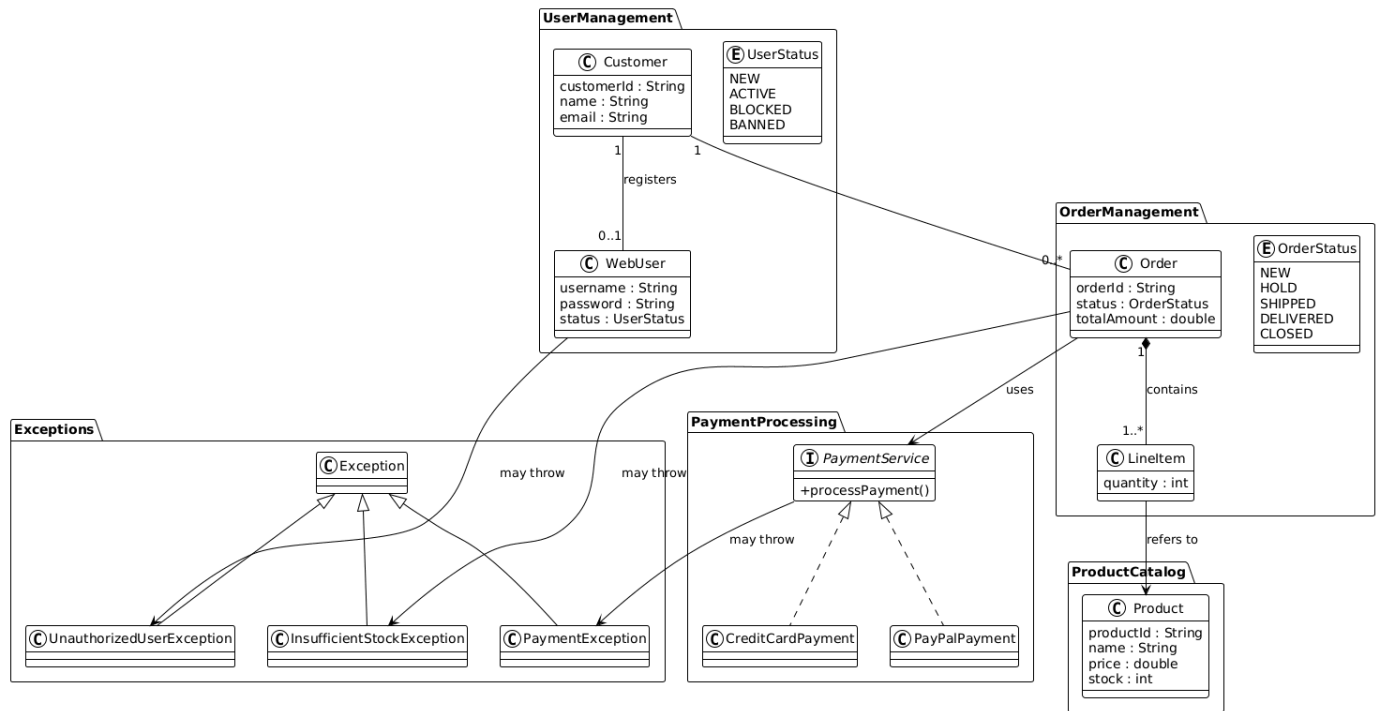
Enums prevent invalid states and make the system more robust.

3 Relationships Summary

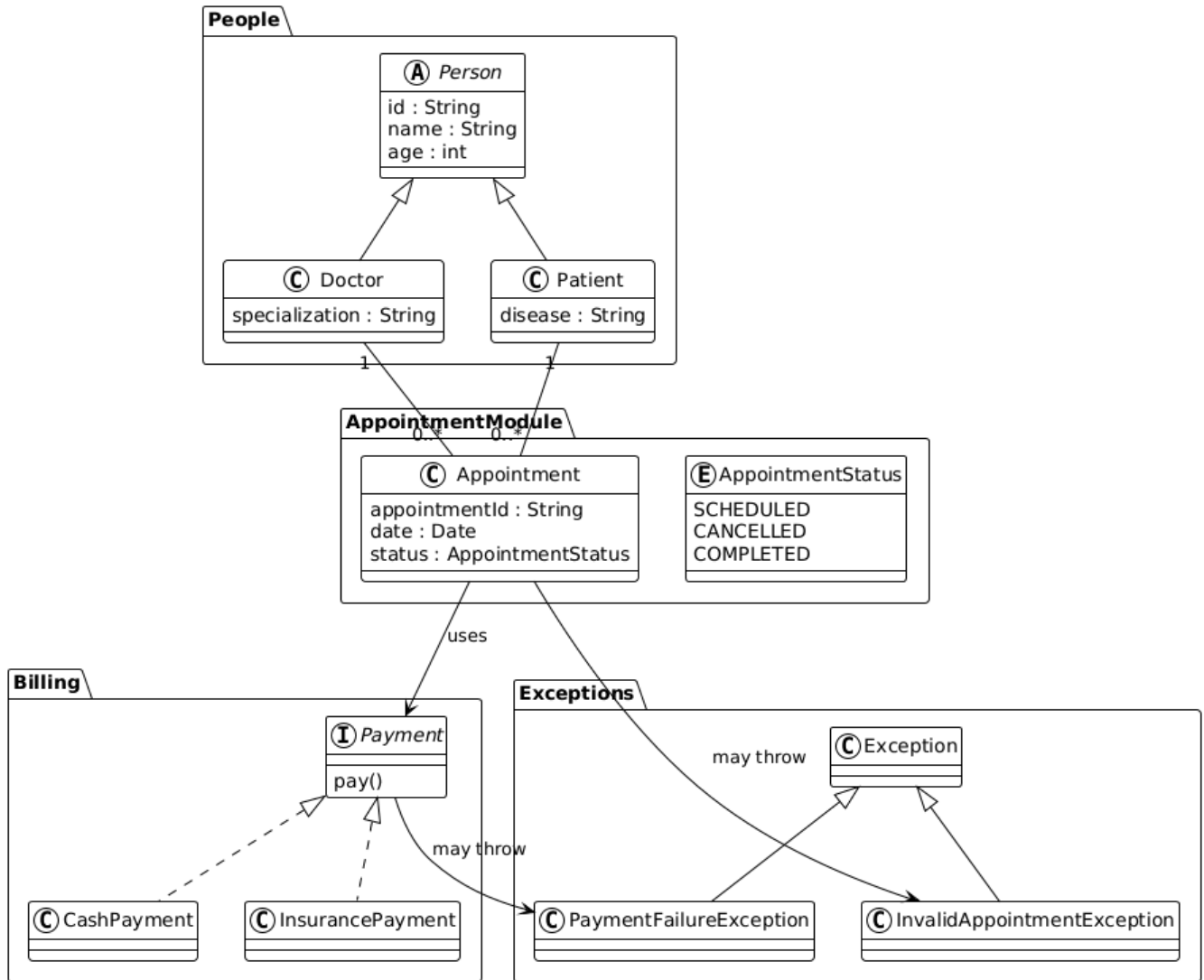
Relationship Type	Description
Association	Customer places Orders
Optional Association	Customer registers as WebUser
Composition	Order contains LineItems
Interface Realization	Payment classes implement PaymentService
Dependency	Order depends on PaymentService
Exception Dependency	Classes may throw exceptions

4 Key OOAD Concepts Used

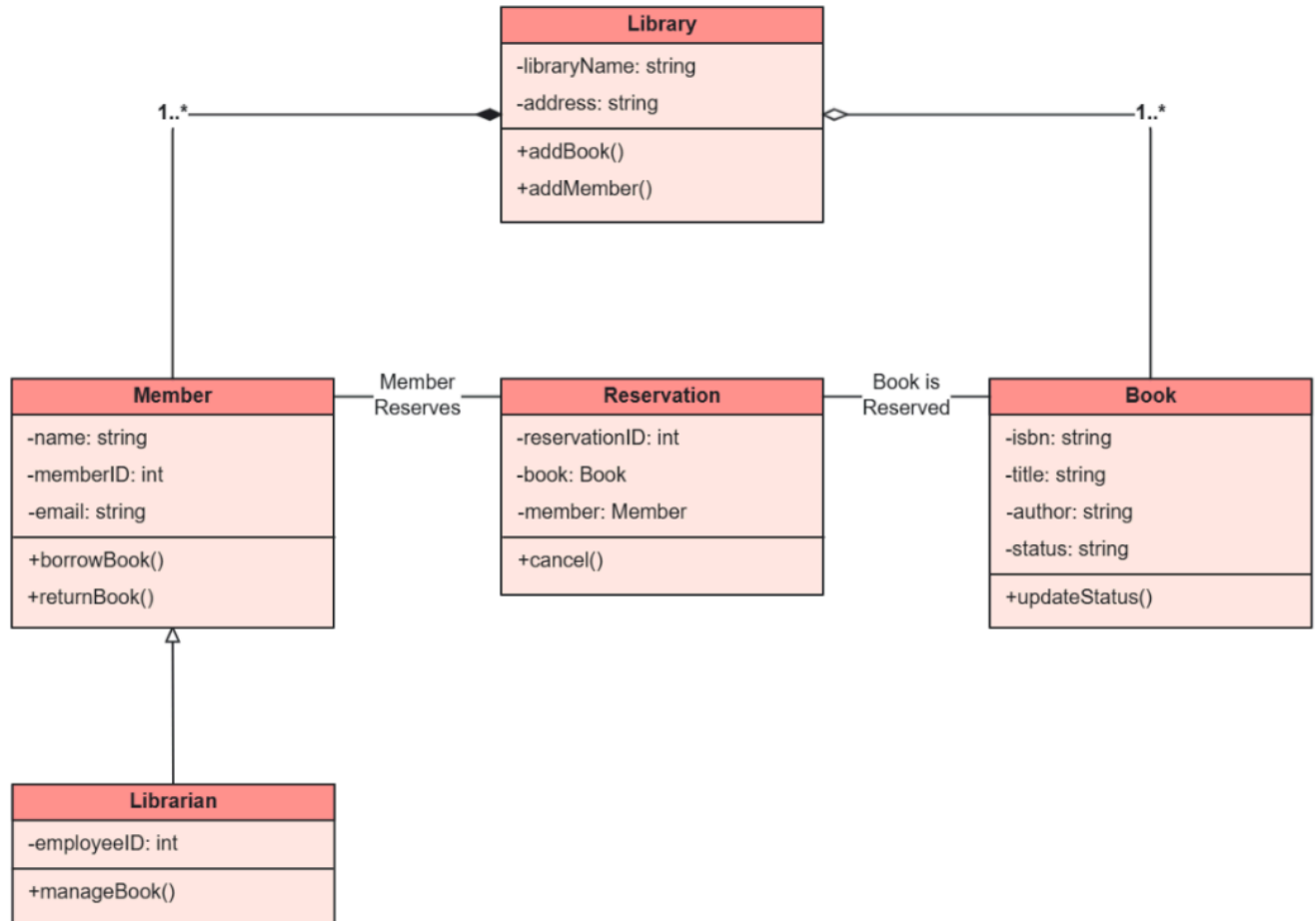
- **Abstraction** → Interface **PaymentService**
- **Encapsulation** → Attributes grouped inside classes
- **Inheritance** → Exception hierarchy
- **Polymorphism** → Multiple payment methods
- **Modularity** → Packages
- **Robustness** → Exception handling

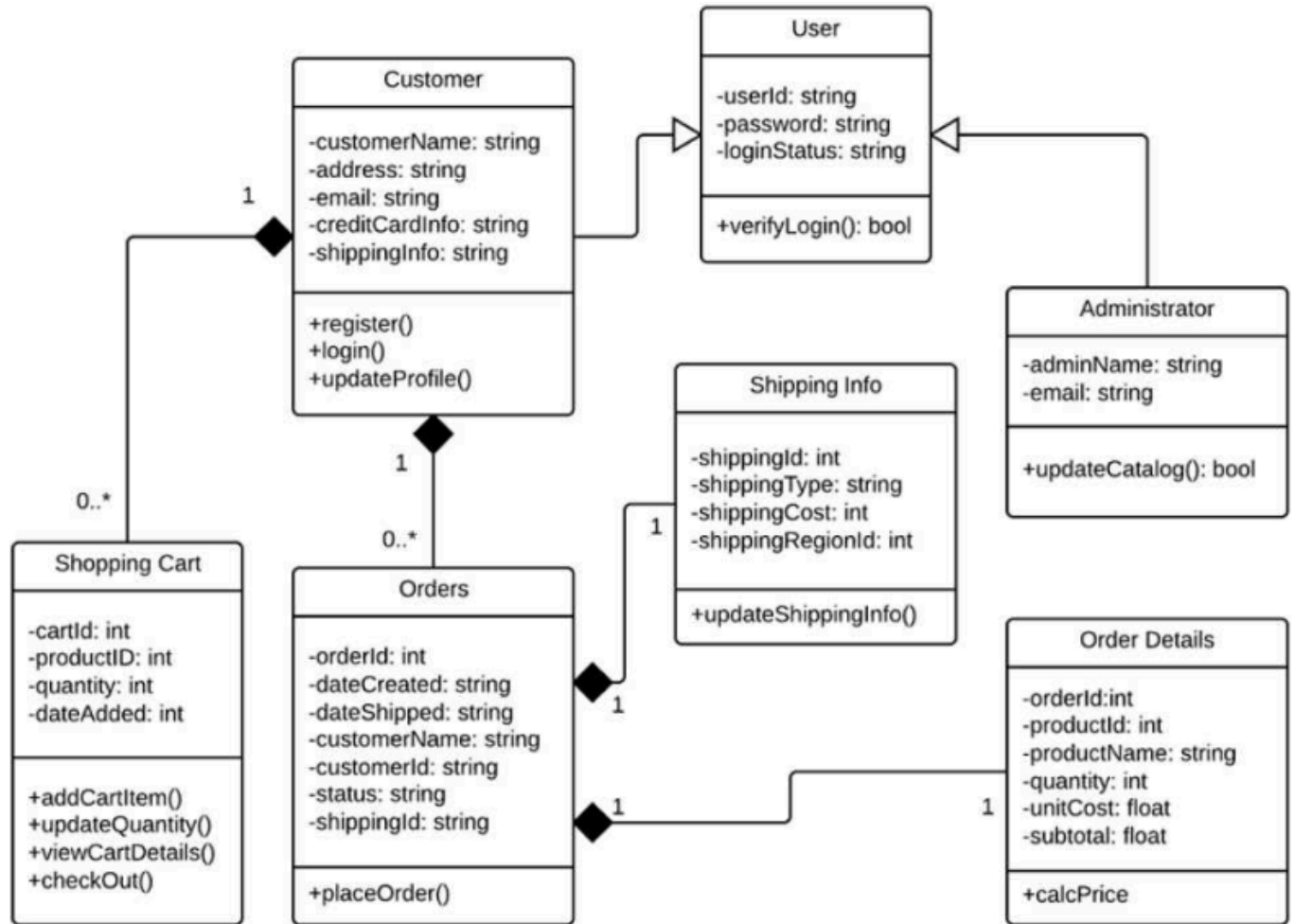


HOSPITAL MANAGEMENT SYSTEM:



UML Class Diagram Template





Domain Modeling:

Key Points

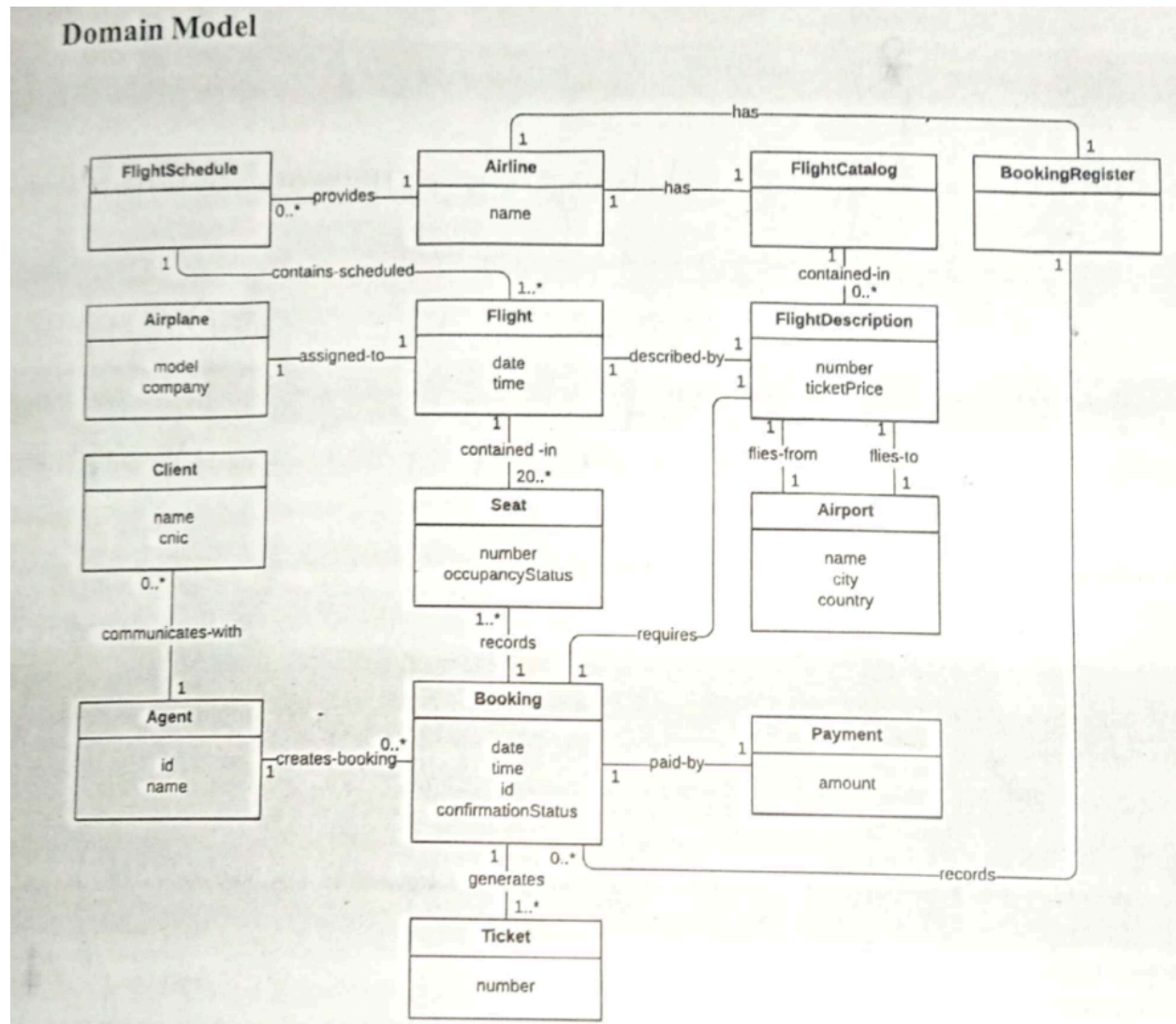
- Focuses on the problem domain, not the technical solution
- Captures real-world entities, attributes, and relationships
- Acts as a bridge between requirements and design
- Usually represented using UML class diagrams

What Does a Domain Model Contain?

- Domain classes (real-world objects)
 - e.g., **Customer**, **Order**, **Product**
- Attributes
 - e.g., **customerId**, **price**, **quantity**
- Associations
 - e.g., Customer *places* Order
- No implementation details
 - **✗** No methods
 - **✗** No database tables
 - **✗** No UI elements

Domain Model	Design Model
Problem-focused	Solution-focused
Real-world concepts	Software classes
No methods	Contains methods
Technology independent	Technology dependent

For flight booking system:



DESIGN PATTERN:

FACTORY DESIGN PATTERN:

Type: Creational

Description:

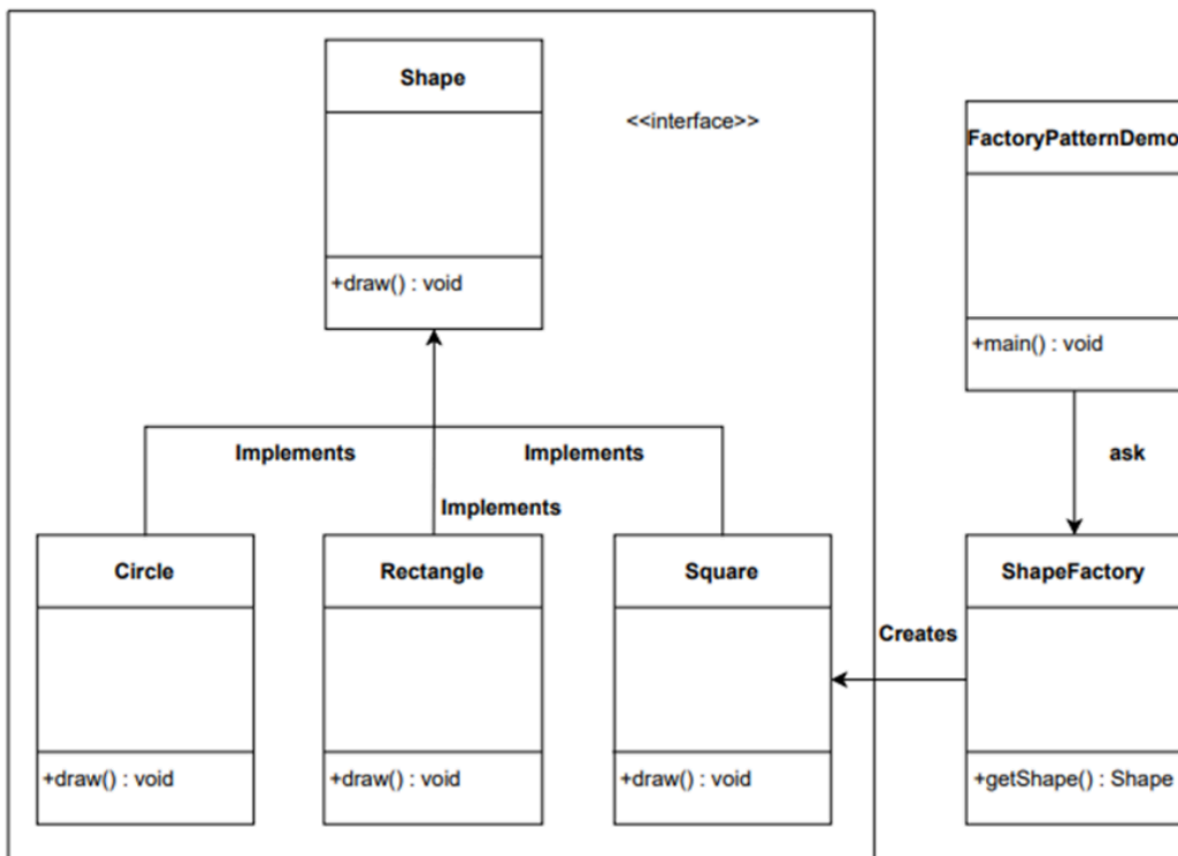
Factory Pattern provides an interface for creating objects but allows subclasses to decide which class to instantiate. It hides object creation logic and promotes loose coupling.

Key Points:

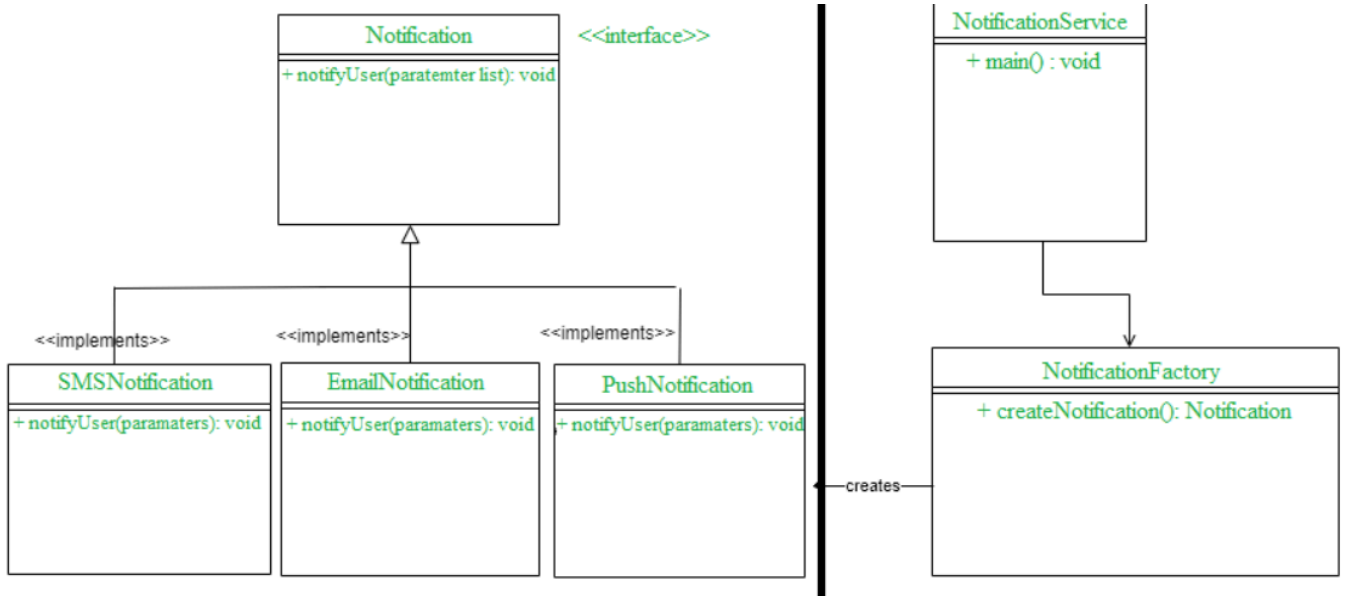
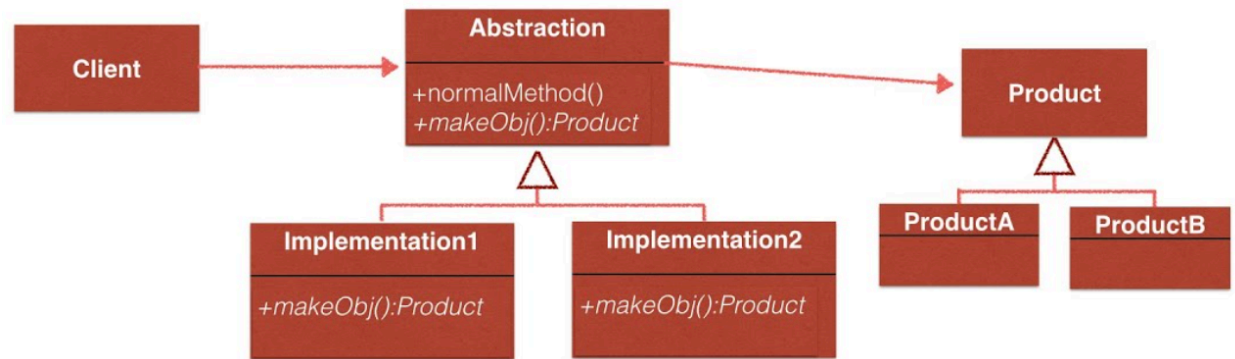
- Object creation is centralized
- Client does not know concrete class
- Improves scalability and maintainability

Example:

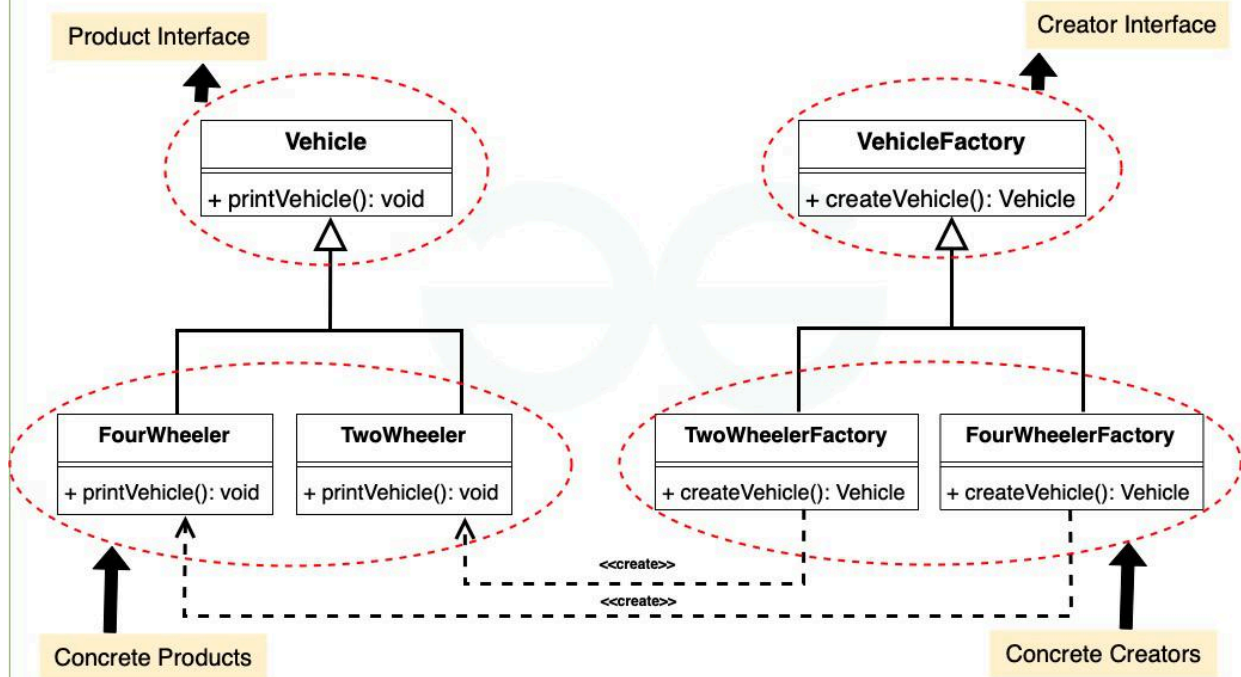
Creating different types of notifications (Email, SMS, Push) using a factory.



Define an **interface** for creating an object, but let **subclasses** decide which object to **instantiate**. Factory Method lets a class defer instantiation to subclasses.



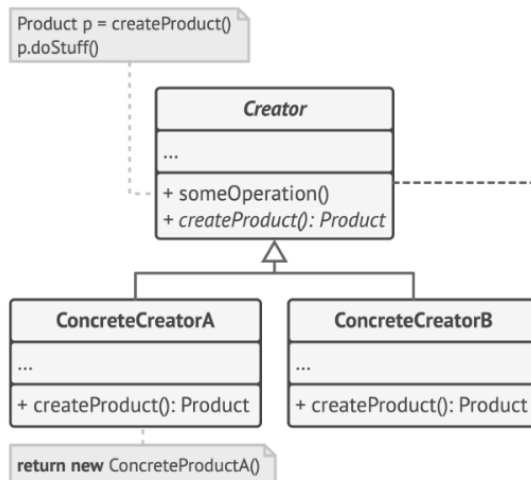
Factory Method Design Pattern Class Diagram



3 The **Creator** class declares the factory method that returns new product objects. It's important that the return type of this method matches the product interface.

You can declare the factory method as `abstract` to force all subclasses to implement their own versions of the method. As an alternative, the base factory method can return some default product type.

Note, despite its name, product creation is **not** the primary responsibility of the creator. Usually, the creator class already has some core business logic related to products. The factory method helps to decouple this logic from the concrete product classes. Here is an analogy: a large software development company can have a training department for programmers. However, the primary function of the company as a whole is still writing code, not producing programmers.



1 The **Product** declares the interface, which is common to all objects that can be produced by the creator and its subclasses.

2 **Concrete Products** are different implementations of the product interface.

4 **Concrete Creators** override the base factory method so it returns a different type of product.

Note that the factory method doesn't have to **create** new instances all the time. It can also return existing objects from a cache, an object pool, or another source.

ADAPTER DESIGN PATTERN:

Type: Structural

Description:

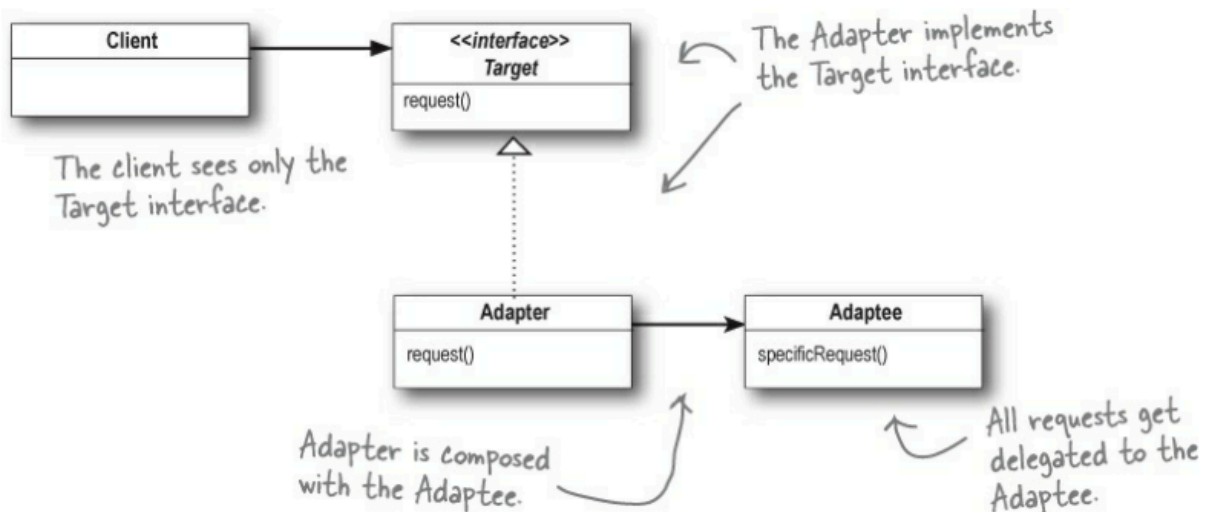
Adapter Pattern allows incompatible interfaces to work together by converting one interface into another expected by the client.

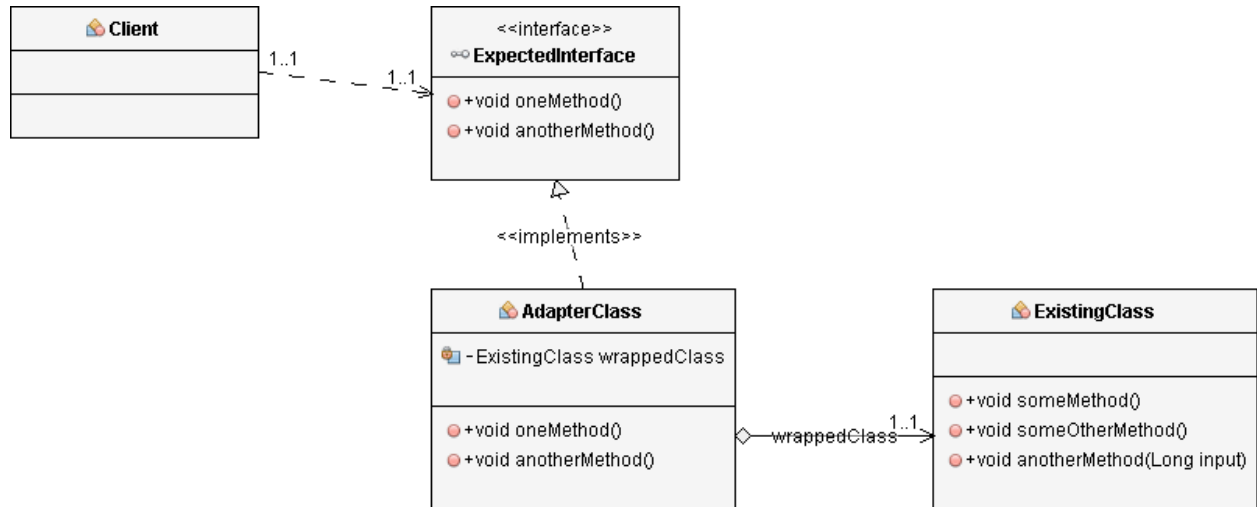
Key Points:

- Acts as a bridge between incompatible classes
- Does not modify existing code
- Useful for legacy systems

Example:

Using a card reader to connect a memory card to a laptop.





3) Singleton Design Pattern

Type: Creational

Description:

Singleton Pattern ensures that a class has only one instance and provides a global point of access to it.

Key Points:

- Only one object instance exists
- Controlled access to shared resource
- Commonly used for logging and configuration

Example:

Database connection manager.

4) Facade Design Pattern

Type: Structural

Description:

Facade Pattern provides a simplified interface to a complex subsystem, making it easier to use.

Key Points:

- Hides system complexity
- Reduces dependencies
- Improves usability

Example:

One button to start a computer (internally handles CPU, memory, disk).

5 Observer Design Pattern

Type: Behavioral

Description:

Observer Pattern defines a one-to-many dependency so that when one object changes state, all its dependents are notified automatically.

Key Points:

- Loose coupling between subject and observers
- Automatic notification
- Supports event-driven systems

Example:

Subscribers receiving updates from a YouTube channel.

6 Strategy Design Pattern

Type: Behavioral

Description:

Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable at runtime.

Key Points:

- Behavior can change dynamically
- Eliminates conditional statements
- Promotes open/closed principle

Example:

Different payment methods (Credit Card, PayPal, Cash).

MCOs – Design Patterns

Q1. Which design pattern ensures only one instance of a class?

- A) Factory
- B) Adapter

- C) Singleton
- D) Observer

Answer: C

Q2. Which pattern hides object creation logic?

- A) Adapter
- B) Factory
- C) Observer
- D) Facade

Answer: B

Q3. Adapter pattern is mainly used to:

- A) Create objects
- B) Simplify subsystem
- C) Make incompatible interfaces work together
- D) Notify objects

Answer: C

Q4. Facade pattern is used to:

- A) Reduce object creation
- B) Provide simplified interface
- C) Notify observers
- D) Select algorithms

Answer: B

Q5. Observer pattern supports:

- A) One-to-one relationship
- B) One-to-many relationship
- C) Many-to-many relationship
- D) One-to-zero relationship

Answer: B

Q6. Strategy pattern helps in:

- A) Object creation
- B) Interface conversion
- C) Runtime algorithm selection
- D) System simplification

Answer: C

Q7. Which pattern is behavioral?

- A) Factory
- B) Adapter
- C) Singleton
- D) Observer

Answer: D

Q8. Which pattern is best for changing behavior without modifying client code?

- A) Facade
- B) Strategy
- C) Factory
- D) Singleton

Answer: B

Q9. Logging system is a common example of:

- A) Adapter
- B) Factory
- C) Singleton
- D) Observer

Answer: C

Q10. Which pattern reduces dependency between client and complex subsystem?

- A) Adapter
- B) Observer
- C) Facade
- D) Strategy

Answer: C

